



**Hochschule
Bonn-Rhein-Sieg**
University of Applied Sciences

Fachbereich Informatik
Department of Computer Science

Bachelorarbeit

Studiengang Bachelor Informatik
Degree programme Bachelor Computer Science

Entwicklung und Evaluation eines Edge-AI-Systems zur Echtzeitgebärdenerkennung auf ressourcenbeschränkten Embedded Systemen

...

Erstprüfer Prof. Dr. Thomas Breuer
Zweitprüfer Martin Nimbach

Vorgelegt am 27. Juli 2026

Von	Fabian Weber	Oliver Groß
Matrikelnummer	9047415	9050158
Adresse	Anne-Frank-Str. 34 53844 Troisdorf	Hartwig-Hüser Str. 8 53227 Bonn
E-Mail	fabian.weber@smail.inf.h-brs.de	oliver.gross@smail.inf.h-brs.de

Erklärung

Hiermit erkläre ich an Eides statt, dass ich die vorliegende Arbeit selbst angefertigt habe; die aus fremden Quellen direkt oder indirekt übernommenen Gedanken sind als solche kenntlich gemacht.

Die Arbeit wurde bisher keiner Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

.....
Unterschrift

.....
Ort, Datum

Inhaltsverzeichnis

Abbildungsverzeichnis	iv
Tabellenverzeichnis	v
Abkürzungsverzeichnis	vi
1. Einleitung	1
1.1. Problemstellung	1
1.2. Motivation und Zielsetzung	2
1.3. Abgrenzung des Untersuchungsgegenstandes	2
1.3.1. Inhaltliche Abgrenzung	2
1.3.2. Personenspezifische Abgrenzung	2
1.4. Aufbau der Arbeit	3
2. Theoretische und technische Grundlagen	4
2.1. Eingebettete Systeme & STM32N6570DK	4
2.1.1. Begriffsbildung und Merkmale	4
2.1.2. Mikrocontroller Architektur	4
2.1.3. Peripherieschnittstellen	5
2.1.4. Speicherarchitektur	6
2.1.5. Clock-Systeme	6
2.1.6. Bare-Metal-Programmierung	6
2.1.7. Hardware-Beschleunigung	7
2.2. Deutsches Einhand-Fingeralphabet	7
2.2.1. Gebärdensprache als eigenständige Sprache	7
2.2.2. Das Fingeralphabet	7
2.2.3. Das deutsche Einhand-Fingeralphabet	8
2.2.4. Handpositionen und Fingerkonfigurationen	8
2.3. Grundlagen des maschinellen Lernens	8
2.3.1. Begriffsbildung	9
2.3.2. Supervised Learning	9
2.3.3. Klassifikation	9
2.3.4. Metriken	9
2.3.5. Overfitting und Gegenmaßnahmen	10
2.3.6. Parameter vs Hyperparameter	10
2.4. Neuronale Netze zur Klassifikation	10
2.4.1. Künstliche Neuronale Netze	10
2.4.2. Aktivierungsfunktionen	12
2.4.3. Architektur: Multi-Layer Perceptron (MLP)	12
2.4.4. Architektur: Convolutional Neural Networks (CNN)	12
2.4.5. Backpropagation und Gradient Descent	12
2.5. Handdetektion und Handlandmarks	12
2.5.1. Zweistufige Erkennungspipeline	12
2.5.2. Palm Detection	13
2.5.3. Handlandmarks	13
2.5.4. ROI	14
2.6. Datenaugmentation und Modellquantisierung	15
2.6.1. Datenaugmentation	15
2.6.2. Modellquantisierung	16
2.7. Edge AI und Neuronale Beschleuniger (NPU)	16
2.7.1. Edge AI	16
2.7.2. Neuronale Beschleuniger (NPU)	17
2.7.3. ST Edge Core CLI	17

3. Stand der Technik	19
3.1. Bestehende Ansätze zur Erkennung von Handzeichen	19
3.2. Kamerabasierte Fingeralphabet- und Gebärdenerkennung	19
3.3. Aktuelle Embedded-AI-Lösungen	19
3.4. Einordnung der Arbeit und Abgrenzung der eigenen Arbeit	20
4. Methodik und Entwicklungsprozess	21
4.1. Vorgehensweise bei der Literatur- und Technologierecherche	21
4.2. Iterativer Entwicklungs- und Integrationsprozess	22
4.3. Vorgehen bei der Modellentwicklung	22
4.4. Verifikation- und Evaluationskonzept	22
5. Anforderungsanalyse	24
5.1. Anwendungsszenario	24
5.2. Anforderungen & Randbedingungen	25
5.2.1. Funktionale Anforderungen	25
5.2.2. Nichtfunktionale Anforderungen	26
5.2.3. Technische Anforderungen	26
5.2.4. Randbedingungen	27
5.3. Abnahmekriterien	27
6. Systementwurf	29
6.1. Systemkonzept	29
6.2. Hardwarekonzept	30
6.3. KI-Verarbeitungskette	30
6.4. Speicher- und Datenflusskonzept	31
6.5. Ablauf vom Kamerabild bis zum Erkennungsergebnis	33
7. Entwicklung und Implementation	35
7.1. Aufbau und Aufbereitung des Datensatzes	35
7.1.1. Rohvideos	35
7.1.2. Frame-Extraktion und Handdetektion	35
7.1.3. Augmentation	35
7.1.4. Normalisierung und Interpolation	35
7.1.5. Datenstrukturen und Speicherung	36
7.2. Entwicklung und Training des Klassifikationsmodells	36
7.2.1. Problemdefinition	36
7.2.2. Datenanalyse	36
7.2.3. Modellauswahl	37
7.2.4. Modellarchitektur	37
7.2.5. Verlustfunktion und Optimierung	39
7.2.6. Trainingsprozess	39
7.2.7. Evaluierung	40
7.3. Quantisierung, Konvertierung und Deployment der Modelle	40
7.3.1. Quantisierung von Tensorflow Modellen	40
7.3.2. Konvertierung INT8-Quantisierte-Modelle zu Embedded binaries	41
7.4. Softwaregrundstruktur und hardwarenahe Basistreiber	44
7.4.1. Aufbau der Embedded-Software	44
7.4.2. Zentrale Konfiguration und Systeminitialisierung	45
7.4.3. Registerbasierte Treiberentwicklung	47
7.4.4. Grundlegende Systemtreiber	47
7.4.5. Einordnung der Basistreiber in die übergeordneten Komponenten	48
7.5. Kamera - Display Pipeline	49
7.5.1. Initialisierung des Kamerasensors	49
7.5.2. Übertragung der Kameradaten über CSI und DCMIPP	49
7.5.3. Verwaltung der Bildpuffer	49
7.5.4. LTDC-Konfiguration und Verwaltung der Displayebenen	50
7.6. Ablaufsteuerung und Software Scheduler	50
7.6.1. Architektur-Übersicht	50

7.6.2. Task-Lebenszyklus (State machine)	51
7.6.3. Interrupt-Hierrarchie & Context Switching	52
7.6.4. Task-Selektion	53
7.6.5. Fault handling	54
7.6.6. CPU-Zyklen-Messung & Statistik	54
7.7. Integration der NPU und einheitliche KI-Schnittstelle	55
7.7.1. Initialisierung des Neural-ART Accelerators	55
7.7.2. Einheitliche KI-Schnittstelle	56
7.8. Vor- und Nachverarbeitungsschritte	57
7.8.1. DCMIPP-gestützte Bildvorverarbeitung	57
7.8.2. Nachverarbeitung der Handdetektion	58
7.8.3. Vorverarbeitung der Landmark-Erkennung	60
7.8.4. Nachverarbeitung der Landmark-Erkennung	61
7.8.5. Vorverarbeitung des Klassifizierungsmodells	62
7.8.6. Nachverarbeitung des Klassifikationsmodells	62
7.9. Visualisierung und Benutzerausgabe	63
7.9.1. Darstellung der Erkennungsergebnisse	63
7.9.2. Benutzeroberfläche und Bedienung	64
8. Evaluation und Validierung	67
8.1. Erkennungsgenauigkeit	67
8.2. Konfusionsmatrix und klassenbezogene Ergebnisse	67
8.3. Versuchsaufbau und Testbedingungen	68
8.4. Feldversuch	69
8.5. Einfluss von Beleuchtung und Hintergrund	69
8.6. Inferenzlatenz und Ende-zu-Ende-Latenz	69
8.7. Bildrate und Datendurchsatz	69
8.8. Speicher- und Ressourcenverbrauch	69
8.9. Abgleich mit den definierten Anforderungen	69
9. Diskussion	70
9.1. Versuchsaufbau und Testbedingungen	70
9.2. Erkennungsgenauigkeit	70
9.3. Konfusionsmatrix und klassenbezogene Ergebnisse	70
9.4. Einfluss von Beleuchtung und Hintergrund	70
9.5. Inferenzlatenz und Ende-zu-Ende-Latenz	70
9.6. Bildrate und Datendurchsatz	70
9.7. Speicher- und Ressourcenverbrauch	70
9.8. Abgleich mit den definierten Anforderungen	70
10. Fazit und Ausblick	71
10.1. Zusammenfassung und Zielerreichung	71
10.2. Ausblick	71
A. Anhang	72

Abbildungsverzeichnis

2.1. STM32N6570-DK - Architektur	5
2.2. Deutsches Einhand-Fingeralphabet [1]	8
2.3. Mathematisch modelliertes Neuron (Gurney, 2018) — Eigene Darstellung	11
2.4. Allgemeine Struktur Künstlicher Neuronaler Netze (Kim, 2016) — Eigene Darstellung	11
2.5. Beispiel Hand Landmark Punkte — Eigene Darstellung	14
2.6. ROI – Example — Eigene Darstellung	15
2.7. NPU – Quantized Model Conversion to NPU Executables [23]	17
2.8. NPU – Compilation [23]	18
5.1. Use-Case Diagramm Fingeralphabet Trainer	25
6.1. Blockdiagramm des Systemkonzepts	29
6.2. KI-Verarbeitungskette zur Erkennung der Fingeralphabetzeichen	31
6.3. Speicher- und Datenflusskonzept des Gesamtsystems	32
6.4. UML Aktivitätsdiagramm – Ablauf vom Kamerabild bis zum Erkennungsergebnis	34
7.1. Architektur des Multi-Layer Perceptrons — Eigene Darstellung	37
7.2. Zusammenfügen von zwei Layern (STMicroelectronics, 2025)	50
7.3. Task-States des Schedulers	51
8.1. Konfusionsmatrix	67
8.2. Normalisierte Konfusionsmatrix	68

Tabellenverzeichnis

2.1. Charakteristische Merkmale eingebetteter Systeme (nach Wolf, 2012)	4
2.2. Ausgewählte Handkonfigurationen des deutschen Einhand-Fingeralphabets [1]	9
2.3. One-Hot-Kodierung vs. Sparse Indexdarstellung	9
2.4. Confusion Matrix – Begriffsdefinitionen	10
2.5. Die 21 Handlandmark-Punkte	13
2.6. Vergleich Float32 vs. INT8	16
2.7. NPU-Epochen-Typen	18
5.1. Funktionale Anforderungen	25
5.2. Nichtfunktionale Anforderungen	26
5.3. Technische Anforderungen	26
5.4. Randbedingungen	27
5.5. Abnahmekriterien	27
6.1. Zuordnung der Systemdaten zu den Speicherbereichen	33
7.1. Hyperparameter des Modelltrainings	40
7.2. ST Edge AI Core Compiler Flags	42
7.3. NPU-Speicherbereich-Konfiguration	43
7.4. Flash-Partitionierung je Modell	43
7.5. Aufgaben der zentralen Core-Dateien	45
7.6. Initialisierungsreihenfolge nach Systemstart	46
7.7. Grundlegende Systemtreiber der Embedded-Software	48
7.8. Zuordnung der Basistreiber zu den übergeordneten Softwarekomponenten	48
7.9. Einheitliche Statuswerte der KI-Komponenten	56
7.10. Konfigurationsparameter der DCMIPP-Bildpfade	58
8.1. Trainingsergebnisse des Klassifizierungsmodells	67
8.2. Grundlegende Bedingungen der Systemevaluation	68

1. Einleitung

Gebärdensprachen sind visuell-manuelle Sprachen, bei denen Informationen nicht nur durch Handformen und Handbewegung, sondern auch durch Körperhaltung und Mimik vermittelt werden. Das Fingeralphabet stellt innerhalb dieser Sprache ein ergänzendes Hilfsmittel dar. Es ermöglicht, Buchstaben und bestimmte Buchstabenkombinationen durch festgelegte Handformen darzustellen. Verwendet wird es insbesondere zum Buchstabieren von Eigennamen, Fachbegriffen, Fremdwörtern oder Begriffen, für die keine gebräuchliche Gebärde vorhanden ist [1],[2]. Für Personen, die das Fingeralphabet erlernen, ist eine unmittelbare Rückmeldung zur ausgeführten Handform hilfreich. Bereits geringe Abweichungen in der Stellung einzelner Finger können dazu führen, dass ein anderes Zeichen dargestellt wird oder die Handform nicht eindeutig zugeordnet werden kann. Beim selbständigen Üben ist die Beurteilung der eigenen Ausführung jedoch nur eingeschränkt möglich. Eine Rückmeldung durch eine andere Person steht zudem nicht jederzeit zur Verfügung. Computergestützte Lernsysteme können an dieser Stelle eine ergänzende Unterstützung bieten, indem sie ausgeführte Handzeichen erfassen, analysieren und den Lernenden unmittelbar eine Rückmeldung geben. Untersuchungen zu digitalen Lernsystemen für Gebärdensprache zeigen, dass automatisiertes Feedback grundsätzlich zur Unterstützung des selbstständigen Übens eingesetzt werden kann [3].

Im Rahmen dieser Arbeit wird die technische Grundlage für einen solchen Fingeralphabet-Trainer entwickelt. Das System erfasst die Hand über eine Kamera, bestimmt deren Position und charakteristische Landmarks und klassifiziert anschließend das dargestellte Zeichen. Die gesamte Verarbeitung soll lokal auf einem eingebetteten System erfolgen. Als Zielplattform wird das STM32N6570-Discovery Kit verwendet, das neben den üblichen Mikrocontroller-Funktionen über einen integrierten Beschleuniger für neuronale Netze verfügt [4].

1.1. Problemstellung

Die Ausführung einer vollständigen kamerabasierten Erkennungskette auf einem Mikrocontroller ist aufgrund der begrenzten Ressourcen anspruchsvoll [5]. Neben den neuronalen Netzen müssen auch die Kameraschnittstelle, die Bildspeicherung, die Vor- und Nachverarbeitung der Daten, die Ablaufsteuerung sowie die grafische Ausgabe berücksichtigt werden. Werden einzelne Verarbeitungsschritte blockierend oder ineffizient umgesetzt, können hohe Latenzen, eine geringe Bildrate oder instabile Erkennungsergebnisse entstehen.

Eine weitere Herausforderung liegt in der zuverlässigen Klassifikation der Handzeichen. Einige Zeichen des deutschen Fingeralphabets unterscheiden sich nur durch geringe Veränderungen der Fingerstellung [2]. Gleichzeitig können die Position, Orientierung und Größe der Hand im Kamerabild sowie unterschiedliche Lichtverhältnisse und individuelle Ausführungen der Handzeichen variieren [6]. Das Klassifikationsmodell muss daher ausreichend robust gegenüber diesen Einflüssen sein, ohne die verfügbaren Ressourcen der Zielplattform zu überschreiten. Damit die neuronalen Netze auf der eingebetteten Plattform ausgeführt werden können, müssen sie an deren technischen Anforderungen und Ressourcenbeschränkungen angepasst werden. Hierzu können unter anderem die Modellkomplexität reduziert und die verwendeten Zahlenformate quantisiert werden. Diese Optimierungen können den Speicherbedarf und den Rechenaufwand verringern, zugleich aber die Erkennungsgenauigkeit beeinflussen [7]. Es muss daher ein geeigneter Kompromiss zwischen Modellgröße, Ausführungsgeschwindigkeit und Erkennungsleistung gefunden werden.

Die zentrale Problemstellung dieser Arbeit besteht darin, eine mehrstufige Verarbeitungskette zur Erkennung des deutschen Fingeralphabets so auf einem ressourcenbeschränkten eingebetteten System umzusetzen, dass eine hinreichend genaue, stabile und echtzeitnahe Verarbeitung erreicht wird.

1.2. Motivation und Zielsetzung

Die Motivation für diese Arbeit ergibt sich aus dem Bedarf an einer kompakten und unmittelbaren nutzbaren Unterstützung beim Erlernen des deutschen Fingeralphabets. Ein lokal arbeitendes System kann ausgeführte Handzeichen direkt erfassen und bewerten, ohne dass die Kamerabilder an

einen externen Rechner oder einen Cloud-Dienst übertragen werden müssen. Dadurch kann der Fingeralphabet-Trainer unabhängig von einer Internetverbindung eingesetzt werden. Gleichzeitig bietet die lokale Verarbeitung Vorteile hinsichtlich der Reaktionszeit und des Schutzes der aufgenommenen Bilddaten. Die Entwicklung technischer Assistenz- und Lernsystem für Gebärdensprachen besitzt darüber hinaus eine besondere praktische Relevanz. Der Deutsche Gehörlosen-Bund e. V. weist daraufhin, dass bislang keine Computerprogramme oder Apps verfügbar sind, die Gebärdensprachen vollständig und zuverlässig übersetzen können [2]. Daraus ergibt sich die Motivation, einzelne Teilbereiche gezielt zu untersuchen und technisch umzusetzen. Die vorliegende Arbeit setzt an einem solchen Teilbereich an und entwickelt ein System zur Erkennung von Handzeichen des deutschen Fingeralphabets.

Ziel der Arbeit ist die Entwicklung und Integration eines prototypischen Systems, das statische Handzeichen des deutschen Fingeralphabets über eine Kamera erfasst und auf dem STM32N6570 Discovery Kit klassifiziert. Hierfür wird ein Machine-Learning-Modell entwickelt, das die zuvor ermittelten Handlandmarks verarbeitet und das dargestellte Zeichen einer definierten Klasse zuordnet. Dazu werden ein Datensatz erstellt und aufbereitet, Augmentationsverfahren umgesetzt und unterschiedliche Modellvarianten trainiert. bzw. evaluiert. Das ausgewählte Klassifikationsmodell wird anschließend für die Ausführung auf der Zielplattform optimiert und gemeinsam mit den Modellen zur Handlokalisierung und Landmark-Erkennung in eine durchgängige Verarbeitungspipeline integriert. Darüber hinaus werden die erforderlichen Softwarekomponenten zur Ablaufsteuerung, Datenverarbeitung und grafische Darstellung der Inferenzergebnisse umgesetzt. Abschließend wird untersucht, inwieweit das entwickelte Gesamtsystem die definierten Handzeichen zuverlässig erkennt und eine für die interaktive Nutzung geeignete Verarbeitungsgeschwindigkeit erreicht. Das Ergebnis der Arbeit ist ein funktionsfähiger Prototyp, der die technische Umsetzbarkeit eines lokal arbeitenden Fingeralphabet-Trainers auf einer ressourcenbeschränkten eingebetteten Plattform demonstriert.

1.3. Abgrenzung des Untersuchungsgegenstandes

1.3.1. Inhaltliche Abgrenzung

Diese Arbeit beschreibt die Entwicklung eines prototypischen Systems zur Erkennung des deutschen Fingeralphabets auf einer eingebetteten Plattform. Im Mittelpunkt steht die Entwicklung eines funktionsfähigen Gesamtsystems, das kamerabasierte Bildverarbeitung und maschinelles Lernen miteinander verbindet. Ein wesentlicher Bestandteil ist die Entwicklung eines Machine-Learning-Modells zur Klassifikation ausgewählter Handzeichen des deutschen Einhand-Fingeralphabets. Dieses Modell wird gemeinsam mit zwei bereits bestehenden Modellen zur Handerkennung und zur Bestimmung von Handlandmarks in eine mehrstufige Verarbeitungspipeline integriert. Darüber hinaus umfasst die Arbeit die Überführung der Modelle auf die eingebettete Zielplattform.

Nicht Gegenstand der Arbeit ist die Erkennung vollständiger Gebärdensprachen. Insbesondere werden keine dynamischen Gebärden, Bewegungsabläufe, Gesichtsausdrücke oder Körperhaltungen berücksichtigt. Das entwickelte System beschränkt sich auf die Erkennung statischer Handzeichen des deutschen Fingeralphabets unter den im Rahmen des Prototypes definierten Einsatzbedingungen. Darüber hinaus erfolgt keine linguistische oder semantische Betrachtung des Fingeralphabets. Ebenso ist auch kein groß angelegtes Training von Modellen mit umfangreichen Datensätzen vorgesehen. Stattdessen werden eigens erstellte aber begrenzte Datensätze verwendet, die dem prototypischen Nachweis der Funktionsfähigkeit dienen. Die Implementierung und Evaluation beschränken sich zudem auf eine spezifische Plattform.

1.3.2. Personenspezifische Abgrenzung

Das Projekt wurde im Rahmen einer Gruppenarbeit durchgeführt. Daher ist eine klare inhaltliche Abgrenzung der jeweiligen Aufgabenbereiche und der erbrachten Eigenleistungen erforderlich. Die Bearbeitung lässt sich grundsätzlich in zwei Schwerpunkte unterteilen: die Entwicklung des Machine-Learning-Modells sowie die Umsetzung der Software- und Hardwarekomponenten, auf denen das Modell ausgeführt und in das Gesamtsystem integriert wird.

Fabian Weber übernimmt überwiegend die Entwicklung des maschinellen Lernmodells. Dazu zählen insbesondere die Erstellung und Aufbereitung des Datensatzes, die Entwicklung der Datenpipeline einschließlich Augmentationsverfahren sowie das Training und die Optimierung des Modells. Oliver Groß ist für die Umsetzung der Hardware und Software verantwortlich. Dies umfasst vor allem den Aufbau und die Integration der notwendigen Camera-Pipeline, sowie die Implementierung der Systemumgebung zur Ausführung des Modells. Gemeinsame Aufgaben betreffen insbesondere die Abstimmung der Systemarchitektur, sowie die Integration der einzelnen Komponenten zu einem funktionsfähigen Gesamtsystems.

1.4. Aufbau der Arbeit

Nach der Einleitung werden in Kapitel 2 die theoretischen und technischen Grundlagen erläutert. Dazu gehören eingebettete Systeme, das deutsche Fingeralphabet, grundlegende Verfahren des maschinellen Lernens, neuronale Netze zur Klassifikation, Handdetektion und Handlandmarken sowie Datenaugmentation, Modellquantisierung und Edge AI. Kapitel 3 stellt den Stand der Technik dar. Es werden bestehende Ansätze zur Erkennung von Handzeichen und Fingeralphabeten sowie aktuelle Embedded-AI-Lösungen betrachtet. Anschließend wird die vorliegende Arbeit gegenüber den bestehenden Ansätzen eingeordnet und abgegrenzt.

In Kapitel 4 werden die verwendete Methodik und der Entwicklungsprozess beschrieben. Dazu gehören das Vorgehen bei der Literatur- und Technologierecherche, das gewählte Vorgehensmodell, der iterative Integrationsprozess, die Modellentwicklung sowie das Verifikations- und Evaluationskonzept. Kapitel 5 leitet aus dem Anwendungsszenario die funktionalen und nichtfunktionalen Anforderungen, die technischen Randbedingungen und die Abnahmekriterien ab. Der Systementwurf wird in Kapitel 6 vorgestellt. Dabei werden das System- und Hardwarekonzept, die KI-Verarbeitungskette sowie der Speicher- und Datenfluss vom Kamerabild bis zum Erkennungsergebnis beschrieben. Kapitel 7 behandelt die konkrete Entwicklung und Implementierung. Hierzu gehören die Aufbereitung des Datensatzes, die Augmentationspipeline, das Training und die Quantisierung des Klassifikationsmodells, die Implementierung der Kamerapipeline, die Handflächen- und Landmark-Erkennung, die Bestimmung der Region of Interest, die Verarbeitung der Landmark-Daten sowie die Integration der neuronalen Netze, die Ablaufsteuerung und die Visualisierung.

Anschließend wird das entwickelte System in Kapitel 8 anhand der in der Anforderungsanalyse festgelegten Kriterien evaluiert und validiert. Hierzu werden zunächst der Versuchsaufbau und die zugrunde liegenden Testbedingungen beschrieben. Darauf aufbauend erfolgt die Untersuchung der Erkennungsgenauigkeit sowie eine klassenbezogene Auswertung mithilfe einer Konfusionsmatrix. Zusätzlich wird analysiert, welchen Einfluss unterschiedliche Beleuchtungsbedingungen und Hintergründe auf die Erkennungsleistung haben. Neben der funktionalen Bewertung werden auch die Inferenz- und Ende-zu-Ende-Latenz, die erreichbare Bildrate und der Datendurchsatz sowie der Speicher- und Ressourcenverbrauch betrachtet. Abschließend werden die erzielten Ergebnisse mit den zuvor definierten Anforderungen abgeglichen.

Die Ergebnisse der Evaluation werden in Kapitel 9 interpretiert und diskutiert. Dabei werden das Gesamtsystem, seine Grenzen und der technische sowie praktische Beitrag der Arbeit betrachtet. Kapitel 10 fasst die wesentlichen Ergebnisse zusammen, bewertet die Erreichung der Zielsetzung und gibt einen Ausblick auf mögliche Weiterentwicklungen.

2. Theoretische und technische Grundlagen

Dieses Kapitel vermittelt die für das Verständnis der Arbeit erforderlichen Grundlagen. Es beginnt mit den zentralen Konzepten eingebetteter Systeme und der Bare-Metal-Programmierung. Im Anschluss wird das deutsche Einhand-Fingeralphabet als zu erkennendes Zeichensystem eingeführt. Abschließend werden die Grundlagen des maschinellen Lernens, der Handdetektion, der Modellquantisierung sowie der Edge AI erläutert.

2.1. Eingebettete Systeme & STM32N6570DK

2.1.1. Begriffsbildung und Merkmale

Eingebettete Systeme (engl. embedded systems) sind Computersysteme, die als Bestandteil eines übergeordneten Gesamtsystems in eine technische Umgebung eingebettet sind [8, s. 2]. Im Gegensatz zu universellen Rechnern (Personal Computer, Server) erfüllen sie eine spezifische, fest definierte Aufgabe und sind darauf optimiert. Typische Merkmale sind [9]:

Tabelle 2.1.: Charakteristische Merkmale eingebetteter Systeme (nach Wolf, 2012)

Aspekt	Embedded Spezifisch
Ressourcenbeschränkung	Eingebettete Systeme verfügen über begrenzten Arbeitsspeicher (RAM), begrenzten Programmspeicher (Flash) und eine eingeschränkte Rechenleistung im Vergleich zu General-Purpose-Computern.
Echtzeitfähigkeit	Viele Anwendungen erfordern, dass Daten innerhalb definierter Zeitgrenzen verarbeitet werden. Man unterscheidet zwischen weicher Echtzeit (eine Überschreitung der Frist ist unerwünscht, aber tolerierbar) und harter Echtzeit (eine Überschreitung führt zum Systemversagen).
Hohe Zuverlässigkeit	Da eingebettete Systeme häufig in sicherheitskritischen oder industriellen Umgebungen eingesetzt werden, muss ein definiertes Fehlverhalten über lange Zeiträume vermieden werden.
Energieeffizienz	Viele Systeme arbeiten mit begrenzter Energieversorgung (Batterien, Netzteil), weshalb der Energieverbrauch eine zentrale Designrandbedingung darstellt.
Spezialisierung	Im Unterschied zu universellen Rechnern ist die Funktionalität eines eingebetteten Systems auf einen konkreten Anwendungsfall beschränkt.

2.1.2. Mikrocontroller Architektur

Als dominierende Hardwareplattform eingebetteter Systeme dienen Mikrocontroller (MCU, engl. Microcontroller Unit). Ein Mikrocontroller integriert auf einem einzigen Halbleiterchip alle für einen vollständigen Computer wesentlichen Komponenten [9]:

- **CPU:** Recheneinheit mit Instruktionssatz (z. B. Arm Cortex-M, RISC-V). Die Taktfrequenz bestimmt die Verarbeitungsgeschwindigkeit und reicht von wenigen MHz bis zu mehreren hundert MHz.
- **Programmspeicher (Flash):** Nichtflüchtiger Speicher für den Programmcode und konstante Daten. Die Größe variiert von einigen Kilobyte bis zu mehreren Megabyte.
- **Arbeitsspeicher (RAM):** Flüchtiger Speicher für Laufzeitdaten, Variablen und Stack. Typisch sind wenige Kilobyte bis einigen hundert Kilobyte.
- **Taktgeber:** Interner Oszillator (HSI, engl. High-Speed Internal) oder externer Quarz (HSE, engl. High-Speed External) als Zeitbasis für die CPU und Peripherie.
- **Peripherie-Einheiten:** Hardwaremodule für Ein-/Ausgabe, Kommunikation und Zeitsteuerung (siehe Abschnitt 2.1.3).

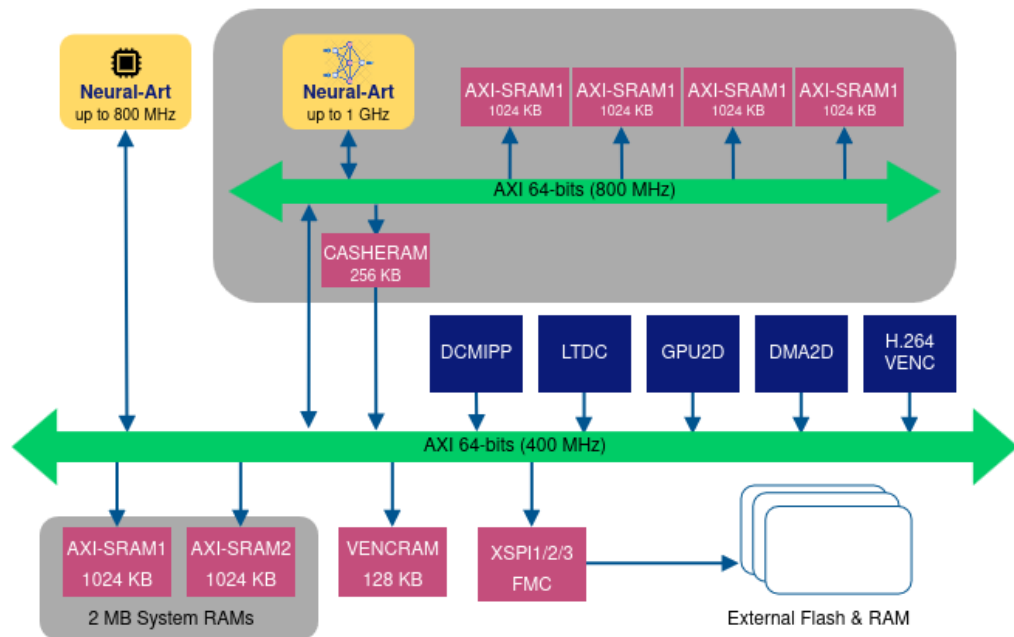


Abbildung 2.1.: STM32N6570-DK - Architektur

2.1.3. Peripherieschnittstellen

Mikrocontroller stellen verschiedene Hardware-Schnittstellen bereit, um mit der Umwelt zu interagieren. Die wesentlichen Peripherietypen sind [4][10]:

- **General Purpose Input/Output (GPIO):** GPIO-Pins sind die einfachste Form der digitalen Ein-/Ausgabe. Jeder Pin kann konfiguriert werden als Eingang (Lesen eines logischen Pegels: 0 oder 1) oder als Ausgang (Setzen eines Pegels). GPIO-Pins werden zum Ansteuern von LEDs, Auslesen von Tasten oder als Steuerleitungen für andere Peripheriemodule verwendet.
- **Universal Synchronous/Asynchronous Receiver/Transmitter (USART):** Serielles Kommunikationsprotokoll zur asynchronen Datenübertragung. Es wird für Debug-Ausgaben, die Kommunikation mit Host-PCs oder seriellen Sensoren eingesetzt.
- **Serial Peripheral Interface (SPI):** Synchrones, voll duplexes Kommunikationsprotokoll mit Transmitter-Receiver-Architektur. Es bietet höhere Datenraten als UART und wird bevorzugt für externe Speicher (Flash, PSRAM), Display-Controller oder Sensoren verwendet. Ein SPI-Bus besteht typischerweise aus den Leitungen COPI, CIPO, SCK und einem Chip-Select (CS).
- **Inter-Integrated Circuit (I²C):** Zweidrahtiges, synchrones Kommunikationsprotokoll (SDA, SCL) mit Adressierung. Es ermöglicht den Anschluss mehrerer Peripherals über denselben Bus und wird für Sensoren, Echtzeituhren (RTC) und kleinere EEPROMs verwendet.
- **Timer:** Hardware-Timer zählen Taktimpulse und können Interrupts auslösen, wenn ein Zählerwert erreicht wird. Sie dienen als Zeitbasis für periodische Aufgaben, zur Erzeugung von PWM-Signalen (Pulsweitenmodulation) und zur Messung von Signalen (Input Capture). In Echtzeitsystemen stellen Timer den Taktgeber für den Scheduler dar.
- **Direct Memory Access (DMA):** DMA-Einheiten ermöglichen Datenübertragungen zwischen Peripherie und Speicher ohne CPU-Beteiligung. Die CPU gibt lediglich den Startbefehl und kann während der Übertragung andere Aufgaben verarbeiten. DMA ist besonders für große Datenströme (z. B. Kamerabilder) relevant, da es die CPU erheblich entlastet.
- **DMA2D (ChromART):** Eine spezielle DMA-Einheit für 2D-Blit-Operationen: Kopieren, Füllen und Alpha-Blending von Bilddaten ohne die CPU zu blockieren (Erweiterung des DMA).

-
- **Interrupts:** Interrupts ermöglichen es der CPU, auf asynchrone Ereignisse (z. B. Eingangsimpuls, abgeschlossene Datenübertragung, Timer-Überlauf) zeitnah zu reagieren, ohne den Ereigniszeitpunkt aktiv abfragen zu müssen. Der Interrupt-Controller (NVIC, engl. Nested Vectored Interrupt Controller bei Arm Cortex-M) verwaltet Prioritäten und erlaubt Verschachtelung (Nested Interrupts). Nach dem Speichern des laufenden Kontexts (Registersatz) springt die CPU über die Vektortabelle auf die zugehörige Interrupt-Service-Routine (ISR)[11, s. 104 ff.].

2.1.4. Speicherarchitektur

Die Speicherarchitektur eingebetteter Systeme weicht wesentlich von klassischen Rechnern ab. Die wesentlichen Speichertypen sind [9],[4],[12]:

- **Flash-Speicher (On-Chip):** Nichtflüchtig, integriert auf dem MCU-Chip. Dient als Programmspeicher (Code) und für konstante Daten (Konstanten-Tabelle, lookup tables). Schreibzugriffe sind langsam und erfordern einen Löschzyklus.
- **SRAM (On-Chip):** Flüchtig, sehr schneller Zugriff (im Takt der CPU). Dient als Hauptspeicher für Variablen, Heap und Stack. Auf Cortex-M85-Systemen ist SRAM in mehrere Banken aufgeteilt (AXISRAM), um parallele Zugriffe durch CPU, DMA und NPU zu ermöglichen.
- **PSRAM (extern):** Pseudo-Static RAM, angeschlossen über OctoSPI oder Quad-SPI. Bietet größere Speicherkapazitäten als On-Chip-SRAM bei moderater Zugriffszeit. Im STM32N6570-System wird PSRAM für Zwischenspeicherung von Bilddaten genutzt.
- **NOR-Flash (extern):** Nichtflüchtiger Speicher mit seitenweisem Zugriff, angeschlossen über OctoSPI. Im vorliegenden Projekt werden die vorkompilierten Modellbinaries (Palm Detection, Hand Landmark, Fingeralphabet) in externem NOR-Flash bei festen Adressen gespeichert.

Die Speicherhierarchie bestimmt maßgeblich die Systemleistung. Cortex-M55-Kerne verfügen über ein Cachesystem (I-Cache, D-Cache), das den Zugriff auf langsame externe Speicher beschleunigt. Da Peripherie-Einheiten (LTDC, DMA2D) jedoch häufig direkt auf den Speicher zugreifen, muss die Cache-Kohärenz manuell verwaltet werden (Flush/Invalidate) [12][13].

2.1.5. Clock-Systeme

Das Clock-System bestimmt die Taktfrequenz aller Komponenten und ist eine fundamentale Voraussetzung für den Betrieb des Mikrocontrollers [4][12].

- **Oszillatoren:** Der interne Hochgeschwindigkeitsozillator (HSI) bietet eine integrierte, quarzfreie Takquelle (typisch 16–64 MHz). Externe Quarze (HSE) liefern eine höhere Genauigkeit und Stabilität.
- **PLL (Phase-Locked Loop):** Ein PLL multipliziert die Taktfrequenz der Basisoszillatoren auf höhere Taktfrequenzen. Das STM32N6570-System verwendet mehrere PLLs: PLL1 für den CPU-Takt (800 MHz), PLL2 für den NPU (1000 MHz), PLL3 für den NPU-Speicher (900 MHz) und PLL4 für Peripherietakte.
- **Takthierarchie:** Der PLL-Ausgang wird über Teiler (Prescaler) auf verschiedene Busdomänen verteilt: AHB (High-Speed Bus), APB1/APB2 (Peripheral Buses). Peripherie-Einheiten erhalten ihren Takt von diesen Bussen.

2.1.6. Bare-Metal-Programmierung

- **Definition und Abgrenzung:** Als Bare-Metal-Programmierung bezeichnet man die Softwareentwicklung auf einem Mikrocontroller ohne Einsatz eines Betriebssystems (OS) oder Echtzeitbetriebssystems (RTOS) [14]. Der Programmcode hat direkten Zugriff auf die Hardwareregister, und die Ausführungsreihenfolge wird vollständig durch den eigenen Code bestimmt. Dies steht im Gegensatz zu OS-basierter Programmierung, bei der ein Betriebssystem (z.B. FreeRTOS, Zephyr) die Ressourcenverwaltung, Scheduling und Synchronisierung übernimmt.

- **Startup und Systeminitialisierung:** Nach dem Einschalten oder Reset beginnt die CPU an einer durch die Vektor-Tabelle definierten Adresse (Reset-Handler). Der Startup-Code führt folgende Schritte aus: (1) Kopieren der Initialisierungsdaten aus Flash nach SRAM (.data-Sektion), (2) Löschen der .bss-Sektion (uninitialisierte globale Variablen), (3) Konfiguration des Stack-Pointers, (4) Aufruf der main-Funktion. Die Vektor-Tabelle enthält außerdem Adressen aller Interrupt-Handler (Exceptions).
- **Register-Level-Programmierung:** Im Bare-Metal-Ansatz werden Peripherie-Einheiten über Memory-Mapped I/O-Register konfiguriert. Jedes Register hat eine fest definierte Adresse im Speicherbereich. Die Konfiguration erfolgt durch Setzen oder Löschen einzelner Bits mit Bitmasken. Beispielsweise wird ein GPIO-Pin als Ausgang konfiguriert, indem im Mode-Register die entsprechenden Bits gesetzt werden. Der Vorteil gegenüber einer Hardware-Abstraktionsschicht (HAL) liegt in der vollen Kontrolle über die Ausführungszeit und den Speicherverbrauch.
- **Vorteile und Trade-Offs:** Bare-Metal bietet minimale Latenz, minimalen Speicherverbrauch und deterministisches Verhalten. Die Nachteile liegen in der höheren Entwicklungskomplexität: Es gibt kein Thread-Management, keine Synchronisationsprimitive (Mutexe, Semaphore) und keine automatische Ressourcenverwaltung. Für ressourcenbeschränkte Systeme mit klar definiertem Ablauf überwiegen jedoch die Vorteile [14][10].

2.1.7. Hardware-Beschleunigung

Neben der CPU stehen dedizierte Hardware-Einheiten zur Verfügung, die bestimmte Operationen mit hoher Effizienz ausführen:

- **DMA:** Entlastet die CPU bei Datenübertragungen (siehe Abschnitt 2.1.3).
- **DMA2D:** Übernimmt 2D-Bildoperationen (Kopieren, Skalieren, Füllen) ohne CPU-Beteiligung.
- **NPU (Neural Processing Unit):** Dedizierter Beschleuniger für neuronale Netze. Die NPU führt Matrix-Multiplikationen und Aktivierungsfunktionen mit hoher Parallelität aus und erreicht dabei deutlich höhere Energieeffizienz als die allgemeine CPU. Im STM32N6570-DK arbeitet die NPU mit einer Taktfrequenz von 1000 MHz und nativer INT8-Arithmetik (siehe Abschnitt 2.7) [12, s. 1011 ff.].

2.2. Deutsches Einhand-Fingeralphabet

2.2.1. Gebärdensprache als eigenständige Sprache

Gebärdensprachen sind natürlich entstandene, vollwertige Sprachen, die auf visuell-manuellen Mitteln basieren [2]. Sie unterscheiden sich von Lautsprachen nicht nur in der Modalität (Gestik und Mimik statt Laut und Klang), sondern verfügen über eigene grammatische Strukturen, Syntax und Semantik. Gebärdensprachen sind eigenständige Sprachen und keine abgeleiteten oder vereinfachten Formen der jeweiligen Lautsprache.

Die Deutsche Gebärdensprache (DGS) ist die natürlich entstandene Sprache der Gehörlosengemeinschaft in Deutschland. Sie wird vor allem von gehörlosen und schwerhörigen Menschen sowie von deren Angehörigen und Gebärdensprachdolmetschern verwendet. Die DGS unterscheidet sich wesentlich von der deutschen Lautsprache in Grammatik, Satzbau und Ausdruck. Gebärdensprachliche Äußerungen bestehen aus einer Kombination von Handformen, Handbewegungen, Handpositionen im Raum sowie Mimik und Körpersprache [2].

2.2.2. Das Fingeralphabet

Das Fingeralphabet ist ein Teilgebiet der Deutschen Gebärdensprache [1],[2]. Es dient dem Buchstabieren einzelner Wörter und wird verwendet, wenn für ein Wort keine eigene Gebärde existiert. Etwa bei Eigennamen, Fachbegriffen oder bei der Kommunikation mit hörenden Gesprächspartnern. Im Gegensatz zur vollständigen Gebärdensprache beschränkt sich das Fingeralphabet auf die Darstellung von Handformen ohne Begleitung durch Mimik oder Bewegung im Raum.

2.2.3. Das deutsche Einhand-Fingeralphabet

- **26 Buchstaben:** A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z, Ä, Ö, Ü
- **1 Sonderzeichen:** SCH (als Trigraph dargestellt, d.h. ein Zeichen für drei Buchstaben)



Abbildung 2.2.: Deutsches Einhand-Fingeralphabet [1]

2.2.4. Handpositionen und Fingerkonfigurationen

Jedes Zeichen des Fingeralphabets wird durch eine spezifische Konfiguration der Hand charakterisiert, die sich aus mehreren Parametern zusammensetzt [1]:

- **Beteiligte Finger:** Welche Finger sind gestreckt, welche sind gebeugt oder angeballt?
- **Fingerkonfiguration:** Sind Finger together (zusammengedrückt) oder gespreizt?
- **Daumenposition:** Liegt der Daumen über den Fingern, neben ihnen, oder ist er abgespreizt?
- **Handfläche:** Zeigt die Handfläche zum Betrachter (ventral) oder weg (dorsal)?
- **Fingerstellung:** Zeigen die Finger nach oben, unten oder zur Seite?

Beispielhafte Beschreibung einiger Zeichen:

2.3. Grundlagen des maschinellen Lernens

#TODO Quellen für diesen Abschnitt!

Tabelle 2.2.: Ausgewählte Handkonfigurationen des deutschen Einhand-Fingeralphabets [1]

Zeichen	Handkonfiguration
A	Faust geschlossen, Daumen liegt seitlich an der Handfläche an.
B	Vier Finger gestreckt und zusammen, Daumen auf die Handfläche geklappt.
C	Alle Finger gebeugt, Hand formt eine C-förmige Öffnung.
E	Alle Finger auf den Daumen geklappt, Daumen sichtbar unter den Fingerspitzen.
I	Nur kleiner Finger gestreckt, übrige Finger geballt.
L	Daumen und Zeigefinger im 90°-Winkel gestreckt, übrige Finger geballt.
S	Faust geschlossen, Daumen vor den Fingern gekreuzt.
Y	Daumen und kleiner Finger gestreckt, übrige Finger geballt.
SCH	Alle Finger vom Handballen weggestreckt.

2.3.1. Begriffsbildung

Maschinelles Lernen (ML, engl. *machine learning*) ist ein Teilgebiet der künstlichen Intelligenz, das Systeme befähigt, in Daten komplexe Muster und Zusammenhänge zu lernen und darauf basierend Vorhersagen oder Entscheidungen zu treffen, ohne explizit programmiert zu werden. Anstelle fester Regeln wird dem Algorithmus ein Trainingsdatensatz zur Verfügung gestellt, aus dem er statistische Muster und Zusammenhänge ableitet.

2.3.2. Supervised Learning

Im "überwachten Lernen" (supervised learning) wird dem Modell ein Datensatz bestehend aus Eingabedaten (Features) und zugehörigen korrekten Ausgaben (Labels) präsentiert [15]. Das Modell lernt, eine Funktion $f : X \rightarrow Y$ zu approximieren, die Eingaben auf die zugehörigen Ausgaben abbildet.

2.3.3. Klassifikation

Klassifikation ist ein zentrales Teilgebiet des maschinellen Lernens, bei dem die Aufgabe besteht, eine Eingabe einer von diskreten Klassenzugehörigkeiten zuzuordnen. Das Modell produziert für jede Klasse einen Wahrscheinlichkeitswert, und die Klasse mit der höchsten Wahrscheinlichkeit wird als Vorhersage ausgegeben. Typischerweise werden die Klassen innerhalb des Datensatzes mit Hilfe des One-Hot-Formats dargestellt [15].

2.3.3.1. Sparse Categorical Crossentropy

Das One-Hot-Format verbraucht bei Hunderten von Klassen viel Speicherplatz. Sparse categorical Crossentropy löst dieses Problem, indem der Datensatz einen einzigen Integer-Wert annimmt und die mathematisch äquivalente Kreuzentropie-Berechnung speichereffizient im Hintergrund durchführt.

Tabelle 2.3.: One-Hot-Kodierung vs. Sparse Indexdarstellung

Kategorie	Index	One-Hot-Vektor
NONE	0	[1, 0, 0, 0, 0]
A	1	[0, 1, 0, 0, 0]
B	2	[0, 0, 1, 0, 0]
C	3	[0, 0, 0, 1, 0]
D	4	[0, 0, 0, 0, 1]

2.3.4. Metriken

Zur Bewertung eines Klassifikationsmodells werden verschiedene Metriken herangezogen (Hastie et al., 2009):

- **Accuracy (Genauigkeit):** Anteil der korrekten Vorhersagen an allen Vorhersagen. Für ausgewogene Datensätze aussagekräftig, bei unausgewogenen Klassen jedoch irreführend.
- **Confusion Matrix:** Tabelle, die die tatsächlichen Klassen den vorhergesagten Klassen gegenüberstellt. Sie macht sichtbar, welche Zeichen miteinander verwechselt werden.

Tabelle 2.4.: Confusion Matrix – Begriffsdefinitionen

	Predicted 0	Predicted 1
Actual 0	True Negative	False Positive
Actual 1	False Negative	True Positive

2.3.5. Overfitting und Gegenmaßnahmen

Overfitting liegt vor, wenn ein Modell die Trainingsdaten zu gut lernt und dabei spezifische Muster, Rauschen und Ausreißer mitinterpretiert, diese allerdings nicht für die Allgemeingültigkeit relevant sind (Pattern Recognition and Machine Learning, n.d.). Das Modell performt auf Trainingsdaten exzellent (Hohe Genauigkeit), auf unbekannten Daten jedoch schlecht (Geringe Genauigkeit).

Gegenmaßnahmen bezüglich Overfitting umfassen:

- Regularisierung: Bestrafung großer Gewichtswerte, um die Modellkomplexität zu begrenzen.
- Dropout: Zufälliges Deaktivieren eines Teils der Neuronen während des Trainings, um Abhängigkeiten zwischen Neuronen zu reduzieren.
- Dataaugmentation: Künstliche Vermehrung der Trainingsdaten durch Transformationen (siehe Abschnitt 2.6).
- Validierung: Aufteilen des Datensatzes in Trainings-, Validierungs- und Testset, um die Generalisierungsfähigkeit zu überwachen.

2.3.6. Parameter vs Hyperparameter

Parameter sind Variablen welche aus dem zur Verfügung gestellten Datensatz beim Training erlernt werden. Diese werden nicht durch den Programmierer festgelegt. Hyperparameter hingegen werden vor dem Trainingsprozessen bestimmt. Diese parameter bestimmen das Lernverhalten [16].

2.4. Neuronale Netze zur Klassifikation

2.4.1. Künstliche Neuronale Netze

#TODO Check those Quellen Ein künstliches neuronales Netz (KNN) ist ein mathematisches Modell, das lose von biologischen neuronalen Netzen inspiriert ist [17], [18]. Es besteht aus einer Menge von künstlichen Neuronen (auch Einheiten oder Knoten genannt), die in Schichten (engl. layers) organisiert sind. Jedes Neuron berechnet eine gewichtete Summe seiner Eingaben, addiert einen Bias-Wert und wendet eine Aktivierungsfunktion an:

$$y = f \left(\sum_{i=1}^n w_i \cdot x_i + b \right)$$

wobei x_i die Eingaben, w_i die Gewichte, b der Bias und f die Aktivierungsfunktion sind.

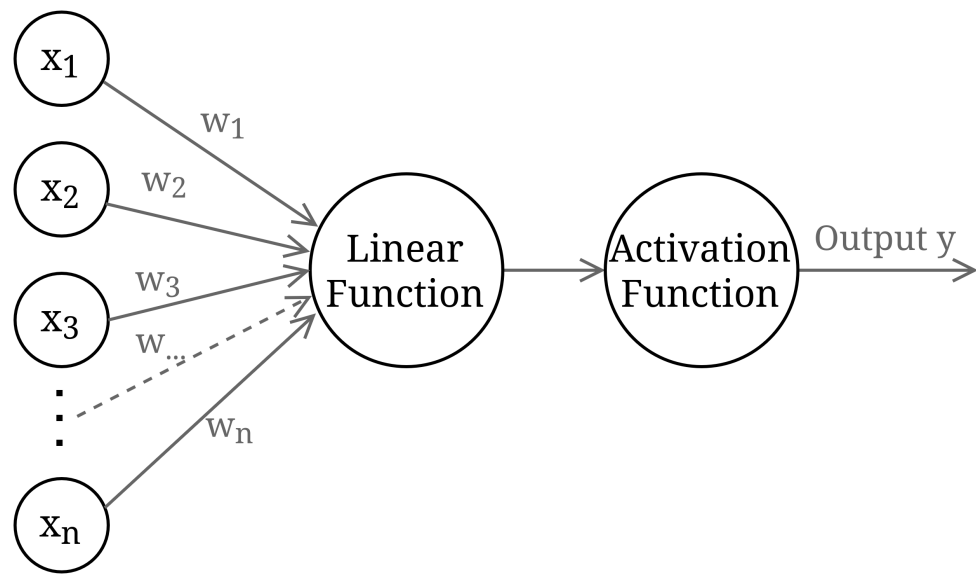


Abbildung 2.3.: Mathematisch modelliertes Neuron (Gurney, 2018) — Eigene Darstellung

Eingabeschicht

Versteckteschichten

Ausgabeschicht

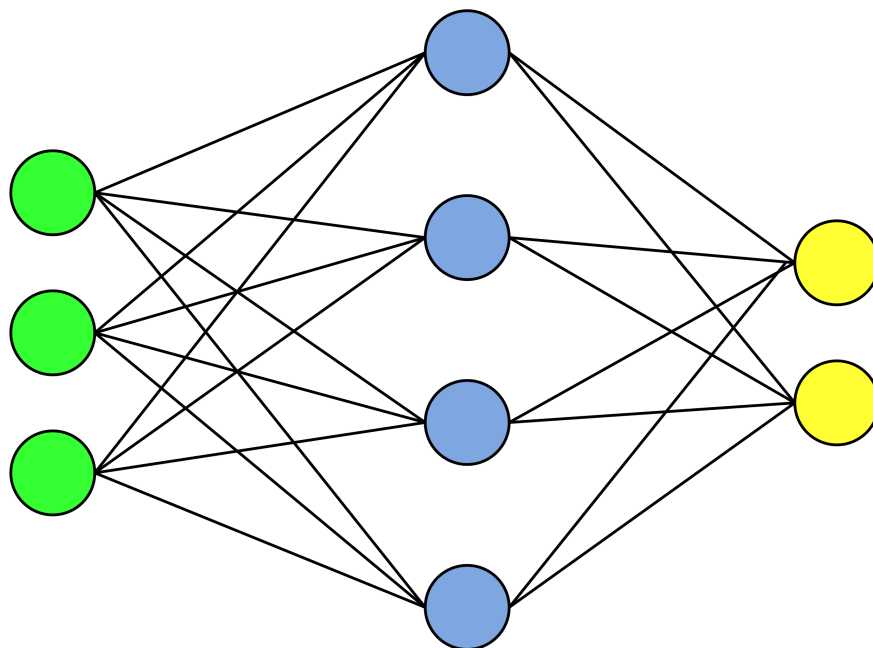


Abbildung 2.4.: Allgemeine Struktur Künstlicher Neuronaler Netze (Kim, 2016) — Eigene Darstellung

2.4.2. Aktivierungsfunktionen

Aktivierungsfunktionen fügen Nichtlinearität in das Netz ein und ermöglichen es, komplexe Muster zu erlernen. Die wesentlichen im Projekt verwendeten Funktionen sind (Kim, 2016):

- **ReLU (Rectified Linear Unit):** $f(x) = \max(0, x)$. Einfach zu berechnen, gradientenfreundlich und in versteckten Schichten Standard. Negative Eingaben werden auf 0 gesetzt.
- **Softmax:** Wandelt einen Vektor von Rohergebnissen (logits) in eine Wahrscheinlichkeitsverteilung um. Für die Ausgabeschicht eines Klassifikators geeignet, da die Summe aller Ausgabe-werte 1 ergibt und jeder Wert zwischen 0 und 1 liegt.

2.4.3. Architektur: Multi-Layer Perceptron (MLP)

Ein Multi-Layer Perceptron (MLP) ist ein vorwärtsgerichtetes Neuronales Netz (FNN) mit mindestens einer versteckten Schicht (Kim, 2016). Die Daten durchlaufen das Netz in eine Richtung, von der Eingabeschicht über die versteckten Schichten zur Ausgabeschicht, ohne Rückkopplungen.

2.4.4. Architektur: Convolutional Neural Networks (CNN)

Ein Convolutional Neural network (CNN) ist ein vorwärtsgerichtetes Neuronales Netz (FNN) mit mindestens einer versteckten Schicht. Die Struktur klassischer Convolutional Neural Networks besteht aus mehreren Convolutional layer gefolgt von einem Pooling layer. Diese Struktur kann beliebig oft innerhalb des Model seriell aufgerufen werden. Typische Eingabevektoren kommen aus den Bereichen Bildverarbeitung, Bildklassifizierung und Objekterkennung und Lokalisierung.

2.4.5. Backpropagation und Gradient Descent

Das Training neuronaler Netze erfolgt mittels *Backpropagation* (Rückpropagierung des Fehlers) in Kombination mit einem Optimierungsalgorithmus wie Stochastic Gradient Descent (SGD) oder Varianten davon (Adam, RMSprop) (Kim, 2016; Raschka et al., 2022):

1. Forward Pass: Die Eingabedaten durchlaufen das Netz, und die Vorhersage wird berechnet.
2. Loss-Berechnung: Die Abweichung zwischen Vorhersage und tatsächlichem Label wird mittels einer Loss-Funktion (z.B. Categorical Cross-Entropy für Multi-Class-Klassifikation) quantifiziert.
3. Backward Pass: Die Ableitungen des Loss nach den Gewichten werden über die Schichten zurückgerechnet (Kettenregel der Differentiation).
4. Gewichts-Update: Die Gewichte werden in Richtung des negativen Gradienten angepasst, um den Loss zu minimieren.

Dieser Zyklus wird über mehrere Epochen (Durchläufe durch den gesamten Datensatz) wiederholt, bis das Modell eine zufriedenstellende Güte erreicht.

2.5. Handdetektion und Handlandmarks

2.5.1. Zweistufige Erkennungspipeline

Die Erkennung von Handzeichen in Kamerabildern erfordert eine mehrstufige Verarbeitung. Der gängigste Ansatz, der auch im vorliegenden Projekt verwendet wird, basiert auf einer zweistufigen Pipeline [19], [20]:

Stufe 1 - Palm Detection (Handflächen-Erkennung): Im gesamten Kamerabild wird die Position einer oder mehrerer Hände lokalisiert. Das Ergebnis ist eine Bounding Box (Umrahmung) um die erkannte Handfläche. Stufe 2 - Hand Landmark-Erkennung: Innerhalb der erstellten Bounding Box werden 21 charakteristische Landmark-Punkte bestimmt, die die Position der Finger-Gelenke und Fingerspitzen beschreiben.

Diese Zweiteilung ist aus Effizienzgründen sinnvoll: Die Palm Detection arbeitet auf einem downgesampelten Bild (192×192 Pixel), während die Landmark-Erkennung auf dem zugeschnittenen Hand-Ausschnitt (224×224 Pixel) arbeitet. Dadurch wird die Rechenlast erheblich reduziert.

2.5.2. Palm Detection

Die Palm Detection identifiziert die Position der Handfläche im Kamerabild [19]. Das Modell (basierend auf MediaPipe) nimmt ein RGB-Bild der Größe 192×192 Pixel als Eingabe und erzeugt als Ausgabe eine Menge von Kandidaten. Jeder Kandidat besteht aus:

- Score: Ein Vertrauenswert (0 bis 1), der angibt, wie wahrscheinlich es sich um eine Handfläche handelt.
- Regression: Vier Werte, die die Bounding Box (x, y, Breite, Höhe) der Handfläche definieren.

Da das Modell pro Bild 2016 Kandidaten ausgibt, wird eine Non-Maximum Suppression (NMS) durchgeführt: Überlappende Bounding Boxes werden zusammengefasst, und nur der Kandidat mit dem höchsten Score in jedem Cluster wird beibehalten. Als Schwellenwerte dienen ein Confidence-Threshold von 0,3 und ein IoU-Threshold (Intersection over Union) von 0,4.

2.5.3. Handlandmarks

Die Handlandmark-Erkennung nimmt den durch die Palm Detection definierten Bildausschnitt (ROI, engl. Region of Interest) und bestimmt darin 21 Landmark-Punkte [20]. Jeder Punkt wird durch seine relative Position (x, y) innerhalb der ROI beschrieben. Die 21 Landmarks entsprechen den anatomisch relevanten Punkten der Hand:

Tabelle 2.5.: Die 21 Handlandmark-Punkte

Index	Beschreibung
0	Handgelenk (Wrist)
1–4	Daumen: MCP, IP, Tip (3 Punkte)
5–8	Zeigefinger: PIP, DIP, Tip (3 Punkte)
9–12	Mittelfinger: PIP, DIP, Tip (3 Punkte)
13–16	Ringfinger: PIP, DIP, Tip (3 Punkte)
17–20	Kleiner Finger: PIP, DIP, Tip (3 Punkte)

MCP = Metacarpophalangeal Joint, IP = Interphalangeal Joint, PIP = Proximal Interphalangeal Joint, DIP = Distal Interphalangeal Joint, Tip = Fingerspitze

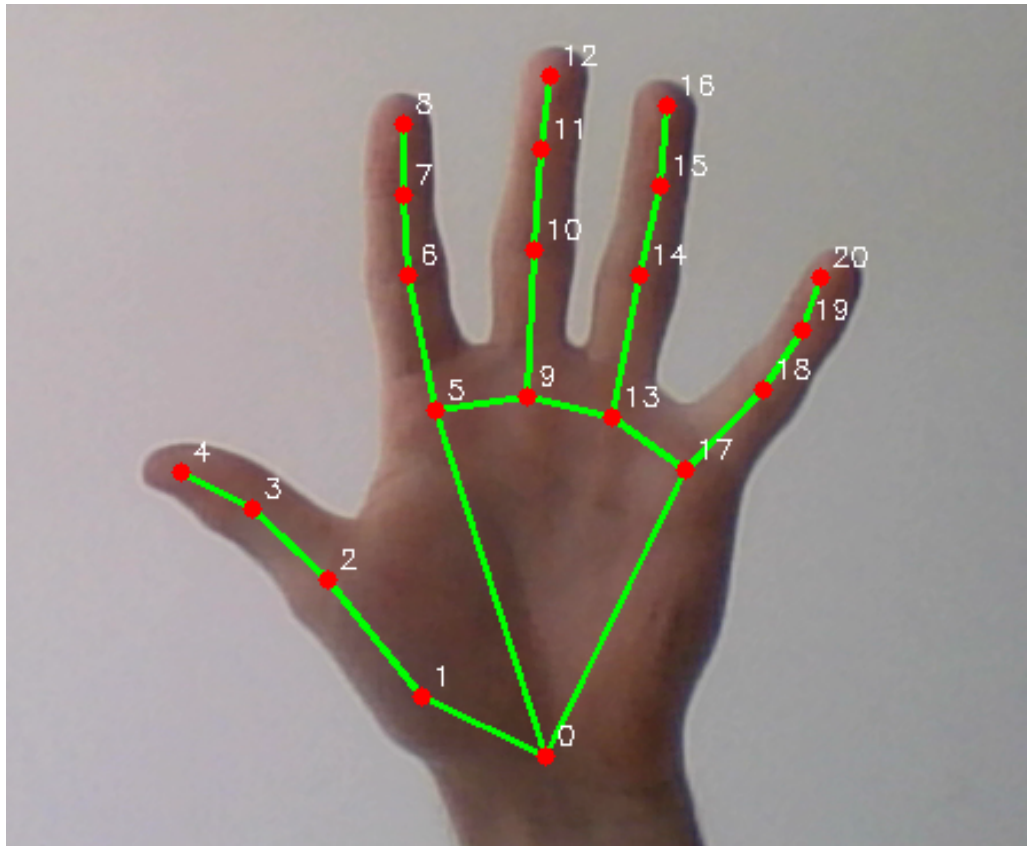


Abbildung 2.5.: Beispiel Hand Landmark Punkte — Eigene Darstellung

Zusätzlich gibt das Modell einen Handedness-Score aus, der angibt, ob es sich um eine linke oder rechte Hand handelt.

2.5.4. ROI

Eine ROI (Region of Interest) ist ein rotiertes Rechteck im Pixelraum, das genau definiert, welchen Bildausschnitt ein Modell fokussieren oder als Eingabe für ML Modelle dienen soll. Dies kann verwendet werden um Modelle gezielt auszuführen.

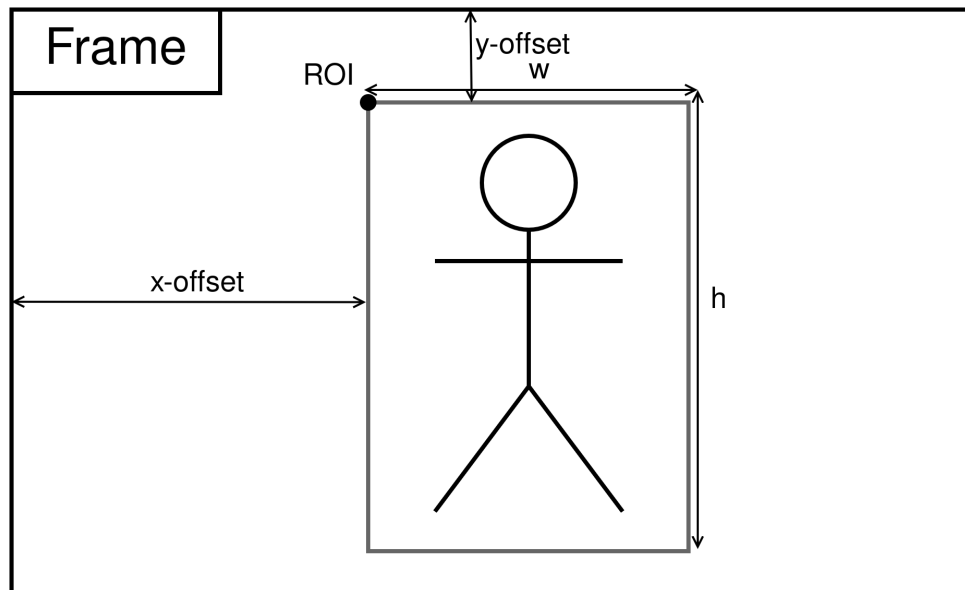


Abbildung 2.6.: ROI – Example — Eigene Darstellung

2.5.4.1. Lifecycle

Eine ROI entsteht durch die Ausgabe des Detector-Modells, das eine Bounding Box sowie Schlüsselpunkte liefert. Aus diesen werden Position, Orientierung und Größe der initialen ROI berechnet. In den folgenden Frames wird innerhalb der ROI das Landmark-Modell ausgeführt. Bei ausreichender Konfidenz des Landmark-Modells werden Landmarken in Bildkoordinaten zurückgerechnet und daraus eine aktualisierte ROI (Position, Orientierung, Größe) bestimmt. Dieser Vorgang wird jeden Frame wiederholt. Fällt die Konfidenz unter den definierten Schwellwert, wird die ROI verworfen und der Lifecycle endet.

2.6. Datenaugmentation und Modellquantisierung

2.6.1. Datenaugmentation

Definition und Ziel: Datenaugmentation bezeichnet die künstliche Vermehrung eines Trainingsdatensatzes durch Transformationen der vorhandenen Daten (Pattern Recognition and Machine Learning, n.d.). Der Zweck besteht darin, die Robustheit und Generalisierungsfähigkeit des Modells zu erhöhen, ohne zusätzliche Daten erfassen zu müssen. Für eingebettete Systeme mit begrenztem Speicher ist eine effiziente Datennutzung besonders relevant.

Transformationsverfahren: Im vorliegenden Projekt werden folgende Augmentationen eingesetzt:

- **Spiegelung (Horizontal Flip):** Die Hand wird horizontal gespiegelt. Dies simuliert Unterschiede zwischen linker und rechter Hand sowie Variationen in der Handhaltung.
- **Zufällige Translation:** Die Landmarks werden um einen zufälligen Betrag in x- und y-Richtung verschoben, um unterschiedliche Handpositionen im Kamerabild zu simulieren.
- **Zufälliger Zoom:** Die Landmarks werden um einen Faktor zwischen $0,5\times$ und $1,5\times$ skaliert, um unterschiedliche Handgrößen und Abstände zur Kamera abzubilden.
- **Jitter:** Zufälliges Rauschen wird zu den Koordinaten hinzugefügt, um natürliche Schwankungen in der Landmark-Erkennung zu simulieren.

Implementierung: Die Augmentationspipeline (AugmentationPipeline) wendet diese Transformationen sequenziell auf die extrahierten Landmarks an und erzeugt pro Original-Datensatz mehrere Variationen. Dadurch wird der effektive Datensatz um einen Faktor von typisch 5–10× vermehrt.

2.6.2. Modellquantisierung

Neuronale Netze verwenden bei Training und Inferenz üblicherweise Fließkommazahlen um eine möglichst hohe Genauigkeit zu erreichen (Float32, 4 Byte pro Wert). Auf ressourcenbeschränkten eingebetteten Plattformen ist dies sowohl speichermäßig als auch rechnerisch ineffizient [7]. Die Quantisierung reduziert die Genauigkeit der Gewichte und Aktivierungen auf Ganzzahlen (typisch INT8, 1 Byte pro Wert).

Tabelle 2.6.: Vergleich Float32 vs. INT8

Eigenschaft	Float32	INT8
Speicher pro Gewicht	4 Byte	1 Byte
Speicherersparnis	–	75 %
Rechengeschwindigkeit	Standard	Deutlich schneller (NPU-nativ)
Genauigkeit	Hoch	Leicht reduziert (hinnehmbar)

- **Scale und Zero-Point:** Die Quantisierung bildet den Float32-Wertebereich auf einen Integer-Bereich ab. Für die affine Quantisierung gilt [21]:

$$q = \text{round}\left(\frac{r}{s}\right) + z$$

wobei r den realen Float-Wert, s den Skalierungsfaktor, z den Zero-Point und q den quantisierten Integer-Wert darstellt. Die Rückrechnung erfolgt über:

$$r = s \cdot (q - z)$$

- **Post-Training-Quantisierung (PTQ):** Bei der Post-Training-Quantisierung wird ein bereits trainiertes Float32-Modell in ein INT8-Modell konvertiert, ohne erneutes Training [21]. Dafür wird ein Repräsentativer Datensatz verwendet. Es handelt sich um eine Stichprobe der Trainingsdaten, anhand derer die Aktivierungsbereiche der Schichten analysiert und die optimalen Skalierungsfaktoren bestimmt werden.
- **Genauigkeitsverluste:** Die Reduktion von Float32 auf INT8 kann zu einem leichten Rückgang der Modellgenauigkeit führen. In der Praxis ist dieser Verlust bei geeigneter Kalibrierung (Repräsentativer Datensatz) jedoch gering (typisch ≈ 2 % Accuracy-Verlust) und für die meisten Anwendungsfälle akzeptabel.

2.7. Edge AI und Neuronale Beschleuniger (NPU)

2.7.1. Edge AI

Edge AI bezeichnet die Ausführung von KI-Inferenzen direkt auf dem Endgerät (engl. edge device) anstelle einer Übertragung der Daten an einen Cloud-Server [22]. Die Mikrocontroller erfassen Daten über Sensoren und verarbeiten diese lokal mithilfe von Modellen des maschinellen Lernens. Die wesentlichen Vorteile gegenüber Cloud-basierten Ansätzen sind:

- **Geringe Latenz:** Die Verarbeitung erfolgt lokal, ohne Netzwerkübertragungszeit. Für Echtzeitanwendungen wie die Gesture-Erkennung ist dies essentiell.
- **Datenschutz:** Biometrische Daten (Kamerabilder der Hand) verlassen das Gerät nicht.
- **Offline-Fähigkeit:** Das System funktioniert ohne Internetverbindung.
- **Reduzierte Bandbreite:** Es müssen keine Bilder übertragen werden.

Die Haupt-Herausforderung liegt in der Ressourcenbeschränkung: Speicher, Rechenleistung und Energieverbrauch sind auf eingebetteten Plattformen stark limitiert. Ein KI-Modell, das auf einem Server mit Gigabytes an Speicher und leistungsstarken GPUs trainiert wurde, muss auf dem Mikrocontroller mit wenigen Kilobytes an SRAM und einem dedizierten Beschleuniger betrieben werden [5].

2.7.2. Neuronale Beschleuniger (NPU)

Der Neural-Art-Beschleuniger ist eine parametrierbare und zur Laufzeit rekonfigurierbare Neuronale Verarbeitungseinheit (NPU). Diese dedizierte Hardware-Einheit dient der Inferenz INT8 quantisierter Convolutional Neural Networks (CNN) [12, s. 1011 ff.]. Im Gegensatz zur allgemeinen CPU, die nur einen Bruchteil ihrer Rechenleistung für Matrix-Operationen nutzt, führt die NPU Matrix-Multiplikationen, Vektoroperationen und spezialisierte CNN Instruktionen mit hoher Parallelität aus.

Die NPU arbeitet mit einer eigenen Speicherhierarchie: Die AXISRAM-Banken dienen als Puffer für Gewichte und Aktivierungen, sodass die NPU unabhängig von der CPU auf Daten zugreifen kann (vgl. Abschnitt 2.1).

2.7.3. ST Edge Core CLI

ST Edge Core ist ein Command Line Interface (CLI) das genutzt wird, um Convolutional Neural Networks (CNN) im ONNX (QDQ) oder TF-Lite-Format in ein für die NPU-Plattform passendes Format zu überführen. Der ST Neural-ART-Compiler ist als Back-End in die ST Edge Core CLI integriert und führt alle Optimierungen, Graph-Planung und Code-Generierung offline durch. Es wird keine interpretierende Engine auf dem eingebetteten System ausgeführt.

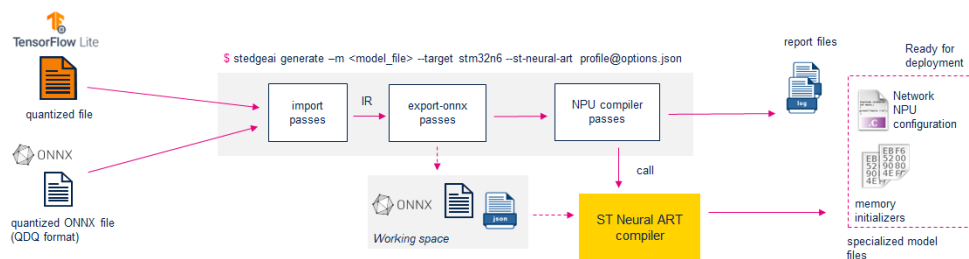


Abbildung 2.7.: NPU – Quantized Model Conversion to NPU Executables [23]

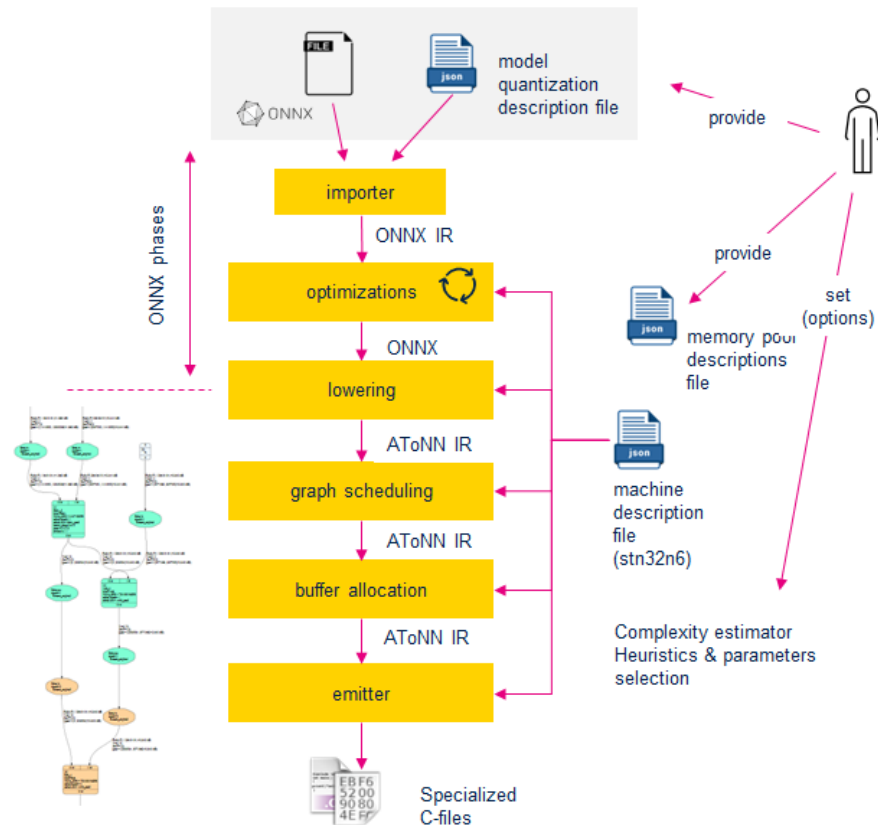


Abbildung 2.8.: NPU – Compilation [23]

Modelle werden in NPU-Epochen unterteilt die in die verfügbaren Hardwareressourcen abbilden. Die Umsetzung der Transformation geschieht mittels Mapping von TF-Lite/ONNX-Befehlen auf Epochen. Folgende Epochen werden unterschieden:

Tabelle 2.7.: NPU-Epochen-Typen

Epochen-Typ	Beschreibung
HW-Epochen	Operatoren vollständig auf NPU-Hardware abgebildet. Die MCU-Arbeitslast beträgt ca. 10–15 % der Inferenzzeit.
SW-Epochen	Operatoren an Host delegiert (Keine Beschleunigung).
Hybrid-Epochen	Teilweise Software, teilweise hardwareunterstützt über vordefinierte HW-Epochen.
Meta-Epoche	Menge von HW-Epochen, gesteuert über Befehlsstrom via Epochen-Controller.

Epochen werden als atomare Operationen in fester Reihenfolge ausgeführt, um Datenabhängigkeiten zu gewährleisten. Zwischen Epochen wird kein interner NPU-Hardware-Zustand erhalten. Zwischenergebnisse werden im externen Speicher abgelegt.

3. Stand der Technik

Dieses Kapitel befasst sich mit bereits bestehenden Ansätzen auf welche die Arbeit basiert oder fortführt. Es werden Bestehende Ansätze benannt und aktuelle Embedded-AI-Lösungen vorgestellt. Zudem wird die Abgrenzung des Projekts beschrieben.

3.1. Bestehende Ansätze zur Erkennung von Handzeichen

Die Forschung zur automatischen Erkennung und Übersetzung von Gebärdensprache verfolgt im Wesentlichen drei zentrale Ansätze, die sich in der Art der Datenerfassung unterscheiden: kamerabasierte Systeme, datenhandschuhbasierte Systeme und armbandbasierte Systeme (EMG/sEMG). Jeder Ansatz nutzt unterschiedliche Sensormodalitäten, um Handkonfigurationen und Bewegungen zu erfassen. Diese werden anschließend durch Machine-Learning-Modelle in Buchstaben, Wörter oder Sätze übersetzt (Rastgoo et al., 2021).

Kamerabasierte Systeme gehören zu den am weitesten verbreiteten Ansätzen der Gebärdenspracherkennung. Eine RGB-Kamera erfasst eine Gebärdende Person, und mithilfe von Bildverarbeitung und Deep Learning werden die Hände, Finger und Gesichtszüge segmentiert und klassifiziert. Moderne Frameworks wie MediaPipe Hands ermöglichen eine präzise 3D-Handpunkterkennung auf Standardkameras (Lugaresi et al., n.d.). Darüber hinaus werden Tiefensensoren (z.B. Microsoft Kinect, Intel RealSense) eingesetzt, um zusätzliche Tiefeninformationen für eine robustere Erkennung zu gewinnen (Rastgoo et al., 2021).

Datenhandschuh-basierte Systeme verwenden physische Handschuhe mit eingebetteten Sensoren, um die Bewegung und Position der Hände direkt zu messen. Typischerweise kommen Biegesensoren (Flex Sensors) zur Erfassung der Fingerkrümmung sowie inertielle Messeinheiten (IMU) zur Bestimmung der Handorientierung zum Einsatz. Zhou et al. (2020) entwickelten einen dehnbaren Sensoren-Handschuh auf Basis triboelektrischer Nanogeneratoren (TENG), der 660 amerikanische Gebärdensprachezeichen übersetzen kann.

Armbandbasierte Systeme erfassen die elektrische Aktivität der Muskeln an dem Unterarm mittels Oberflächen-Elektromyographie (sEMG). Das verbreitetste Gerät in der Forschung ist das Myo-Armband von Thalmic Labs, das acht sEMG-Sensoren sowie eine inertielle Messeinheit (IMU) integriert. Kürzliche Arbeiten zeigen, dass sEMG-basierte Systeme in Kombination mit Deep-Learning-Verfahren wie Random Forest oder LSTM-Netzwerken Genauigkeiten von über 99% bei der Erkennung einzelner Gebärden erreichen können (Umut & Kumdereli, 2024).

3.2. Kamerabasierte Fingeralphabet- und Gebärdenerkennung

Kamerabasierte Erkennungssysteme verwenden meist ein nicht ressourcenbeschränktes System (z.B. PC oder Server), auf dem die Inferenz durchgeführt und das Ergebnis ausgegeben wird. Die Erkennungspipeline besteht typischerweise aus einer Verkettung von Machine-Learning-Modellen. Ein Palm-Detection-Modell erkennt die Handfläche im Bild. Ein Hand-Landmark-Modell bestimmt die Fingerpositionen. Ein abschließendes Sign-Language-Modell übersetzt die erkannten Handkonfigurationen in Buchstaben oder Wörter (Lugaresi et al., n.d.).

3.3. Aktuelle Embedded-AI-Lösungen

Aktuelle Embedded-AI-Lösungen lassen sich grundsätzlich in zwei Kategorien einteilen: Single-Board Computer (SBC) und Mikrocontroller (MCU). Beide Plattfortmentypen werden für die Inferenz neuronaler Netze eingesetzt, unterscheiden sich jedoch hinsichtlich ihrer Ressourcenverfügbarkeit, Programmierungsumgebung und des Entwicklungsansatzes erheblich.

Single-Board Computer als dominierende Plattform: Die verbreitetste Plattform in der Forschung zur eingebetteten Gebärdenspracherkennung ist der Raspberry Pi. Verschiedene Studien nutzten den Raspberry Pi 4 und 5 als Zielplattform für Echtzeitinferenz von CNN-basierten Modellen (Sharma et al., 2024; Mohalkar et al., 2025). Der Raspberry Pi bietet als Linux-basierter SBC eine umfangreiche Softwarebibliothek, native Kameraunterstützung (CSI-Schnittstelle) sowie Zugriff auf etablierte Frameworks wie TensorFlow Lite, PyTorch und ONNX. Ein wesentlicher Vorteil ist die Programmierung in Python, was die Entwicklung und Prototypisierung erheblich beschleunigt. Allerdings ist Python als interpretierte Sprache in Hinblick auf Speicherverbrauch und Ausführungsgeschwindigkeit limitiert, was bei ressourcenkritischen Inferenzpipelines zu Engpässen führen kann. Um dieses Defizit auszugleichen, werden zunehmend dedizierte Beschleuniger in Form von Zusatzmodulen eingesetzt. Der Raspberry Pi 5 AI HAT+ (Hailo-8L NPU) ergänzt den SBC um einen Neural Processing Unit mit 13 TOPS, sodass auch komplexere Modelle in Echtzeit betrieben werden können.

Mikrocontroller als Nische: Mikrocontroller-basierte Ansätze sind in der Forschung zur Gebärdenspracherkennung selten. Der Grund liegt in der starken Ressourcenbeschränkung: MCUs verfügen über begrenzten SRAM (typisch 256 KB bis 1 MB), eine eingeschränkte Taktfrequenz und arbeiten ohne Betriebssystem (Bare-Metal). Klassische Plattformen wie der STM32H7 oder ESP32 bieten keine dedizierte Hardwarebeschleunigung für neuronale Netze, wodurch die Inferenz auf der CPU erfolgen muss und die Latenz/Laufzeit der auszuführenden Modelle erheblich steigt. In der Praxis werden MCU-Ansätze hauptsächlich für stark vereinfachte Modelle oder für die reine Datenerfassung genutzt, während die eigentliche Inferenz auf einem leistungstärkeren System ausgelagert wird (Abadade et al., 2023).

Eine Nische stellt die Verwendung von MCUs mit integriertem Neural Processing Unit (NPU) dar. Der STM32N6 von STMicroelectronics integriert einen dedizierten NPU (Neural-ART Accelerator) mit 6 GOPS und ermöglicht die Inferenz quantisierter INT8-Modelle auf einem Mikrocontroller. Damit schließt die Plattform die Lücke zwischen ressourcenbeschränkten MCUs und leistungstarken SBCs. In der Forschung wird der STM32N6 jedoch bisher kaum eingesetzt, was zum Teil an der erstmaligen Verfügbarkeit seit 2024 sowie an der Komplexität der Bare-Metal-Programmierung liegt.

3.4. Einordnung der Arbeit und Abgrenzung der eigenen Arbeit

Die vorliegende Arbeit ordnet sich im Bereich kamerabasierter Embedded-AI-Lösungen ein. Einerseits wird die Zielplattform STM32N6570 Discovery Kit von STMicroelectronics Bare-Metal programmiert, andererseits wird ein Sign-Language-Modell entworfen, das Hand-Landmarks dem Fingeralphabet zuordnet.

Die Machine-Learning-Grundlage der Arbeit bildet ein von MediaPipe bereitgestelltes Hand-Landmark- und Palm-Detection-Modell, das Bare-Metal auf der STM32N6570DK Plattform verwendet wird. Die bestehende Inferenzpipeline (Palm-, Hand-Landmark-Modell) wird um ein selbstentwickeltes Sign-Language-Modell erweitert, das die erkannten Handlandmarks dem Fingeralphabet zuordnet. Für das Sign-Language-Modell wird ein Datensatz erstellt. Aus dem Datensatz werden die Hand-Landmarks extrahiert und augmentiert. Die entstandenen Daten werden zum Training des Modells verwendet. Das Modell wird anschließend INT8-quantisiert und als Modellbinary auf der Zielplattform integriert.

Hardwareseitig bilden NPU-Treiber und STEdgeAI CLI von STM die Grundlage für die weitere Entwicklung. Alle anderen Hardware-Treiber (LTDC, CSI, DCMIPP, DMA2D, etc.) werden Bare-Metal eigens entwickelt.

4. Methodik und Entwicklungsprozess

4.1. Vorgehensweise bei der Literatur- und Technologierecherche

Die Literatur- und Technologierecherche dient dazu, die fachlichen Grundlagen der Arbeit zu erarbeiten, bestehende Lösungsansätze zu untersuchen und geeignete Technologien für die Umsetzung des Systems auszuwählen. Die Auswahl des Rechercheansatzes richtet sich nach dem Zweck der Literaturauswertung und der zugrunde liegenden Fragestellung [24, s. 334]. Die Literaturrecherche bildete in dieser Arbeit keinen eigenständigen Untersuchungsgegenstand, sondern diente der fachlichen Fundierung und der Begleitung des Entwicklungsprozesses. Daher wurde keine systematische Literaturstudie mit dem Anspruch auf eine vollständige Erfassung und Synthese aller Veröffentlichungen durchgeführt. Stattdessen erfolgte eine zielgerichtete und entwicklungsbegleitende Recherche, die sich an den jeweils zu lösenden fachlichen und technischen Fragestellungen orientierte.

Die wissenschaftliche Literaturrecherche umfasste insbesondere die Themengebiete Fingeralphabet- und Handzeichenerkennung, kamerabasierte Handdetektion, Handlandmark-Erkennung, Klassifikation mittels neuronaler Netze, Datenaugmentation, Modellquantisierung und Embedded AI. Für die Suche wurden zunächst Mind-Maps und Wortwolken mit den genannten Begriffen erstellt und in verschiedenen Kombinationen in wissenschaftliche Datenbanken (u. a. Google Scholar und BibDiscover) eingegeben und recherchiert. Zu den verwendeten Suchbegriffen gehörten unter anderem:

- Hand Gesture Classification
- Embedded AI / AI on the Edge
- Quantisierung von neuronalen Netzen
- TensorFlow / TensorFlow Lite
- Konvertierung von Modellen (TFLite)
- Fingeralphabet
- Sign Language Recognition
- Gebärdensprache
- Modeltraining / Modelquantization Parameters

Bei der Auswahl der Quellen wurden deren fachliche Relevanz, Aktualität und Nachvollziehbarkeit berücksichtigt. Bevorzugt wurden Fachartikel und wissenschaftliche Ausarbeitungen. Für grundlegende Informationen zum deutschen Fingeralphabet wurden Veröffentlichungen und Informationsangebote von Verbänden und Organisationen herangezogen. Die ausgewählten Quellen wurden mithilfe der Literaturverwaltungssoftware Zotero erfasst und verwaltet.

Die Technologierecherche konzentrierte sich überwiegend auf die verwendete Plattform, das STM32N6570 Discovery Kit, sowie auf die für die Implementierung erforderlichen Hard- und Softwarekomponenten. Als Quellen dienten insbesondere die Referenz- und Benutzerhandbücher, Datenblätter, Anwendungshinweise und Beispielprojekte des Herstellers STMicroelectronics. Ergänzend wurden technische Informationen zu den verwendeten Peripherien und Schnittstellen recherchiert. Dazu gehörten unter anderem die Digital Camera Memory Interface Pixel Pipeline (DCMIPP), das Camera Serial Interface (CSI), der LCD-TFT Display Controller (LTDC) und das Extended Serial Peripheral Interface (XSPI).

Die Recherche wurde parallel zum Entwicklungsprozess fortgeführt. Technische Fragestellungen, die während der Implementierung oder Integration auftraten, führten zu einer gezielten Vertiefung einzelner Themenbereiche. Die daraus gewonnenen Erkenntnisse flossen unmittelbar in die Auswahl und Anpassung der Modelle, die Entwicklung der Verarbeitungspipeline sowie die hardwarenahe Umsetzung auf der Zielplattform ein.

4.2. Iterativer Entwicklungs- und Integrationsprozess

Eine vollständig vorausgeplante und sequenzielle Entwicklung war für das Gesamtsystem nur eingeschränkt geeignet, da sich viele technische Anforderungen und Wechselwirkungen erst bei der praktischen Integration auf der Ziellplattform beurteilen ließen. Dies betraf insbesondere das Zusammenspiel zwischen Kamerapipeline, neuronalen Netzen, Speicherverwaltung, Ablaufsteuerung und grafischer Ausgabe. Die Entwicklung erfolgte daher iterativ und prototypisch. In jeder Iteration wurde ein abgegrenzter Funktionsumfang umgesetzt, in den bestehenden Entwicklungsstand integriert und anschließend praktisch überprüft. Die dabei gewonnenen Erkenntnisse bildeten die Grundlage für nachfolgende Anpassungen und weitere Entwicklungsschritte [25, s. 70].

Zu Beginn einer Iteration wurden die nächsten Entwicklungsschritte gemeinsam festgelegt und auf konkrete Aufgaben verteilt. Da das Projekt als Gruppenarbeit durchgeführt wurde, erfolgte eine personenspezifische Aufteilung in die bereits in Kapitel 1.3.2 genannten Aufgabenbereiche. Während des Entwicklungsprozesses fanden regelmäßige Absprachen zum aktuellen Stand, zu aufgetretenen Problemen und zum weiteren Vorgehen statt. Dabei wurden Ergebnisse der vorherigen Iteration bewertet und die Ziele der nächsten Iteration festgelegt. Falls sich eine Lösung als ungeeignet erwies oder neue technische Einschränkungen erkannt wurden, wurde das weitere Vorgehen entsprechend angepasst. Der Entwicklungsprozess wies damit agile Elemente auf, ohne einem festgelegten agilen Vorgehensmodell wie Scrum vollständig zu folgen [26].

Zur Versionsverwaltung des Quellcodes wurde Git eingesetzt. Git ermöglicht es, Änderungen am Quellcode zu dokumentieren, unterschiedliche Entwicklungsstände zu verwalten und bei Bedarf auf frühere Versionen zurückzugreifen. Die gemeinsame Entwicklung erfolgte über ein Remote-Repository [27, s. 27ff.]. Neue Funktionen und umfangreiche Änderungen wurden überwiegend in separaten Feature-Banches entwickelt. Dadurch konnten Änderungen zunächst unabhängig vom stabilen Entwicklungsstand umgesetzt und getestet werden. Gleichzeitig erleichterte die Trennung der Entwicklungsstände die Zuordnung und Überprüfung einzelner Änderungen. Nach erfolgreicher Überprüfung wurden die Branches in den gemeinsamen Hauptzweig integriert. Insbesondere bei großen Änderungen und vor der Zusammenführung der Branches wurden gegenseitige Code-Reviews durchgeführt. Diese dienten dazu, Fehler zu erkennen, die Verständlichkeit des Quellcodes zu verbessern und mögliche Auswirkungen auf andere Komponenten zu berücksichtigen. Festgestellte Probleme wurden vor oder während der Integration behoben.[27, s. 191ff.].

Durch die Kombination aus iterativer Prototypentwicklung, klarer Aufgabenverteilung, regelmäßiger Abstimmung und gemeinsamer Versionsverwaltung konnte der Entwicklungsfortschritt fortlaufend nachvollzogen werden. Gleichzeitig blieb das Vorgehen flexibel genug, um auf technische Schwierigkeiten und neue Erkenntnisse während der Entwicklung reagieren zu können.

4.3. Vorgehen bei der Modellentwicklung

Die Entwicklung des Klassifikationsmodells erfolgte in mehreren aufeinander aufbauenden Schritten. Zunächst wurden die zu erkennenden Handzeichen festgelegt und ein eigener Datensatz erstellt. Die aufgenommenen Daten wurden anschließend aufbereitet und in Trainings-, Validierungs- und Testdaten unterteilt. Auf Grundlage des Datensatzes wurden verschiedene Modellvarianten entwickelt und trainiert. Dabei wurden sowohl die Modellarchitektur als auch die Trainingsparameter schrittweise angepasst. Zur Erweiterung des Datensatzes kamen unterschiedliche Augmentationsverfahren zum Einsatz. Diese sollten die Variabilität der Trainingsdaten erhöhen und das Modell gegenüber unterschiedlichen Ausführungen der Handzeichen robuster machen.

Das Modell mit der besten Erkennungsgenauigkeit wurde ausgewählt und abschließend für die eingebettete Plattform verfügbar gemacht und integriert. Dabei wurde geprüft, ob und in welchem Umfang sich die Erkennungsleistung durch die Anpassungen verändert hatte.

4.4. Verifikation- und Evaluationskonzept

Die Verifikation und Evaluation des entwickelten Systems erfolgten sowohl entwicklungsbegleitend als auch abschließend anhand der zuvor festgelegten Anforderungen. Im Rahmen dieser Arbeit bezeichnet die Verifikation die Überprüfung, ob die einzelnen Komponenten und Funktionen entsprechend

ihrer Festlegung umgesetzt wurden. Die Evaluation dient dagegen der abschließenden Bewertung des integrierten Gesamtsystems anhand der definierten Anforderungen und Abnahmekriterien. Auf diese Weise wurden sowohl die technische Funktionsfähigkeit als auch die Eignung des entstandenen Prototyps für den vorgesehenen Anwendungsfall untersucht.

Die Verifikation wurde entsprechend dem Entwicklungsprozess iterativ durchgeführt. Neu entwickelte Funktionen wurden zunächst unabhängig getestet, bevor sie in den gemeinsamen Entwicklungsstand übernommen wurden. Nach der Integration wurde kontrolliert, ob die hinzugefügte Funktion wie vorgesehen arbeitete und bereits vorhandene Funktionen weiterhin verfügbar waren. Letzteres entspricht dem Grundgedanken des Regressionstestens, mit dem unbeabsichtigte Auswirkungen von Änderungen auf bereits bestehende Funktionen erkannt werden sollen [28, s. 30].

Die Überprüfung erfolgte auf mehreren Ebenen. Zunächst wurden einzelne Komponenten unabhängig voneinander getestet. Anschließend wurde im Rahmen von Integrationstests untersucht, ob die Komponenten und ihre Schnittstellen korrekt zusammenarbeiteten. Abschließend wurde das Gesamtsystem unter festgelegten Versuchsbedingungen geprüft. Dieses Vorgehen orientierte sich an den im ISTQB-Lehrplan beschriebenen Teststufen, die unter anderem Komponenten-, Integrations- und Systemtests unterscheiden [28, s. 28-30].

Die Evaluation des Gesamtsystems wurde anhand der funktionalen und nichtfunktionalen Anforderungen sowie der daraus abgeleiteten Abnahmekriterien vorgenommen. Jeder zu bewertenden Anforderung wurden geeignete Versuche oder Bewertungsparameter zugeordnet. Dadurch konnte nachvollziehbar dargestellt werden, welche Anforderungen erfüllt, teilweise erfüllt oder nicht erfüllt wurden. Die konkreten Versuchsbedingungen, Bewertungsparameter und Ergebnisse werden im Evaluationskapitel beschrieben. Auf Grundlage dieser Ergebnisse wird beurteilt, inwieweit die Zielsetzung der Arbeit erreicht wurde und welche Einschränkungen beim entwickelten Prototyp bestehen.

5. Anforderungsanalyse

Die Anforderungsanalyse bildet die Grundlage für den Entwurf, die Implementierung und die anschließende Bewertung des zu entwickelnden Systems. Die festgelegten Anforderungen beeinflussen den Aufbau des Systems und bestimmen, welche Funktionen und Eigenschaften bei der Entwicklung besonders berücksichtigt werden müssen [29, s. 53 ff.].

Zur Ermittlung und Beschreibung der Anforderungen werden zunächst das vorgesehene Anwendungsszenario und die daraus abgeleiteten Anwendungsfälle betrachtet. Darauf aufbauend werden die funktionalen und nichtfunktionalen Anforderungen sowie die durch die Zielplattform vorgegebenen Randbedingungen festgelegt. Abschließend werden überprüfbare Abnahmekriterien definiert, die im weiteren Verlauf der Arbeit als Grundlage für die Verifikation und Evaluation des Gesamtsystems dienen.

5.1. Anwendungsszenario

Das entwickelte System ist als prototypischer Fingeralphabet-Trainer vorgesehen. Es soll Personen beim selbständigen Üben ausgewählter statischer Handzeichen des deutschen Fingeralphabets unterstützen. Die nutzende Person positioniert hierzu eine Hand im Erfassungsbereich der Kamera und führt ein Zeichen aus. Das Kamerabild wird vom System aufgenommen und lokal verarbeitet.

Die Verarbeitung erfolgt in mehreren aufeinanderfolgenden Schritten. Zunächst wird die Hand im Kamerabild lokalisiert. Anschließend werden charakteristische Handlandmarks bestimmt und für die Klassifikation aufbereitet. Auf Grundlage dieser Merkmale ordnet das Klassifikationsmodell die dargestellte Handform einer der unterstützten Zeichenklassen zu. Das ermittelte Ergebnis wird über das Display des Systems ausgegeben und ermöglicht eine unmittelbare Rückmeldung zur dargestellten Handform. Das System ist für die Verwendung durch jeweils eine Person und die Erfassung einer einzelnen Hand vorgesehen. Die Handzeichen werden nacheinander und innerhalb eines geeigneten Abstands zur Kamera dargestellt. Die Verarbeitung setzt ausreichende Lichtverhältnisse und eine weitgehend freie Sicht auf die Hand voraus. Verdeckte Hände, mehrere gleichzeitig dargestellte Hände sowie dynamische Bewegungsabläufe gehören nicht zum vorgesehenen Anwendungsszenario.

Sämtliche Verarbeitungsschritte werden auf einem eingebetteten System ausgeführt. Eine Verbindung zu einem externen Rechner oder einem Cloud-Dienst ist für den Betrieb des fertigen Prototyps nicht vorgesehen. Abbildung 5.1 stellt das Anwendungsszenario des Fingeralphabet-Trainers als Use-Case-Diagramm dar [29, s. 61 ff.]. Die übende Person startet das System und zeigt ein Handzeichen im Erfassungsbereich der Kamera. Der Anwendungsfall „Handzeichen erkennen“ umfasst die Erfassung des Kamerabildes, die Lokalisierung der Hand, die Bestimmung der Handlandmarks und die Klassifikation des dargestellten Zeichens. Das ermittelte Erkennungsergebnis wird anschließend auf dem Display ausgegeben.

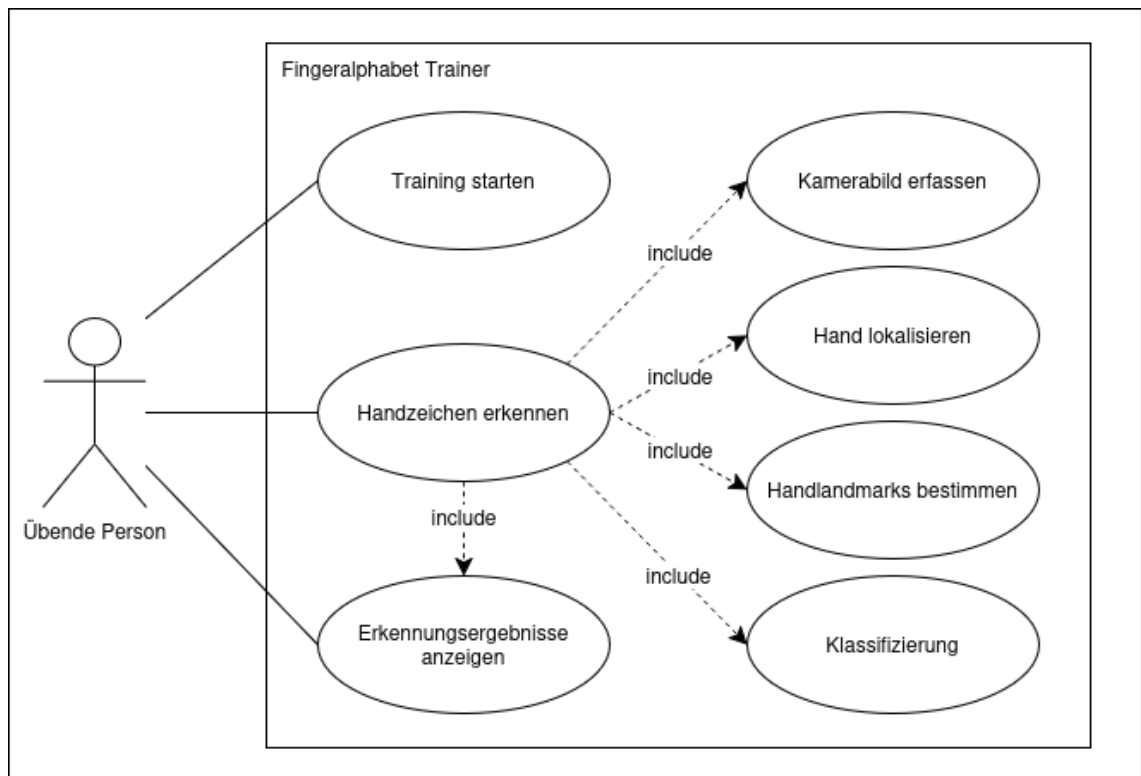


Abbildung 5.1.: Use-Case Diagramm Fingeralphabet Trainer

5.2. Anforderungen & Randbedingungen

Aus dem zuvor festgelegten Anwendungsszenario und dem zugehörigen Use-Case-Diagramm lassen sich die folgenden funktionalen, nichtfunktionalen und technischen Anforderungen, bzw. Randbedingungen ableiten [29, s. 73 ff.].

5.2.1. Funktionale Anforderungen

Tabelle 5.1.: Funktionale Anforderungen

ID	Funktionale Anforderung
FA_01	Das System muss über die angeschlossene Kamera laufend Bilddaten erhalten können.
FA_02	Das System muss eine einzelne Hand im Kamerabild lokalisieren können.
FA_03	Aus einer lokalisierten Hand muss eine Region of Interest gebildet und fortlaufend aktualisiert werden können.
FA_04	Das System muss innerhalb der Region of Interest die charakteristischen Handlandmarks bestimmen können.
FA_05	Die Handlandmarks müssen für die Fingeralphabet-Klassifizierung vorbereitet werden können.
FA_06	Das System muss anhand der aufbereiteten Handlandmarks das statische Handzeichen einer Klassifizierung ausgeben können.
FA_07	Das Klassifizierungs- bzw. Inferenzergebnis muss auf dem Display ausgegeben werden können.
FA_08	Das System muss die Erkennung, bzw. Inferenz wiederholt ausführen und Änderungen von Handzeichen erkennen können.
FA_09	Das System muss mit Situationen umgehen können, in denen keine Hand oder kein eindeutig klassifizierbares Handzeichen erkannt wird.

Fortsetzung auf nächster Seite

ID	Funktionale Anforderung
FA_10	Das System muss die für die Bewertung relevanten Ergebnisse und Laufzeitinformationen erfassen oder zur Auswertung bereitstellen können.

5.2.2. Nichtfunktionale Anforderungen

Tabelle 5.2.: Nichtfunktionale Anforderungen

ID	Nichtfunktionale Anforderung
NFA_01	Das System muss unter den festgelegten Bedingungen eine Erkennungsgenauigkeit von mindestens 80 % erreichen.
NFA_02	Das System muss eine Bild- bzw. Verarbeitungsrate von mindestens 15 Bildern pro Sekunde erreichen.
NFA_03	Die Zeitspanne von der Bereitstellung eines Kamerabildes bis zur Ausgabe des zugehörigen Klassifikationsergebnisses darf 80 ms nicht überschreiten.
NFA_04	Das System muss mindestens 20 Minuten kontinuierlich arbeiten können, ohne abzustürzen oder einen Neustart zu erfordern.
NFA_05	Das System muss die Hand unter den festgelegten Testbedingungen auch bei Positionsänderungen und veränderten Lichtverhältnissen ausreichend zuverlässig lokalisieren und verfolgen können.
NFA_06	Das Erkennungsergebnis muss auf dem Display eindeutig und für die nutzende Person verständlich dargestellt werden.
NFA_07	Der Quellcode soll modular aufgebaut und ausreichend dokumentiert sein, sodass einzelne Komponenten unabhängig angepasst und getestet werden können.
NFA_08	Der Speicherbedarf der Modelle und Laufzeitdaten darf die hierfür vorgesehenen internen und externen Speicherbereiche nicht überschreiten.

5.2.3. Technische Anforderungen

Tabelle 5.3.: Technische Anforderungen

ID	Technische Anforderung
TA_01	Als Zielplattform muss das STM32N6570 Discovery Kit verwendet werden.
TA_02	Das System muss die vollständige Verarbeitung vom Kamerabild bis zur Ausgabe des Klassifikationsergebnisses lokal auf der eingebetteten Zielplattform ausführen können.
TA_03	Die Kamerabilder müssen über die an das Board angeschlossene Kamera erfasst und innerhalb der eingebetteten Plattform verarbeitet werden. Dabei handelt es sich um das IMX-335 Sensormodul.
TA_04	Die Ausgabe des Kamerabildes und des Erkennungsergebnisses muss über das angeschlossene Display erfolgen. Dabei handelt es sich um das MB1860B.
TA_05	Die Kamerapipeline wird mit den vom Board bereitgestellten Schnittstellen implementiert. Das sind unter anderem Camera Serial Interface (CSI), Digital Camera Interface Pipe Processing (DCMIPP), LCD-TFT Display Controller (LTDC).
TA_06	Bildsignalanpassungen wie z. B. Gammakorrektur, Blacklevelkorrektur, RGB-Conversion müssen embedded mittels DCMIPP erfolgen.
TA_07	Hardwarezugriffe sollen in abgegrenzten Treiber- und Schnittstellenmodulen gekapselt werden. Soweit für die erforderliche Kontrolle und Ausführungsgeschwindigkeit notwendig, sollen Low-Layer-Treiber oder direkte Registerzugriffe verwendet werden.
TA_08	Die verwendeten neuronalen Netze müssen in das INT8-Zahlenformat quantisiert und mit ST Edge AI Core für die Zielplattform konvertiert werden.
TA_09	Die von der Hardware unterstützten Operationen der neuronalen Netze müssen auf dem integrierten ST Neural-ART Accelerator ausgeführt werden. Nicht unterstützte Operationen dürfen auf dem Hauptprozessor ausgeführt werden.

Fortsetzung auf nächster Seite

ID	Technische Anforderung
TA_10	Die Gewichte der neuronalen Netze müssen im externen Octo-SPI-NOR-Flash gespeichert werden.
TA_11	Die Bilddaten und die hierfür erforderlichen Puffer müssen in der externen Hexadeca-SPI-PSRAM abgelegt werden.
TA_12	Der reguläre Betrieb des Prototyps muss ohne aktive Verbindung zu einem externen Rechner oder einem Cloud-Dienst möglich sein.

5.2.4. Randbedingungen

Tabelle 5.4.: Randbedingungen

ID	Randbedingung
RB_01	Als Zielplattform steht ausschließlich das STM32N6570 Discovery Kit mit dem Mikrocontroller STM32N657X0H3Q zur Verfügung.
RB_02	Der Mikrocontroller verfügt nur über begrenzten internen Arbeits- und Programmspeicher. Größere Datenbestände müssen daher in den externen Speichern abgelegt werden.
RB_03	Als externer nichtflüchtiger Speicher steht ein 1-Gbit-Octo-SPI-NOR-Flash zur Verfügung. Zur Zwischenspeicherung größerer Datenmengen steht eine 256-Mbit-Hexadeca-SPI-PSRAM zur Verfügung.
RB_04	Zur Beschleunigung der neuronalen Netze steht der integrierte ST Neural-ART Accelerator zur Verfügung. Dessen Nutzung ist auf die unterstützten Operationen und Datenformate beschränkt.
RB_05	Die Konfiguration und Konvertierung der neuronalen Netze erfolgt mit den Werkzeugen der ST Edge AI Suite beziehungsweise mit ST Edge AI Core.
RB_06	Für die Handlokalisierung und Handlandmark-Erkennung werden bereits bestehende Modelle verwendet. Lediglich das nachgelagerte Klassifikationsmodell wird im Rahmen dieser Arbeit neu entwickelt.
RB_07	Modellgewichte müssen vor Inbetriebnahme manuell oder per Skript auf den NOR-Flash geladen werden.

5.3. Abnahmekriterien

Die Abnahmekriterien werden aus den zuvor festgelegten Anforderungen abgeleitet. Sie legen fest, anhand welcher Prüfungen und Grenzwerte beurteilt wird, ob das entwickelte Gesamtsystem die jeweiligen Anforderungen erfüllt. Inhaltlich zusammengehörige Anforderungen können dabei gemeinsam überprüft werden [29, s. 85 ff.].

Tabelle 5.5.: Abnahmekriterien

ID	Zugeordnete Anforderungen	Abnahmekriterium	Prüfverfahren
AK_01	FA_01	Das System erfasst fortlaufend gültige Kamerabilder.	Visuelle Überprüfung des Displays.
AK_02	FA_02 bis FA_04	Eine Hand wird lokalisiert, eine ROI wird gebildet und die Handlandmarks werden bestimmt.	Prüfung anhand ausgewählter Bilder und Visualisierung der ROI, bzw. der Landmarks.
AK_03	FA_05, FA_06, NFA_01	Das Klassifikationsmodell erreicht eine Gesamtgenauigkeit von mindestens 80 %.	Auswertung der Modellvorhersagen und Erstellung einer Konfusionsmatrix.
AK_04	FA_07, NFA_06	Das erkannte Zeichen wird eindeutig und lesbar auf dem Display dargestellt.	Visuelle Prüfung der Anzeige für alle unterstützten Zeichenklassen.

Fortsetzung auf nächster Seite

ID	Zugeordnete Anforderungen	Abnahmekriterium	Prüfverfahren
AK_05	FA_08	Mehrere nacheinander dargestellte Handzeichen werden ohne Neustart erkannt und die Anzeige wird aktualisiert.	Durchführung einer festgelegten Zeichenfolge und Vergleich.
AK_06	FA_09	Wird keine Hand erkannt, gibt das System kein gültiges Klassifikationsergebnis aus.	Tests ohne Hand sowie mit nicht unterstützten Handformen.
AK_07	FA_10	Erkennungs-, Laufzeit- und Speicherinformationen können für die Evaluation ausgelesen werden.	Überprüfung der Messwerte über die Debug-Schnittstelle.
AK_08	NFA_02	Das Gesamtsystem verarbeitet mindestens 15 Kamerabilder pro Sekunde.	Zählung der verarbeiteten Bilder und Berechnung der mittleren Verarbeitungsrate.
AK_09	NFA_03	Die Zeitspanne bis zum Klassifikationsergebnis beträgt höchstens 80 ms.	Zeitmessung an festgelegten Punkten der Verarbeitungskette.
AK_10	NFA_04	Das System arbeitet mindestens 20 Minuten ohne Absturz.	Kontinuierlicher Dauerlauf-test.
AK_11	NFA_05	Die Hand wird in mindestens 70 % der Bilder erfolgreich lokalisiert.	Durchführung und Auswertung definierter Testszenarien.
AK_12	TA_05, TA_06	Die Kameradaten werden über die festgelegten Schnittstellen verarbeitet.	Überprüfung der Peripheriekonfiguration und Register-einstellungen.
AK_13	TA_10, TA_11	Modellgewichte im NOR-Flash, Bilddaten und Puffer im PSRAM abgelegt.	Überprüfung der Speicher-adressen mittels Linker-Map-Datei.
AK_14	TA_01, TA_12	Das Gesamtsystem wird auf dem STM32N6570 Discovery Kit ausgeführt.	Durchführung sämtlicher Tests auf der Zielplattform.
AK_15	NFA_07, TA_07	Der Quellcode ist modular aufgebaut und ohne HAL implementiert.	Code-Review.
AK_16	TA_08	Die Modelle sind quantisiert und für die NPU optimiert.	Überprüfung der Modelle und des Quellcodes.

6. Systementwurf

Auf Grundlage der zuvor definierten Anforderungen wird in diesem Kapitel der Entwurf des Gesamtsystems beschrieben. Zunächst wird ein übergeordnetes Systemkonzept vorgestellt, das die zentralen Komponenten und deren Zusammenwirken darstellt. Anschließend werden die Funktionen den verfügbaren Hardwarekomponenten der Zielplattform zugeordnet. Abschließend wird die mehrstufige KI-Verarbeitungskette von der Bilderfassung bis zur Klassifikation des dargestellten Fingeralphabetzeichens beschrieben.

Der Systementwurf legt damit die grundlegende Architektur und die Schnittstellen zwischen den einzelnen Komponenten fest. Die konkrete technische Umsetzung, darunter die Konfiguration der Peripherie, die Speicherverwaltung sowie die Vor- und Nachverarbeitung der Modelldaten, wird im nachfolgenden Implementierungskapitel behandelt.

6.1. Systemkonzept

Ausgehend von den zuvor definierten Anforderungen wurde ein Systemkonzept entwickelt, das die funktionalen Komponenten des Gesamtsystems und den zwischen ihnen stattfindenden Datenfluss beschreibt. Das Konzept stellt zunächst eine weitgehend von der konkreten Zielplattform unabhängige Betrachtung des Systems dar. Dadurch können die erforderlichen Verarbeitungsschritte festgelegt werden, bevor diese im Hardware- und Softwareentwurf konkreten Komponenten der Zielplattform zugeordnet werden. Das in Abbildung 6.1 dargestellte Blockdiagramm veranschaulicht den Datenfluss von der Aufnahme eines Kamerabildes bis zur Ausgabe des erkannten Fingeralphabetzeichens auf dem Display.

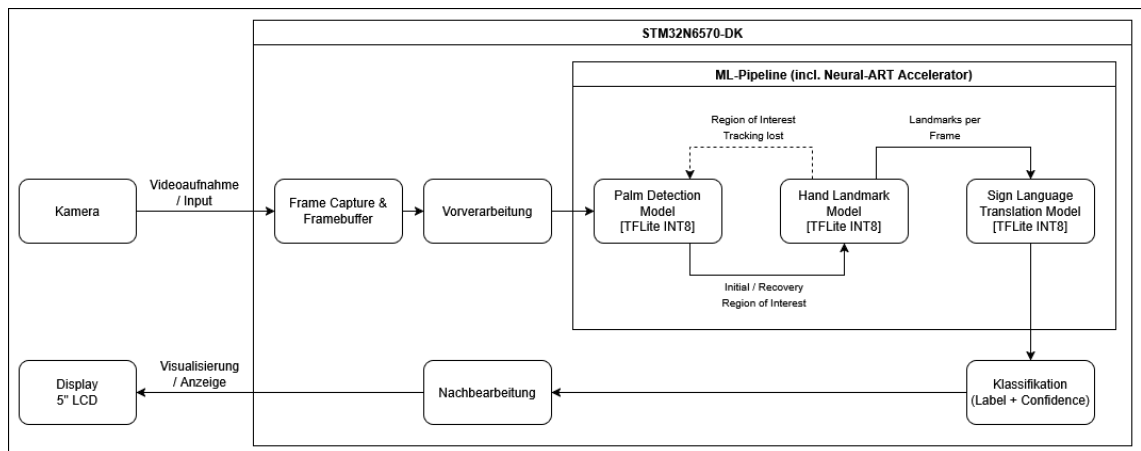


Abbildung 6.1.: Blockdiagramm des Systemkonzepts

Die Kamera erfasst kontinuierlich Bilddaten und überträgt diese an das System. Die aufgenommenen Bilder werden zunächst in einem Bildpuffer zwischengespeichert. Das Bild wird für die Palmdetektion skaliert und anschließend der Verarbeitungskette zur Verfügung gestellt. Innerhalb der Verarbeitungskette kommen drei aufeinander aufbauende neuronale Netze zum Einsatz.

Im ersten Verarbeitungsschritt wird mithilfe des Handdetektionsmodells geprüft, ob sich eine Hand im Kamerabild befindet. Für eine erkannte Hand werden deren Position und räumliche Ausdehnung bestimmt. Aus diesen Informationen wird eine Region of Interest gebildet, welche den für die weiteren Verarbeitungsschritte relevanten Bildbereich enthält. Die Region of Interest wird entsprechend der erkannten Handposition aktualisiert und für die nachfolgende Landmark-Erkennung aufbereitet.

Das zweite Modell bestimmt innerhalb der Region of Interest die charakteristischen Handlandmarks. Die ermittelten Koordinaten werden anschließend nachverarbeitet und in eine für das Klassifikationsmodell geeignete Form überführt. Hierzu gehören insbesondere die Anpassung des Datenformats sowie die für die Klassifikation erforderliche Normalisierung der Landmark-Koordinaten. Außerdem

übernimmt das Modell das Tracking und aktualisiert die Region of Interest, sobald sich die Hand bewegt. Sobald die Hand verloren geht, übernimmt wieder das erste Modell und erzeugt eine neue Region of Interest.

Das dritte Modell klassifiziert die aufbereiteten Handlandmarks und ordnet die dargestellte Handform einer der unterstützten Zeichenklassen zu. Das Klassifikationsergebnis wird gemeinsam mit dem Kamerabild auf dem Display ausgegeben. Wird keine Hand erkannt oder erreicht die Klassifikation nicht die erforderliche Konfidenz, wird kein gültiges Zeichen ausgegeben.

Die Verarbeitungskette wird zyklisch ausgeführt, sodass Änderungen der Handposition und des dargestellten Handzeichens fortlaufend berücksichtigt werden können. Eine übergeordnete Ablaufsteuerung koordiniert dabei die Bildaufnahme, die Ausführung der Modelle, die Vor- und Nachverarbeitung sowie die Aktualisierung der grafischen Ausgabe.

6.2. Hardwarekonzept

Das Hardwarekonzept überführt die im Systemkonzept festgelegten Funktionen auf die Komponenten der eingebetteten Zielplattform. Als zentrale Verarbeitungseinheit wird das STM32N6570 Discovery Kit verwendet. Neben dem Mikrocontroller umfasst der Hardwareaufbau das Kameramodul MB1854B mit dem Bildsensor IMX335, das Displaymodul MB1860B sowie die auf dem Board vorhandenen externen Speicher [4].

Die Kameradaten werden über die CSI-Schnittstelle empfangen und mithilfe der Digital Camera Interface Pixel Pipeline verarbeitet. Die DCMIPP übernimmt hardwarenahe Verarbeitungsschritte wie die Anpassung des Bildformats, die Skalierung und die Aufbereitung der Kamerabilder für die weitere Verarbeitung. Die erzeugten Bilddaten und die erforderlichen Bildpuffer werden aufgrund ihres Umfangs in der externen PSRAM abgelegt. Die Gewichte der neuronalen Netze werden im externen NOR-Flash gespeichert. Für die Ausführung der unterstützten Modelloperationen wird der integrierte ST Neural-ART Accelerator verwendet. Der Hauptprozessor übernimmt insbesondere die Ablaufsteuerung, die Vor- und Nachverarbeitung der Modelldaten sowie die Konfiguration und Koordination der eingesetzten Peripheriekomponenten.

Die grafische Ausgabe erfolgt über den LCD-TFT Display Controller. Dieser liest die darzustellenden Bild- und Ergebnisdaten aus den zugeordneten Speicherbereichen und überträgt sie an das angeschlossene Display. Dadurch können das aktuelle Kamerabild und das ermittelte Klassifikationsergebnis gemeinsam dargestellt werden.

6.3. KI-Verarbeitungskette

Die KI-Verarbeitungskette besteht aus drei aufeinander aufbauenden Modellen zur Handdetektion, Handlandmark-Erkennung und Klassifikation des dargestellten Fingeralphabetzeichens. Abbildung 6.2 zeigt die einzelnen Verarbeitungsschritte sowie die beiden durch die DCMIPP bereitgestellten Bildpfade.

Ausgangspunkt der Verarbeitungskette ist das von der Kamera aufgenommene Bild mit einer Auflösung von 2592×1944 Pixeln. Die DCMIPP stellt den Kameradatenstrom parallel über zwei getrennte Verarbeitungspfade bereit. Pipe 1 erzeugt ein Bild mit einer Auflösung von 800×480 Pixeln, das für die Displayausgabe und als Ausgangsbild der Handlandmark-Erkennung verwendet wird. Pipe 2 erzeugt parallel dazu ein Bild mit einer Auflösung von 192×192 Pixeln für die Handdetektion. Das Handdetektionsmodell verarbeitet das von Pipe 2 bereitgestellte Bild und ermittelt die Position einer vorhandenen Hand. Im Rahmen der Nachverarbeitung wird aus dem Modellergebnis eine initiale Region of Interest bestimmt. Da die Handdetektion und die Landmark-Erkennung unterschiedliche Bildpfade verwenden, wird die Region of Interest anschließend in das Koordinatensystem des von Pipe 1 erzeugten Bildes übertragen. Auf Grundlage der transformierten Region of Interest wird aus dem Bild von Pipe 1 ein Ausschnitt mit einer Auflösung von 224×224 Pixeln erzeugt. Dieser Ausschnitt wird für das Handlandmark-Modell aufbereitet und anschließend an das Modell übergeben. Das Handlandmark-Modell bestimmt die Positionen der charakteristischen Landmark-Punkte innerhalb der Handregion.

Nach einer erfolgreichen Landmark-Erkennung wird die Region of Interest anhand der Modellausgabe aktualisiert. Für die Verarbeitung des folgenden Kamerabildes wird dadurch nicht erneut die gesamte

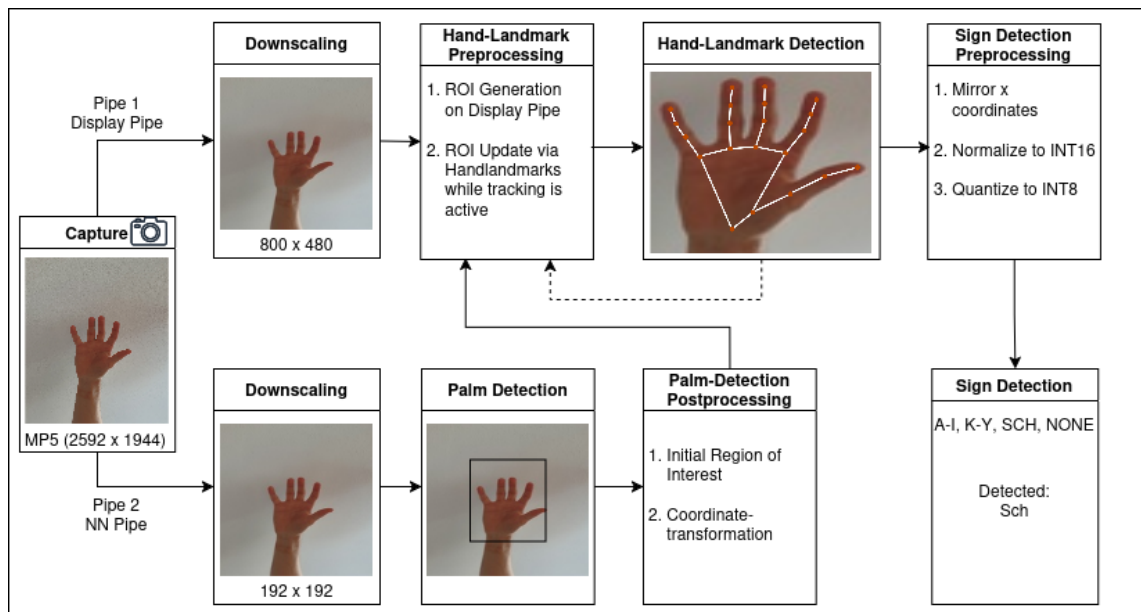


Abbildung 6.2.: KI-Verarbeitungskette zur Erkennung der Fingeralphabetzeichen

Handdetektion ausgeführt. Stattdessen wird aus dem nächsten Bild von Pipe 1 unmittelbar eine neue Landmark-Eingabe auf Grundlage der aktualisierten Region erzeugt. Dieser Rückkopplungspfad bildet das Landmark-Tracking. Kann das Handlandmark-Modell über mehrere aufeinanderfolgende Bilder keine gültige Hand mehr bestimmen, wird das Tracking beendet. Das System wechselt daraufhin zurück zur Handdetektion, um erneut eine initiale Region of Interest im vollständigen Bild zu bestimmen. Die Handdetektion und das Landmark-Tracking bilden somit zwei aufeinander abgestimmte Zustände der Verarbeitungskette.

Für die anschließende Zeichenklassifikation werden ausschließlich die ermittelten Landmark-Koordinaten verwendet. Vor der Übergabe an das Klassifikationsmodell werden diese in eine einheitliche Darstellung überführt und an das erforderliche Eingabeformat angepasst. Dazu gehören die Berücksichtigung der erkannten Händigkeit, die Normalisierung der Koordinaten und deren Überführung in das vom quantisierten Klassifikationsmodell verwendete Zahlenformat. Die konkrete Berechnung dieser Schritte wird im Implementierungskapitel beschrieben.

Das Klassifikationsmodell ordnet die aufbereiteten Landmark-Koordinaten einer der für den Prototyp festgelegten Zeichenklassen zu. Zusätzlich wird die Klasse NONE berücksichtigt, wenn keine der unterstützten Handformen vorliegt. Das ermittelte Klassifikationsergebnis wird anschließend an die grafische Ausgabe übergeben und auf dem Display dargestellt.

6.4. Speicher- und Datenflusskonzept

Die im Gesamtsystem anfallenden Daten unterscheiden sich deutlich hinsichtlich ihres Umfangs, ihrer Lebensdauer und ihrer Verwendung. Aus diesem Grund werden die Kamerabilder, Modellgewichte, Modellein- und -ausgaben sowie Steuerungsdaten auf unterschiedliche Speicherbereiche der Zielplattform verteilt. Abbildung 6.3 zeigt die vorgesehene Zuordnung der Daten zu den verfügbaren Speichern sowie den Datenfluss zwischen den beteiligten Hardware- und Softwarekomponenten.

Die großformatigen Kamera- und Displaydaten werden im externen PSRAM gespeichert. Für das von Pipe 1 erzeugte Displaybild stehen vier Hintergrundbildpuffer zur Verfügung. Die mehreren Puffer ermöglichen es, die Bilderfassung, die KI-Verarbeitung, das Einzeichnen der Erkennungsergebnisse und die Displayausgabe auf voneinander getrennten Bildständen durchzuführen. Die Hintergrundbildpuffer werden durch den LTDC als Layer 1 ausgegeben. Zusätzlich greift der Hauptprozessor auf diese Puffer zu, um den Bildausschnitt für das Handlandmark-Modell zu erzeugen und die Region of Interest sowie die Handlandmarks direkt in das Displaybild einzuzeichnen.

Tabelle 6.1.: Zuordnung der Systemdaten zu den Speicherbereichen

Speicherbereich	Gespeicherte Daten
Externer PSRAM	Displaypuffer, Eingabebilder der Handdetektion
Externer NOR-Flash	Modellgewichte aller Modelle
NPU-erreichbarer Arbeitsspeicher (AXI-RAM)	Eingabe-, Aktivierungs- und Ausgabepuffer der neuronalen Netze
Interner SRAM	Steuerungsdaten, Zustandsinformationen, Landmark-Daten und Klassifikationsergebnisse

6.5. Ablauf vom Kamerabild bis zum Erkennungsergebnis

In den vorherigen Abschnitten wurden zunächst die Komponenten des Gesamtsystems, deren Zuordnung zur Zielplattform sowie die KI-Verarbeitungskette und der zwischen den Komponenten stattfindende Datenfluss beschrieben. Darauf aufbauend stellt dieser Abschnitt den zyklischen Ablauf und die dabei auftretenden Entscheidungen von der Erfassung eines Kamerabildes bis zur Ausgabe des Klassifikationsergebnisses dar. Der Ablauf berücksichtigt insbesondere den Wechsel zwischen der erstmaligen Handdetektion und dem anschließenden Landmark-Tracking.

Abbildung 6.4 führt die zuvor beschriebenen Komponenten und Verarbeitungsschritte in einem gemeinsamen Ablauf zusammen. Nach der Erfassung wird das Kamerabild durch die DCMIPP verarbeitet und über Pipe 1 für die Display- und Landmark-Verarbeitung sowie über Pipe 2 für die Handdetektion bereitgestellt.

Ist noch kein Landmark-Tracking aktiv, wird zunächst die Handdetektion auf dem von Pipe 2 bereitgestellten Bild ausgeführt. Wird keine Hand erkannt, beginnt die Verarbeitung mit dem nächsten Kamerabild erneut. Bei einer erfolgreichen Erkennung wird eine initiale Region of Interest bestimmt und in das Koordinatensystem von Pipe 1 übertragen. Aus dem dort vorliegenden Displaybild wird anschließend ein 224×224 Pixel großer Ausschnitt für das Landmark-Modell erzeugt. Bei aktivem Tracking kann hierfür unmittelbar die anhand des vorherigen Bildes aktualisierte Region verwendet werden.

Nach der Ausführung des Landmark-Modells wird geprüft, ob eine gültige Modellausgabe vorliegt. Ist dies der Fall, wird die Region of Interest für den nächsten Durchlauf aktualisiert, das Tracking aktiviert beziehungsweise fortgeführt und der Verlustzähler zurückgesetzt. Anschließend werden die Landmark-Koordinaten entsprechend der in Abschnitt 6.3 beschriebenen KI-Verarbeitungskette aufbereitet und durch das Klassifikationsmodell einem Fingeralphabetzeichen zugeordnet. Das ermittelte Ergebnis wird nachverarbeitet und über die in Abschnitt 6.4 beschriebenen Anzeige- und Speicherbereiche für die grafische Ausgabe bereitgestellt.

Liegt keine gültige Landmark-Ausgabe vor, wird der Verlustzähler erhöht. Solange der festgelegte Grenzwert nicht erreicht ist, bleibt das Tracking aktiv und die Landmark-Erkennung wird mit dem nächsten Kamerabild erneut ausgeführt. Wird der Grenzwert erreicht, wird das Tracking zurückgesetzt, sodass im folgenden Durchlauf erneut eine Handdetektion auf dem vollständigen Eingabebild erfolgt.

Unabhängig vom jeweiligen Verarbeitungspfad kehrt der Ablauf anschließend zur Erfassung eines neuen Kamerabildes zurück. Dadurch entsteht ein kontinuierlicher Verarbeitungszyklus, der sowohl die erstmalige Lokalisierung einer Hand als auch deren anschließende Verfolgung und die wiederholte Klassifikation dargestellter Fingeralphabetzeichen ermöglicht.

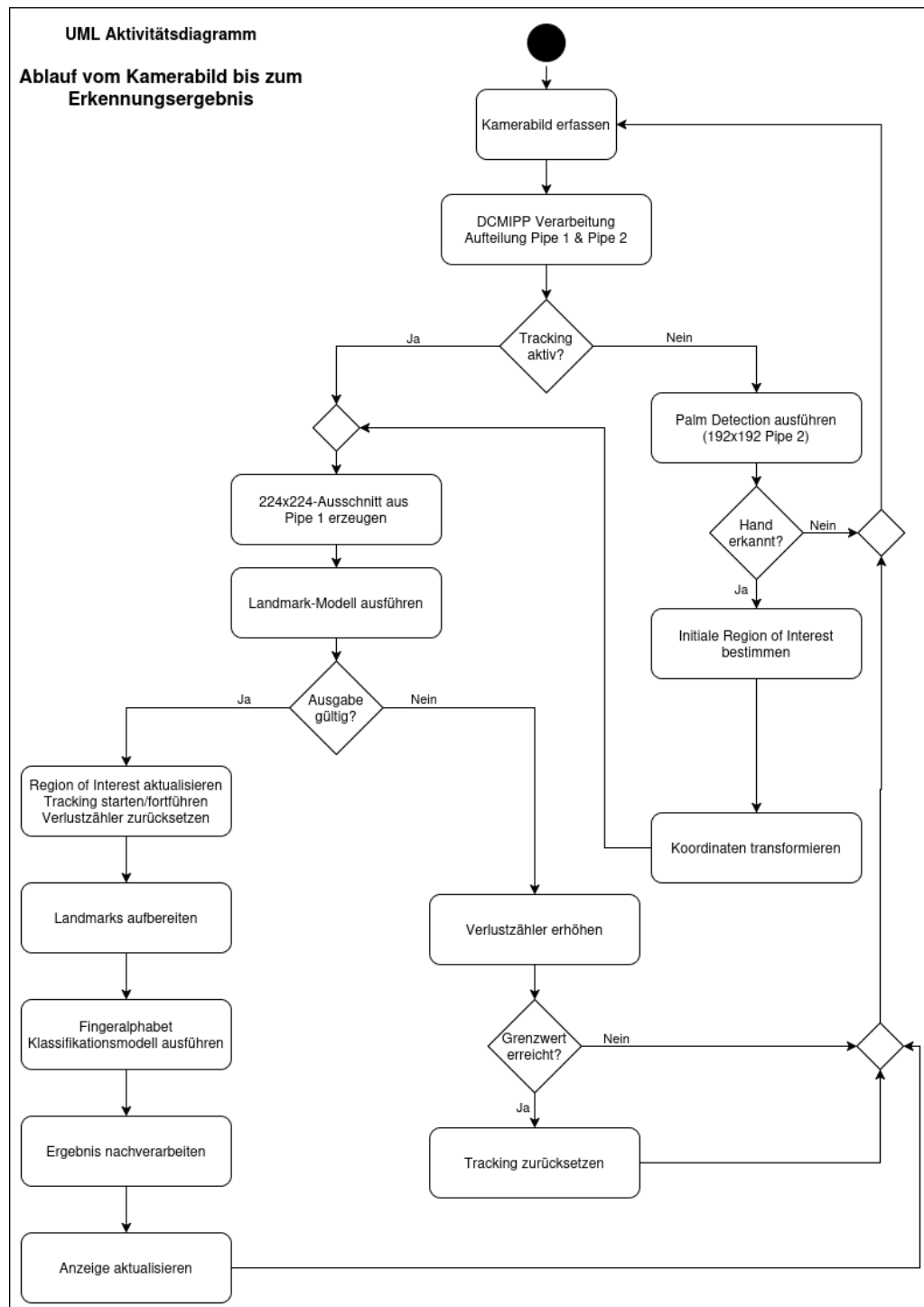


Abbildung 6.4.: UML Aktivitätsdiagramm – Ablauf vom Kamerabild bis zum Erkennungsergebnis

7. Entwicklung und Implementation

7.1. Aufbau und Aufbereitung des Datensatzes

7.1.1. Rohvideos

Der Datensatz basiert auf einer eigens erstellten Sammlung von Videoaufnahmen der deutsche Gebärdensprache (DGS) und wurde speziell für die Klassifikation statischer Fingeralphabetzeichen konzipiert. Die Rohvideos wurden manuell aufgezeichnet und liegen im MP4-Format vor.

Die Benennungskonvention der Videodateien lautet `alph_[og|fw]_[Label] [Nummer].mp4`, wobei `[og|fw]` den aufzeichnenden protokolliert, `[Label]` das jeweilige Fingersprachezeichen und `[Nummer]` die fortlaufende Nummer innerhalb eines Labels spezifiziert. Die Speicherung erfolgt im Verzeichnis `ml_pipeline/resources`.

Der Datensatz umfasst 226 Rohvideos, die auf 26 Klassen verteilt sind. Die Klassenauswahl beschränkt sich auf die 24 statischen Buchstaben A–Y (ohne J und Z, da diese dynamische Handbewegungen erfordern und somit mit dem statischen Ansatz des Klassifikators nicht kompatibel sind), das Sonderzeichen SCH sowie die Leerklasse NONE, die die Unbestimmtheit des Ergebnisses darstellt. Die Anzahl der Originalaufnahmen pro Klasse variiert zwischen 3 und 5, wobei 16 Klassen mit je 5, 8 Klassen mit je 4 und 2 Klassen (F, NONE) mit je 3 Aufnahmen vertreten sind.

7.1.2. Frame-Extraktion und Handdetektion

#TODO MediaPipe Hand/Palm Detection

Zur Verarbeitung der Rohvideos wird jeder Einzelrahmen (Frame) mittels OpenCV Videocapture einzeln extrahiert und ohne vorherige Vorfilterung der Erkennungspipeline zugeführt.

Die Hand-Erkennung und Landmark-Extraktion erfolgt mithilfe des MediaPipe-Handdetektors (Lugaresi et al., 2019) in der Konfiguration für statische Bildanalyse mit einer maximalen Detektionskapazität von 2 Händen pro Frame und einer Mindestkonfidenz von 70. Pro Hand werden 21 Landmarks extrahiert, die den Handgelenkspunkt (WRIST) sowie die Gelenke der fünf Finger (Daumen: CMC, MCP, IP, TIP; Indexfinger: MCP, PIP, DIP, TIP; Mittelfinger: MCP, PIP, DIP, TIP; Ringfinger: MCP, PIP, DIP, TIP; Kleinfinger: MCP, PIP, DIP, TIP) umfassen.

7.1.3. Augmentation

Die Augmentationspipeline wendet sequenziell Transformationen auf die extrahierten Landmarks an. Jeder Schritt kann mehrere Variationen erzeugen, die kumulativ kombiniert werden:

Die Extrahierten Gesten Daten werden im folgenden schritt mittels Augmentation vervielfacht und gegen Overfitting angepasst. Die Augmentationspipeline bietet die folgenden Augmentationsoptionen: - Translate: addiert einen Offset auf die Gesture Daten - Spiegelung: Spiegelt gesten horizontal - Scale: skaliert die gesten - Jitter: Fügt der Geste zufällige individuelle Fehler hinzu - Zoom: Zufälliger zoom der Geste - DropFrames: Skips frames in order to simulate missing Frames Für die verwendeten Optionen sind: - Mirror - 2xTranslation - 5xZoom

#TODO Pickle Inspect Gegenüberstellung der Gesten nach Augmentation

7.1.4. Normalisierung und Interpolation

Die extrahierten Landmarks werden für eine effiziente und positionsinvariante Darstellung normalisiert. Hierbei werden alle Landmark-Koordinaten relativ zum Handgelenk positioniert und in den int16-Zahlenbereich (-32.768 bis 32.767) umgerechnet. Die Landmark-Daten werden als Dictionary mit den 21 benannten Schlüsseln und den zugehörigen $[x, y, z]$ -Koordinaten abgebildet. Zusätzlich wird die absolute Handgelenkposition (`wrist_pos`) in Skalierung auf den int16-Bereich sowie die Bounding-Box der Hand in Pixelkoordinaten (`hand_area`) gespeichert. Die absoluten Landmark-Positionen in Pixelkoordinaten (`landmark_pos`) werden ebenfalls erfasst.

Für eine einheitliche Darstellung aller Sequenzen erfolgt eine FPS-Normalisierung auf 60 FPS mittels linearer Interpolation zwischen den Frames. Frames, in denen eine Hand nicht erkannt wird, werden durch leere Handstrukturen mit der Sentinel-Koordinate $[-100, -100]$ repräsentiert und bei der Interpolation als Sprungpunkte behandelt.

7.1.5. Datenstrukturen und Speicherung

Die verarbeiteten Geste- und Handdaten werden in den folgenden Datenstrukturen abgebildet:

Gesture – Repräsentation einer vollständigen Geste-Sequenz:

- `label`: Klasse der Geste (str)
- `fps`: Aktuelle Bildwiederholrate (float)
- `frames`: Liste von Frames, wobei jeder Frame eine Liste von zwei Hand-Objekten [`left`, `right`] enthält

Hand – Darstellung einer einzelnen Hand in einem Frame (frozen dataclass):

- `left_hand`: Seitenzugehörigkeit (bool)
- `wrist_pos`: Absolute Handgelenkposition in int16-Skalierung [`x`, `y`]
- `landmarks`: Dictionary mit 21 benannten Landmarks relativ zum Handgelenk
- `hand_area`: Bounding-Box in Pixelkoordinaten (`x`, `y`, `width`, `height`)
- `landmark_pos`: Absolute Landmark-Positionen in Pixelkoordinaten ($21 \times [\text{x}, \text{y}]$)

Die verarbeiteten Geste-Daten werden im Pickle-Format (`.pkl`) abgelegt. Der Dateiname folgt dem Muster `{Label}-{Augmentierungstyp}-{UUID}.pkl`, wobei die UUID die ersten vier Hexadezimalzeichen eines UUID4 darstellt. Die Speicherung erfolgt im Verzeichnis `resources/gestures/`. Die Verarbeitung der Videodateien und die Speicherung der Ergebnisse werden mittels Multiprocessing unter Auslastung von 80 % der verfügbaren CPU-Kerne parallelisiert.

7.2. Entwicklung und Training des Klassifikationsmodells

7.2.1. Problemdefinition

Das zu entwickelnde Modell soll die Fähigkeit besitzen, Gesten bestimmten Gruppen zuzuordnen. Eine Gruppe entspricht dabei immer einer Geste. Diese Einteilung von Datenpunkten (Gesten) in vordefinierte Klassen (Gebärdenalphabet-Zeichen) beschreibt Klassifikationsmodelle. Für jede Geste sollen die Wahrscheinlichkeiten für die zuzuordnenden Klassen ausgegeben werden.

Es liegt keine Binäreklassifikation vor, da mehr als zwei Klassen zu unterscheiden sind. Mit den 26 Klassen (NONE, A–Y ohne J und Z, SCH) handelt es sich um eine Multiklassen-Klassifikation.

7.2.2. Datenanalyse

Input-Format: Der Klassifikator empfängt einen 88-dimensionalen Feature-Vektor. Dieser setzt sich nach Ausgabe des Hand-Landmark Modells aus 44 Werten für die linke Hand ($22 \text{ Gelenke} \times 2 \text{ Koordinaten}$) und 44 Werten für die rechte Hand ($22 \text{ Gelenke} \times 2 \text{ Koordinaten}$) zusammen.

Die 22 Gelenke pro Hand umfassen die 21 MediaPipe-Landmarks sowie die absolute Handgelenkposition. Koordinaten werden wrist-relativ normalisiert und auf den int16-Bereich (-32.768 bis 32.767) skaliert.

Klassenverteilung: Der Datensatz umfasst 26 Klassen: NONE (Index 0) für Keine Hand erkannt, A–Y (Indizes 1–24) für 24 statische Buchstaben (J und Z ausgeschlossen, da diese dynamische Handbewegungen erfordern) sowie SCH (Index 25) als Sonderzeichen. Die Klassenverteilung ist leicht ungleichmäßig (16 Klassen mit 185, 8 Klassen mit 148, NONE mit 111, F mit 110 Dateien).

Normalisierung: Alle Koordinaten werden durch den Maximalwert des int16-Bereichs (POS.MAX = 32.767) dividiert, sodass die Werte im Bereich $[-1, 1]$ liegen.

7.2.3. Modellauswahl

Für die Modellauswahl wurden verschiedene Ansätze evaluiert, wobei die Anforderungen des Embedded-Systems (schnelle Inferenz, kleine Modellgröße) und die Input-Struktur (88D-feature-Vektor aus MediaPipe-Landmarks) im Vordergrund standen.

Ein mehrschichtiges Perceptron (MLP) wurde als geeignetstes Modell identifiziert. Die Input-Struktur war dafür ausschlaggebend. Die MediaPipe wandelt die räumliche Information der Hand in strukturierte Landmarks um. Daher ist eine convolutionale Verarbeitung (CNN) unnötig. Studien belegen, dass CNNs bei Landmark-basiertem Input keine signifikanten Genauigkeitsvorteile gegenüber MLPs aufweisen. (Fritz, 2025)

Random Forest und SVM wurden ebenfalls in Betracht gezogen, erreichen jedoch in der Literatur geringere Genauigkeiten (70–75% bzw. 61–76%) bei vergleichbaren Tasks. Zudem fehlen beiden Modellen nativ kalibrierte Wahrscheinlichkeiten: Random Forest gibt nur harte Klassifikationen aus, während SVM ein zusätzliches Platt-Scaling für Wahrscheinlichkeiten erfordert. (Rahman et al., 2025). Es ist anzumerken das SVM und Random forest auf einer anderen Datenbasis (Photoplethysmography (PPG)) trainiert worden sind.

Das MLP erfüllt die Anforderungen an Ressourcenbeschränkungen und Echtzeitfähigkeit. Die Softmax-Ausgabeschicht liefert direkt kalibrierte Klassenwahrscheinlichkeiten, die für die nachfolgende Verarbeitung benötigt werden. Während direkte Quantisierung zu Genauigkeitsverlust führen kann, zeigen Studien, dass INT8-Quantisierung bei MLP-Modellen mit geeigneten Techniken (LayerNorm, Kalibrierung) Verluste von unter 1% erreicht. Ein akzeptabler Kompromiss für den Embedded-Einsatz (blogdevelopspot, 2025).

7.2.4. Modellarchitektur

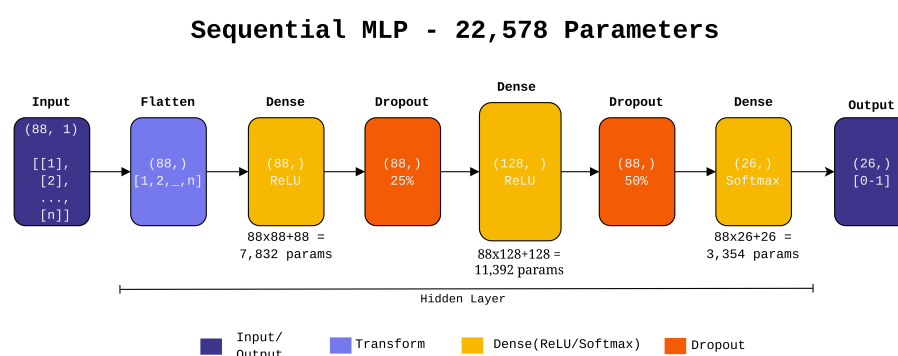


Abbildung 7.1.: Architektur des Multi-Layer Perceptrons — Eigene Darstellung

Das Klassifikationsmodell setzt sich aus sieben Schichten zusammen, die sequenziell eine direkte Informationsflussrichtung von der Eingabe zur Ausgabe aufweisen. Die Architektur folgt dem Prinzip der schrittweisen Transformation und Abstraktion der Eingabemerkmale. Nachdem der Input-Tensor in eine eindimensionale Darstellung überführt wurde, transformiert die erste Dense-Schicht die Rohmerkmale in einen repräsentativen Feature-Raum. Die nachfolgende Dense-Schicht erweitert diesen Raum, um

komplexere Inter-Feature-Beziehungen modellieren zu können. Zwischen den Dense-Schichten werden Dropout-Schichten eingebunden um Zufall in das Modell zu bringen, um Overfitting zu verhindern. Die finale Dense-Schicht projiziert die gelernten Repräsentationen auf die 26 Ausgabeklassen.

Der Input-Layer empfängt einen Tensor der Form $(88, 1)$. Dieser Tensor repräsentiert die normalisierten Merkmale zweier Hände mit je 44 Werten. Pro Hand sind 21 MediaPipe-Hand-Landmarks jeweils als (x, y) -Koordinate sowie die absolute Handgelenkposition ebenfalls als (x, y) . Alle Koordinaten sind Handgelenksrelativ normalisiert und auf den int15-Bereich skaliert, sodass der Input-Wertebereich $[-1, 1]$ beträgt.

Der Flatten-Layer überführt den mehrdimensionalen Input-Tensor in einen eindimensionalen Vektor der Länge 88. Dense-Schichten erwarten als Eingabe einen flachen Vektor, sodass die räumliche Struktur des Tensors (2 Hände x 44 Werte) für die nachfolgende Gewichtsmatrix aufgelöst werden kann.

Der erste Dense-Layer mit 88 Neuronen und ReLU-Aktivierung führt per-Feature-Transformationen durch. Das bedeutet durch die Gewichtsmatrix (88×88) und den Bias-Vektor wird jedes Eingabemerkmal einzeln transformiert. Die ReLU-Aktivierung ($f(x) = \max(0, x)$) fügt Nicht-Linearität hinzu und ermöglicht es dem Modell, nicht-triviale Muster in den Landmark-Daten zu erkennen. (Sharma et al., 2020) Die Wahl von 88 Neuronen hält die Kapazität zunächst auf gleicher Dimension wie der Input und zwingt das Modell, die Merkmale in einem äquivalenten Raum abzubilden, bevor eine Dimensionserweiterung erfolgt.

Der Dropout-Layer (0.25) deaktiviert während des Trainings zufällig 25% der Ausgaben des vorherigen Dense-Layers. Jedes Neuron wird mit einer Wahrscheinlichkeit von 0.25 auf den Wert 0 gesetzt. Die Summe über alle Neuronen bleibt dabei Gleich. Dies verhindert die Ko-Adaptierung von Neuronen, bei der sich mehrere Neuronen auf ein einzelnes Eingabemuster spezialisieren und dadurch das Modell anfällig für Overfitting wird. (Ying, 2019)

Der zweite Dense-Layer mit 128 Neuronen und ReLU-Aktivierung erweitert den Feature-Raum von 88 auf 128 Dimensionen. Durch die Erhöhung der Kapazität kann das Modell Inter-Feature-Beziehungen modellieren, die über die per-Feature-Transformationen des vorherigen Layers hinausgehen. Typische Beziehungen umfassen:

- Intra-Hand-Abhängigkeiten: Korrelation zwischen benachbarten Landmarks des Selben fingers (z.B. die Auslenkung von PIP und DIP des Indexfingers), zwischen verschiedenen fingers einer Hand sowie die Orientierung der Hand und deren Rotation basierend auf der Handflächen features.
- Inter-Hand-Beziehungen: Zusammenhänge zwischen den Landmarks beider Hände, die bei einhändigen Gesten durch symmetrische oder asymmetrische Konfigurationen entstehen können.
- Komplexe Muster: Nicht-lineare Kombinationen mehrerer Landmarks, die erst in der erweiterten Repräsentation als discriminative Merkmale erkennbar werden.

Der zweite Dropout-Layer (0.5) wendet eine stärkere Regularisierung an als der erste Dropout-Layer. Die Rate von 50% ist bewusst höher gewählt, da der vorherige Dense-Layer mit 128 Neuronen eine größere Modellkapazität aufweist und die Gefahr des Overfitting steigt. Die stärkere Regularisierung vor der Ausgabeschicht stellt sicher, dass das Modell nur die robustesten Feature-Repräsentationen für die finale Klassifikation verwendet.

Der dritte Dense-Layer mit 26 Neuronen und Softmax-Aktivierung bildet die finale Klassifikationsschicht. Die 26 Ausgabeneuronen entsprechen den 26 Klassen (NONE, A–Y ohne J und Z, SCH). Die Softmax-Funktion

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

normalisiert (Sharma et al., 2020). Die Rohausgaben (Logits) in Wahrscheinlichkeiten, deren Summe exakt 1 ergibt. Jedes Ausgabeneuron liefert damit die Wahrscheinlichkeit dafür, dass der Input der entsprechenden Gebärdensprache-Klasse angehört.

7.2.5. Verlustfunktion und Optimierung

Für das Training des Modells wird die Verlustfunktion Sparse Categorical Crossentropy verwendet. Diese eignet sich insbesondere, da die Zielklassen als Integer und nicht als One-Hot-Vektoren kodiert vorliegen. Dadurch wird der Speicherbedarf reduziert und gleichzeitig eine numerisch stabile Berechnung der Gradienten ermöglicht. Die Verlustfunktion berechnet die Abweichung zwischen den tatsächlichen Klassen und den durch die Softmax-Ausgabe vorhergesagten Klassenwahrscheinlichkeiten und lässt sich durch folgende Formel beschreiben:

$$L = -\log(p_{y_{true}})$$

(What Is Sparse Categorical Crossentropy, 10:49:00+00:00) wobei $p_{y_{true}}$ die vom Modell vorhergesagte Wahrscheinlichkeit der tatsächlichen Klasse bezeichnet.

7.2.6. Trainingsprozess

Um einen Trainingsprozess durchlaufen zu können müssen Hyperparameter bestimmt werden. Zentral sind Optimizer, Lernrate, Verlustfunktion, Epochen, Trainingssplit, Batch Size und Frühzeitiges Stoppen.

Der Adam-Optimizer (Adaptive Moment Estimation) kombiniert die Vorteile von Momentum und adaptiven Lernraten (RMSProp). Er berechnet für jeden Parameter individuelle Lernraten auf Basis des ersten und zweiten Moments der Gradienten. Dadurch konvergiert das Modell in der Regel schneller und stabiler als klassische Optimierungsverfahren wie Stochastic Gradient Descent (SGD). Insbesondere bei kleinen oder mittelgroßen Datensätzen sowie komplexen neuronalen Netzen liefert Adam häufig robuste Ergebnisse, da weniger aufwendige Hyperparameteranpassungen erforderlich sind. (A. G. et al., 2026)

Die Lernrate bestimmt die Größe der Aktualisierungsschritte während der Optimierung. Eine sehr kleine Lernrate von 10^{-5} reduziert das Risiko, das Minimum der Verlustfunktion zu überschreiten oder instabile Trainingsverläufe zu erzeugen. Der Nachteil einer geringeren Lernrate besteht in einer längeren Trainingsdauer, die jedoch durch eine stabilere Konvergenz ausgeglichen wird.

Als Verlustfunktion eignet sich Sparse Categorical Crossentropy für Mehrklassenklassifikationen, bei denen die Zielklassen als Ganzzahlen (z. B. 0, 1, 2, ...) codiert sind. Im Gegensatz zur Categorical Crossentropy ist keine One-Hot-Kodierung der Labels erforderlich, wodurch Speicherbedarf und Vorverarbeitungsaufwand reduziert werden. Die Verlustfunktion misst die Abweichung zwischen den vorhergesagten Klassenwahrscheinlichkeiten und den tatsächlichen Klassen und liefert somit eine geeignete Optimierungsgrundlage für Klassifikationsaufgaben.

Die Sparse categorical Accuracy Metrik gibt den Anteil der korrekt klassifizierten Beispiele an und stellt damit eine leicht interpretierbare Leistungskennzahl dar. Da die Zielklassen als Integer vorliegen, passt sie direkt zur verwendeten Verlustfunktion SparseCategoricalCrossentropy. Während die Verlustfunktion zur Optimierung dient, ermöglicht die Accuracy eine intuitive Bewertung der Modellleistung während des Trainings und auf den Validierungsdaten.

Eine Epoche entspricht einem vollständigen Durchlauf des Trainingsdatensatzes. Die Wahl von 20 Epochen bietet dem Modell ausreichend Möglichkeiten, die zugrunde liegenden Muster zu erlernen, ohne die Trainingszeit unnötig zu verlängern. Da zusätzlich Early Stopping eingesetzt wird, dient dieser Wert als maximale Obergrenze. Das Training endet automatisch früher, sobald keine Verbesserung der Validierungsleistung mehr festgestellt wird.

Ein weiterer Hyperparameter ist die Aufteilung der Trainingsdaten in Training, Validation und Testdaten. Durch die Aufteilung des Datensatzes in 64% Trainingsdaten, 16% Validierungsdaten und 20% Testdaten kann die Generalisierungsfähigkeit des Modells während des Trainings überprüft werden. Die Validierungsdaten werden nicht für das Lernen verwendet und ermöglichen daher eine unabhängige Beurteilung der Modellleistung.

Die Batch Size legt fest, wie viele Trainingsbeispiele gleichzeitig verarbeitet werden, bevor eine Aktualisierung der Modellgewichte erfolgt. Eine Batchgröße von 128 stellt einen guten Kompromiss zwischen Rechenleistung, Speicherbedarf und Trainingsstabilität dar. Größere Batches ermöglichen eine

effizientere Nutzung von GPU Hardware dar und liefern stabilere Gradienten, während kleinere Batches zwar stärkeres Rauschen erzeugen, jedoch teilweise eine bessere Generalisierung fördern. Die gewählte Batchgröße bietet daher eine ausgewogene Balance zwischen Trainingsgeschwindigkeit und Modellqualität.

Early Stopping dient der Vermeidung von Overfitting, indem das Training automatisch beendet wird, sobald sich die Validierungsleistung über mehrere Epochen hinweg nicht mehr verbessert. Mit einer Patience von 3 werden kleinere Schwankungen der Validierungsverluste toleriert, bevor das Training gestoppt wird. Dadurch wird verhindert, dass unnötig lange trainiert wird oder das Modell beginnt, sich zu stark an die Trainingsdaten anzupassen. Gleichzeitig wird das Modell mit der besten Validierungsleistung gespeichert bzw. verwendet.

Tabelle 7.1.: Hyperparameter des Modelltrainings

Parameter	Wert
Optimizer	Adam
Learning Rate	0,00001
Verlustfunktion	Sparse Categorical Crossentropy
Metrik	Sparse Categorical Accuracy
Epochen	20
Validation Split	0,2 (80/20)
Batch Size	128
Early Stopping	3

7.2.7. Evaluierung

Die Evaluierung der Ergebnisse des Machine Learning Modells sind in Kapitel 8 zu finden. #TODO Füge kapitelreferenz ein!

7.3. Quantisierung, Konvertierung und Deployment der Modelle

Das folgende Kapitel beschreibt die Quantisierung des Tensorflow Fingeralphabet Modells, die Konvertierung der Quantisierten Handdedektion-, Landmarkerkennung- und Fingeralphabet- Modelle in NPU ausführbare Binaries. Im letzten Schritt wird das Flashen der Modelle auf den STM32N650DK beschrieben.

7.3.1. Quantisierung von Tensorflow Modellen

a die Modelle zur Handdetektion und Landmark-Erkennung bereits in einer quantisierten Form vorliegen, ist für diese keine weitere Quantisierung erforderlich. Lediglich das Modell zur Erkennung des Fingeralphabets muss für den Einsatz auf der Zielhardware quantisiert werden. Die Quantisierung erfolgt im Rahmen der Konvertierung des trainierten Keras-Modells in das TensorFlow-Lite-Format (TFLite). Dieser Schritt ist notwendig, da die Generierung der Embedded-Binärdateien mit STEdgeAI ausschließlich Modelle im TensorFlow-Lite- oder ONNX-Format unterstützt.

Bei der Konvertierung wird eine Post-Training-Quantisierung (PTQ) durchgeführt. Hierbei werden die Gewichte und Aktivierungen des bereits trainierten Modells von Gleitkommazahlen (Float32) auf 8-Bit-Ganzzahlen (INT8) abgebildet. Da die eingesetzte Neural Processing Unit (NPU) ausschließlich vollständig INT8-quantisierte Modelle unterstützt, müssen sowohl die internen Operationen als auch die Ein- und Ausgabedaten des Modells diesem Format entsprechen.

Zunächst wird das trainierte Keras-Modell geladen und ein Tensorflow-Lite-Konverter erzeugt.

```
1 model = keras.models.load_model(MODEL_DIR)
2 converter = tf.lite.TFLiteConverter.from_keras_model(model)
```

Listing 7.1: TensorFlow-Lite Konverter erzeugen

Anschließend werden die Standardoptimierungen des Konverters aktiviert:

```
1 converter.optimization = [tf.lite.Optimize.DEFAULT]
```

Listing 7.2: Optimierung aktivieren

Für die Kalibrierung der Quantisierung wird ein repräsentativer Datensatz bereitgestellt:

```
1 converter.representative_dataset = representative_dataset
```

Listing 7.3: Repräsentativer Datensatz

Während der Kalibrierung bestimmt TensorFlow Lite anhand dieser Daten die Wertebereiche der Aktivierungen, um geeignete Skalierungsfaktoren und Nullpunkte für die Quantisierung zu berechnen. Sind die Trainingsdaten nicht verfügbar, wird ersatzweise ein Datensatz aus gleichverteilten Zufallswerten im Bereich '[-1, 1]' verwendet.

Da die Zielhardware ausschließlich INT8-Operationen unterstützt, wird die Konvertierung entsprechend eingeschränkt und zusätzlich die Ein- und Ausgabedatentyp des Modells auf 8-Bit-Ganzzahlen festgelegt:

```
1 converter.target_spec.supported_ops = [  
2     tf.lite.OpsSet.TFLITE_BUILTINS_INT8  
3 ]  
4  
5 converter.inference_input_type = tf.uint8  
6 converter.inference_output_type = tf.uint8
```

Listing 7.4: INT8-Quantisierung konfigurieren

Abschließend wird das Modell in das Tensorflow-Lite-Format konvertiert und als .tflite-Datei gespeichert.

Ergebnis ist ein vollständig quantisiertes TensorFlow-Lite-Modell, das den Anforderungen der eingesetzten NPU entspricht und anschließend mit STEdgeAI in eine für das Embedded-System ausführbare Binärdatei überführt werden kann. Durch die Quantisierung werden der Speicherbedarf und der Rechenaufwand reduziert, während die Modellgenauigkeit durch die Verwendung repräsentativer Kalibrierungsdaten weitgehend erhalten bleibt.

7.3.2. Konvertierung INT8-Quantisierte-Modelle zu Embedded binaries

Nach der Quantisierung liegen alle drei Modelle (Handdetektion, Handlandmark-Erkennung und Fingeralphabet-Klassifikation) im TensorFlow-Lite-Format mit einer INT8-Quantisierung vor. Der nächste Verarbeitungsschritt ist diese Modelle mithilfe der ST Edge AI Core CLI in für die Zielhardware ausführbare Binärdateien zu überführen. Die Verwendung der Kommandozeilenschnittstelle (CLI) wurde bewusst der grafischen Benutzeroberfläche (GUI) vorgezogen, da sich hierdurch der gesamte Konvertierungsprozess automatisieren lässt. Auf diese Weise können Skripte erstellt werden, welche die Generierung aller Modellartefakte reproduzierbar und mit einem einzelnen Aufruf durchführen.

Die Verzeichnisstruktur der Modellgenerierung:

```
models/  
+-- source/                                <- Raw TFLite models (input)  
|   +-- 033_palm_detection_full_quant_pc_ff_od.tflite  
|   +-- 033_hand_landmark_full_quant_pc_uf_handl.tflite  
|   \-- fingeralphabet_model_int8.tflite  
|  
+-- my_mpool/                               <- Memory pool definitions per model  
|   +-- palm_detection.mpool  
|   +-- hand_landmark.mpool  
|   \-- fingeralphabet.mpool  
|  
+-- user_neuralart.json   <- Per-model config (links model -> mpool+compiler flags)  
+-- generate_n6_models.sh <- Orchestrator script (calls stedgeai, copies outputs)  
+-- copy_n6_models.sh    <- Copies generated files into the firmware project tree
```

```

|
| +-- generated/          <- Output: C source + headers ready for compilation
| | +-- palm/
| | +-- landmark/
| | \-- finger/
| |
| \-- st_ai_output/      <- Intermediate build artifacts (gitignored)

```

Listing 7.5: Verzeichnisstruktur der Modellgenerierung

Modell Profile In der Struktur definiert `user_neuralart.json` wie die einzelnen Modelle bei der Compilierung für die STM32N6 NPU zu behandeln sind. Sie enthält einen Abschnitt `Profiles` mit jeweils einem Eintrag pro Modell:

```

1 {
2   "Profiles": {
3     "palm_detection_model_v3": {
4       "memory_pool": "./my_mpool/palm_detection.mpool",
5       "options": "--enable-epoch-controller"
6         "-O3"
7         "--all-buffers-info"
8         "--cache-maintenance"
9     }
10  }
11 }

```

Listing 7.6: Modellprofil in `user_neuralart.json`

memory_pool

Verweist auf die `.mpool`-Datei, die das Speicherlayout der NPU für dieses Modell definiert (welche SRAM-Bänke, Flash-Bereiche, Adressen und Größen).

options

Definiert pro Modell welche Compilerflags verwendet werden sollen. Folgend werden verwendete Compiler flags in ?? kurz erläutert.

Tabelle 7.2.: ST Edge AI Core Compiler Flags

Flag	Effekt
<code>--enable-epoch-controller</code>	Ermöglicht die epochenbasierte Ausführung der NPU (schichtweise Ablaufplanung)
<code>-O3/-O2</code>	Optimierungsstufe für den NPU-Codegenerator
<code>--all-buffers-info</code>	Vollständige Puffer-Metadaten für die Laufzeit ausgeben
<code>--cache-maintenance / -Ocache-opt</code>	Befehle zum Leeren/Ungültigmachen des Caches einfügen
<code>--Oauto-sched</code>	Automatisches planen der Epochen (default/Ohne Verweis)
<code>--native-float</code>	Verwendet Hardware-Float-Operationen wenn möglich
<code>--enable-virtual-mem-pools</code>	Erlaubt dem Linker Tensoren über mehrere physische Speicherregionen zu verteilen
<code>--Omax-ca-pipe <num></code>	Legt die maximale Anzahl an Compute-&Accumulate-(CA-)Pipelines bzw. parallelen Hardware-Ausführungspfaden fest, die der Compiler gleichzeitig zuweisen darf.
<code>--Oconv-split-kw</code>	Aufteilen von Faltungskernen auf die verfügbaren NPU-MAC-Arrays

Memory-Pool-Definition Die `.mpool`-Dateien beschreiben die STM32N6 Hardware Speicherabbild der NPU. Alle `.mpool`-Dateien haben den identischen Aufbau. Der einzige wesentliche Unterschied zwischen den Modellen besteht in der xSPI2 (flash) Region (Basisadresse und Größe) an welcher die Modelle gespeichert sind.

Hauptbereiche:

params

```
1 { "paramname": "max_onchip_sram_size", "value": "1024", "magnitude": "KBYTES" }
```

Listing 7.7: Globale Speicher einschränkung in .mpool

Beschränkt die maximale verfügbare On-Chip-SRAM-Größe auf 1024 KB für die NPU.

cacheinfo Beschreibt den Cache.

mempools Speicherbereich-Konfiguration pro Modell auf der NPU.

Tabelle 7.3.: NPU-Speicherbereich-Konfiguration

Name	Region	Adresse	Größe	Rolle
npuRAM3	AXISRAM3	0x34200000	448 KB	NPU activation scratch
npuRAM4	AXISRAM4	0x34270000	448 KB	NPU activation scratch
npuRAM5	AXISRAM5	0x342e0000	448 KB	NPU activation scratch
npuRAM6	AXISRAM6	0x34350000	448 KB	NPU activation scratch
hyperRAM	xSPI1	0x90000000	0 MB	External HyperRAM (Deaktiviert)
octoFlash	xSPI2	0x71000000	2 MB	Weight storage in OctoSPI flash

Die vier AXISRAM-Bänke sind On-Chip-SRAM-Speicher mit hohem Durchsatz und geringer Latenz, die für Laufzeit-Aktivierungen (Zwischentensoren) genutzt werden. octoFlash ist zur Laufzeit schreibgeschützt (ACC.READ) und speichert die Modellgewichte bzw. -konstanten. Die Einstellung `constants_preferred: true` weist das Tool an, konstante Daten in diesem Bereich abzulegen.

Die Flash-Basisadressen unterscheiden sich je nach Modell, um den 16-MB-xSPI2-Flash zu partitionieren:

Tabelle 7.4.: Flash-Partitionierung je Modell

Modell	Flash Start	Flash Size
Finger	0x71000000	2 MB
Handdetektion	0x71200000	4 MB
Handlandmark	0x71600000	10 MB

Script - Generierung Das `generate_n6_models.sh`-Script führt für jedes Modell 3 Schritte durch (Konvertierung, Kopieren und Project-Build).

Im ersten Schritt wird die ST Edge AI core CLI zur konvertierung TF-Lite → NPU-Artefakte verwendet.

```
1 stedgeai generate \  
2   --name palm_detection_model_v3 \  
3   --model source/033_palm_detection_full_quant_pc_ff_od.tflite \  
4   --target stm32n6 \  
5   --st-neural-art "palm_detection_model_v3@user_neuralart.json" \  
6   --input-data-type uint8 \  
7   --output st_ai_output/
```

Listing 7.8: STedgeAI Modellgenerierung

Dies liest das TensorFlow-Lite-Modell ein, lädt die `palm_detection_model_v3` Konfiguration in `user_neuralart.json` um den Memory-Pool und Compiler-Optionen zu erhalten und generiert:

- Eine `.c`-Datei mit Gewichtsdaten als C-Arrays (die rohen Binärdaten sind eingebettet)

- Ein Header namens `_ecblobs.h`, der diese Arrays mit Attributen für Linker-Sektionen deklariert
- `stai_.c / stai_.h` – der ST-AI-Runtime-API-Wrapper (Zuweisung, Laden, Ausführung der Inferenz)
- Ein `_atobuf.xSPI2.raw` file – Der Binär-Blob mit den Gewichten für die Flash-Programmierung

Der zweite Schritt konvertiert `.raw` Gewichts-Blob in eine Intel `.hex`-Datei mit der korrekten Basisadresse (z. B. `0x71000000`). Diese Hex-Datei kann via STM32CubeProgrammer auf den Mikrocontroller geflashed werden.

Der letzte Schritt führt das `copy_n6_models.sh`-Script aus, es kopiert die generierten Dateien in das STM32N6570DK-Embedded-Projekt:

- `*.c` und `*_ecblobs.h` → `embedded/STM32N6570DK/FSBL/Src/AI/<Model>/Src/Inc/`
- `stai_.c/h` (with `--stai` flag)
- `*.bin` weight blobs (with `--weights` flag) → `FSBL/Assets/AI/`

Zudem wird `_ecblobs.h` gepatcht, um `\#include "ecblob_sections.h"` einzufügen. Dadurch wird sichergestellt, dass die Gewichts-Arrays im ECBLOBS-Linker-Abschnitt (`0x72000000`, 8 MB) landen, anstatt des internen ROM (255 KB) zu sprengen.

7.4. Softwaregrundstruktur und hardwarenahe Basistreiber

Die Umsetzung der Kamera-, Display- und KI-Verarbeitung setzt zunächst eine funktionsfähige hardwarenahe Basissoftware voraus. Hierzu wurden grundlegende Treiber für die Initialisierung und Steuerung des Mikrocontrollers sowie seiner Peripherie entwickelt. Diese abstrahieren die direkten Registerzugriffe und stellen den übergeordneten Softwarekomponenten einheitliche Funktionen zur Verfügung. Zu den grundlegenden Komponenten gehören insbesondere die Taktsteuerung, die Konfiguration der Ein- und Ausgänge sowie Treiber für UART, Timer und I2C.

Die Implementierung greift auf den Cortex Microcontroller Software Interface Standard (CMSIS) zurück. Das sind bereitgestellte Prozessor- und Gerätedefinitionen für Cortex Microcontroller. CMSIS stellt dabei unter anderem die Registerstrukturen, Interruptnummern und Funktionen für den Zugriff auf den Cortex-M55-Prozessorkern bereit [30]. Die eigentliche Konfiguration der Peripherie wurde dagegen überwiegend durch projektspezifische Treiber umgesetzt. Dadurch konnten die benötigten Funktionen gezielt an die Anforderungen des Systems angepasst und nicht benötigte Bestandteile umfangreicher Abstraktionsschichten vermieden werden.

Die Basissoftware bildet die unterste anwendungsspezifische Softwareschicht des Systems. Auf ihr bauen die Treiber und Komponenten für Kamera, Display, externe Speicher und neuronalen Beschleuniger auf. Erst durch diese Schichtung können die übergeordneten Funktionen, beispielsweise die Kamera-Display-Pipeline und die KI-Verarbeitungskette, unabhängig von einzelnen Registerzugriffen strukturiert umgesetzt werden. [31, s. 3-4].

7.4.1. Aufbau der Embedded-Software

Mit der Integration der Kamera, des Displays, der externen Speicher und der neuronalen Netze umfasst die Firmware mehrere Komponenten mit unterschiedlichen Aufgaben und Hardwareabhängigkeiten. Der Quellcode wurde daher in getrennte Bereiche gegliedert. Die Struktur orientiert sich an den Prinzipien der Modularisierung und der Trennung von Zuständigkeiten. Dabei werden unterschiedliche Funktionen in eigenständigen Komponenten mit definierten Schnittstellen zusammengefasst und Implementierungsdetails gegenüber den übergeordneten Komponenten gekapselt [31, s. 3-4]. Listing 7.9 zeigt die grundlegende Projektstruktur:

```
Sign_Language_Translator/
  \-- Src/
    +-- AI/                                <- KI-Modelle und Verarbeitung
    |   +-- Base/                          <- Generische KI-Schnittstelle
    |   +-- Fingeralphabet/                <- Fingeralphabet-Klassifikation
    |   +-- Hand_landmark/                 <- Hand-Landmark-Erkennung
```

```

|   \-- Palm_detection/      <- Handflaechen-Detektion
|
+-- Assets/                  <- Bildmaterial & Fonts
+-- Core/                    <- Systemkonfiguration und Ablaufsteuerung
|   +-- main.c               <- Einstiegspunkt
|   +-- app.c                <- Hardware-Init-Koordination
|   +-- config.c             <- Zentrale Konfigurationsstrukturen
|   \-- tasks.c              <- Scheduler-Aufgaben
|
+-- Drivers/                 <- Treiberschicht
|   +-- CMSIS/               <- Arm CMSIS-Bibliothek
|   +-- HAL/                 <- ST HAL-Bibliothek
|   \-- Simple/              <- Projektspezifische Treiber
|       +-- Src/              <- Implementierungen
|       \-- Inc/              <- Oeffentliche Schnittstellen
|
\-- Middlewares/             <- Externe Bibliotheken
    \-- ST/AI/               <- ST Neural-ART NPU-Middleware

```

Listing 7.9: Verzeichnisstruktur der Embedded-Software

7.4.2. Zentrale Konfiguration und Systeminitialisierung

Die Konfiguration und Initialisierung der Systemkomponenten werden an zentralen Stellen der Software zusammengeführt. Dadurch müssen hardwarespezifische Parameter nicht innerhalb der einzelnen Anwendungskomponenten festgelegt werden. Gleichzeitig wird eine definierte Initialisierungsreihenfolge sichergestellt, da mehrere Komponenten von zuvor eingerichteten Taktquellen, Speicherbereichen oder Basistreibern abhängig sind.

Die Dateien `config.c/h` enthalten die systemweit verwendeten Konfigurationswerte und Konfigurationsstrukturen. Dazu gehören unter anderem Bildauflösungen, Speicherzuordnungen, Schnittstellenparameter sowie Einstellungen der Kamera-, DCMIPP- und Displaykomponenten. Die Konfiguration wird damit von der eigentlichen Treiberimplementierung getrennt. Die jeweiligen Treiber erhalten die benötigten Parameter über definierte Strukturen und übertragen sie während der Initialisierung in die zugehörigen Hardwareregister.

Tabelle 7.5.: Aufgaben der zentralen Core-Dateien

Datei	Aufgabe
<code>main.c</code>	Einstiegspunkt der Software und Übergabe an die übergeordnete Anwendungsinitialisierung.
<code>app.c</code>	Koordination der Initialisierung der einzelnen Hardware- und Softwarekomponenten.
<code>config.c/h</code>	Definition & Instanziierung zentraler Konfigurationsstrukturen.
<code>tasks.c</code>	Definition der später durch den Software-Scheduler ausgeführten Verarbeitungsaufgaben.

Nach dem Systemstart wird zunächst die Adresse der Interruptvektortabelle in das Vector Table Offset Register des Prozessors eingetragen. Anschließend erfolgt die Konfiguration des Resource Isolation Framework Security Controllers (RIFSC). Dieser verwaltet die Zugriffsrechte auf interne Speicherbereiche und Peripheriekomponenten des STM32N6570. Eine fehlerhafte Zuordnung kann dazu führen, dass einzelne Komponenten trotz aktivierter Taktversorgung nicht auf benötigte Register oder Speicherbereiche zugreifen können. Die Zugriffsrechte werden daher eingerichtet, bevor die entsprechenden Hardwarekomponenten initialisiert werden [32].

Darauf folgen die Konfiguration der Spannungsversorgung sowie die Einrichtung der System- und Peripherietakte. Zusätzlich werden die benötigten Takte für den Energiesparmodus aktiviert. Dies ist insbesondere für die asynchrone Ausführung der neuronalen Netze relevant, da der Prozessorkern während der Inferenz durch Warteanweisungen vorübergehend in einen Energiesparzustand wechseln kann. Die von der NPU und den externen Speichern benötigten Takte müssen währenddessen weiterhin

aktiv bleiben. Dieses Verhalten wird auch in der Dokumentation zur Ausführung neuronaler Netze auf dem STM32N6 beschrieben [13].

Nach der grundlegenden Systemkonfiguration werden der Instructions- und der Datencache aktiviert. Der Instructioncache reduziert die Zugriffszeiten beim Laden von Programmcode. Der Datencache beschleunigt den Zugriff auf häufig verwendete Daten, erfordert jedoch bei gemeinsam durch Prozessor und Peripherie genutzten Speicherbereiche eine explizite Cache-Synchronisation. Dies betrifft beispielsweise die Ein- und Ausgabepuffer der neuronalen Netze. Die erforderlichen Cache-Operationen werden deshalb innerhalb der jeweiligen Treiber bzw. Modulschnittstellen ausgeführt.

Anschließend werden der externe PSRAM- und der NOR-Flash-Speicher über die XSPI-Schnittstellen eingerichtet. Der PSRAM wird vor allem für die Aufnahme der Bild- und Anzeigepuffer verwendet, während der NOR-Flash die für die neuronalen Netze benötigten Modell- und Laufzeitdaten enthält. Da das Display und die Kamerapipeline auf diese Speicherbereiche zugreifen, müssen die externen Speicher vor diesen Komponenten initialisiert werden. Beide Speicher werden im Memory-mapped-Modus betrieben und damit in den Adressraum des Mikrocontrollers eingebunden. Zugriffe können dadurch über reguläre Speicheradressen erfolgen, ohne dass für jeden Lese- oder Schreibvorgang eine separate XSPI-Übertragung angestoßen werden muss. Die Umsetzung der Speicherzugriffe in die erforderlichen XSPI-Kommandos übernimmt der jeweilige Schnittstellencontroller. Für die übergeordneten Softwarekomponenten erscheinen die externen Speicher somit wie direkt adressierbare Speicherbereiche. Dabei wird der PSRAM lesend und schreibend verwendet, während aus dem NOR-Flash hauptsächlich die Modellparameter ausgelesen werden.

Daraufhin wird die Kamera initialisiert und die Bildverarbeitung über die DCMIPP-Komponente gestartet. Dabei werden zwei Verarbeitungspfade verwendet. Der erste Pfad stellt die aufgenommenen Kamerabilder für die Anzeige bereit. Der zweite Pfad erzeugt die Eingabebilder für die Verarbeitung durch die neuronalen Netze. Durch diese Trennung können Anzeige und KI-Pipeline unterschiedliche Bildauflösungen und Speicherbereiche verwenden. Nach dem Start der Kamerapipeline werden der Touchcontroller und die grafische Benutzeroberfläche eingerichtet. Die Benutzeroberfläche stellt unter anderem Auswahl- und Anzeigeelemente für die einzelnen Verarbeitungsstufen bereit. Über diese können die Ergebnisse der Handerkennung, der Landmarkenerkennung und der Fingeralphabetklassifikation getrennt dargestellt und konfiguriert werden.

Im Anschluss wird die KI-Laufzeitumgebung initialisiert. Dabei werden die NPU, die zugehörige Middleware sowie die Ein- und Ausgabepuffer der eingebundenen neuronalen Netze vorbereitet. Schlägt dieser Schritt fehl, wird der weitere Programmablauf gestoppt, da die eigentliche Anwendung ohne die neuronalen Netze nicht vollständig ausgeführt werden kann. Abschließend wird der Software-Scheduler gestartet, um die eigentliche Ablaufsteuerung zu verwalten. Dazu wird Genauer in Kapitel 7.6 beschrieben. Die Initialisierungsreihenfolge lässt sich damit wie folgt zusammenfassen:

Tabelle 7.6.: Initialisierungsreihenfolge nach Systemstart

Schritt	Komponente	Zweck
1	Interruptvektortabelle	Zuordnung der verwendeten Interruptbehandlungen
2	RIFSC	Freigabe der benötigten Speicher- und Peripheriezugriffe
3	Spannungsversorgung und Takte	Grundlage für alle weiteren Hardwarekomponenten
4	Instruktions- und Datencache	Beschleunigung der Code- und Datenzugriffe
5	Debugschnittstelle	Diagnose
6	PSRAM und NOR-Flash	Bereitstellung der Bild- und Modelldaten
7	LTDC und LTDC-Layer	Ausgabe des Kamerabildes und der Benutzeroberfläche
8	Kamera und DCMIPP	Aufnahme und Aufbereitung der Bilddaten
9	Touch und Benutzeroberfläche	Interaktion und Ergebnisdarstellung
10	KI-Laufzeitumgebung	Vorbereitung der NPU und der neuronalen Netze
11	Software-Scheduler	Zyklische Ausführung der Verarbeitungsaufgaben

7.4.3. Registerbasierte Treiberentwicklung

Die Hardwareanbindung wurde überwiegend durch projektspezifische, registerbasierte Treiber umgesetzt. Diese greifen mithilfe der von CMSIS bereitgestellten Registerstrukturen und Bitmasken direkt auf die Peripherieregister des Mikrocontrollers zu. Die Konfiguration erfolgt entsprechend den Vorgaben des Referenzhandbuchs des STM32N6570. Die direkten Registerzugriffe werden hinter einheitlichen Treiberfunktionen gekapselt. Übergeordnete Komponenten müssen daher weder die verwendeten Register noch die Position der einzelnen Registerfelder kennen. Konfigurationsparameter werden überwiegend in Strukturen zusammengefasst und bei der Initialisierung an den jeweiligen Treiber übergeben.

Das Vorgehen zeigt sich beispielsweise im GPIO-Treiber. Die Funktion `GPIO_Config()` aktiviert zunächst den Takt des ausgewählten GPIO-Ports und überträgt anschließend die übergebene Konfiguration in die Register `MODER`, `OTYPER` und `PUPDR`:

```
1 void GPIO_Config(GPIO_TypeDef *GPIOX, uint32_t pinNr, GPIO_cfg_TypeDef cfg){
2     RCC_enable_GPIO(GPIOX);
3
4     GPIOX->MODER = (GPIOX->MODER & ~(3U << (2U * pinNr)))
5                   | (cfg.mode << (2U * pinNr));
6
7     GPIOX->OTYPER = (GPIOX->OTYPER & ~(1U << (pinNr)))
8                   | (cfg.otyp << (pinNr));
9
10    GPIOX->PUPDR = (GPIOX->PUPDR & ~(3U << (2U * pinNr)))
11                  | (cfg.pupdr << (2U * pinNr));
12
13    if (cfg.mode == GPIO_MODE_AF) {
14        GPIO_set_af(GPIOX, pinNr, cfg.af, cfg.speed);
15    }
16 }
```

Listing 7.10: GPIO-Konfigurationsfunktion

Durch die Maskierung werden ausschließlich die zum ausgewählten Anschluss gehörenden Registerfelder verändert. Bei Verwendung einer Alternativfunktion übernimmt die interne Hilfsfunktion `GPIO_set_af()` zusätzlich die Konfiguration des `AFR`- und des `OSPEEDR`-Registers.

Die Konfiguration erfolgt mithilfe der Struktur `GPIO_cfg_TypeDef`. Diese fasst den Betriebsmodus, den Ausgangstyp, die Pull-up- beziehungsweise Pull-down-Konfiguration, die Ausgangsgeschwindigkeit und die Nummer der Alternativfunktion zusammen. Für wiederkehrende Anwendungsfälle werden passende Konfigurationen vorab definiert und anschließend gemeinsam mit dem GPIO-Port und der Pinnummer an `GPIO_Config()` übergeben. Dadurch bleibt die eigentliche Registerkonfiguration unabhängig von der späteren Verwendung des Anschlusses. Der GPIO-Treiber muss beispielsweise nicht wissen, ob ein Anschluss für I²C, USART oder eine andere Peripherie vorgesehen ist. Er überträgt lediglich die in der Konfigurationsstruktur enthaltenen Parameter in die entsprechenden Register. Die Zuordnung einer konkreten Schnittstelle zu den benötigten Anschlüssen und Alternativfunktionen erfolgt außerhalb des Treibers.

Dieses Prinzip wird auch bei weiteren Treibern verwendet. Hardwareparameter werden in Konfigurationsstrukturen zusammengefasst, während die Treiberfunktionen deren Prüfung und Übertragung in die Register übernehmen. Dadurch können unterschiedliche Instanzen derselben Peripherie mit einer gemeinsamen Implementierung konfiguriert werden. Gleichzeitig bleiben die hardwarespezifischen Registerzugriffe auf die jeweilige Treiberschicht begrenzt.

7.4.4. Grundlegende Systemtreiber

Die projektspezifische Treiberschicht umfasst sowohl grundlegende Systemtreiber als auch funktionspezifische Komponenten. Als grundlegende Systemtreiber werden im Rahmen dieser Arbeit diejenigen Komponenten eingeordnet, die von mehreren übergeordneten Modulen verwendet werden und

zunächst keinen eigenständigen Anwendungszweck erfüllen. Sie stellen unter anderem die Taktversorgung, den Zugriff auf Ein- und Ausgänge, Zeitfunktionen sowie serielle Kommunikations- und Speicherschnittstellen bereit. Die Treiber für Kamera, DCMIPP, Display und Benutzereingabe bauen auf diesen grundlegenden Funktionen auf. Ebenso werden die Komponenten für den Software-Scheduler und die NPU-Anbindung getrennt behandelt. Tabelle 7.7 beschränkt sich daher auf die allgemeinen Systemtreiber der Software.

Tabelle 7.7.: Grundlegende Systemtreiber der Embedded-Software

Datei	Aufgabe
simple_rcc.c	Konfiguration der Energieversorgung sowie Aktivierung und Auswahl der System- und Peripherietakte.
simple_rifsc.c	Konfiguration der Zugriffsrechte und Sicherheitsattribute der verwendeten Systemressourcen.
simple_gpio.c	Konfiguration und Ansteuerung der digitalen Ein- und Ausgänge sowie ihrer alternativen Funktionen.
simple_timer.c	Konfiguration der Timer und Bereitstellung von einer Systemzeit und einer Verzögerungsfunktion.
simple_usart.c	Konfiguration der seriellen Schnittstelle zur Übertragung von Diagnose- und Debugdaten.
simple_i2c.c	Kommunikation mit externen Komponenten, insbesondere die Kamera und der Touchcontroller.
simple_xspi.c	Initialisierung und Ansteuerung des externen PSRAM und NOR-Flash über die XSPI-Schnittstellen.

Die grundlegenden Systemtreiber bauen teilweise aufeinander auf. Der RCC-Treiber aktiviert zunächst die benötigten Peripherietakte, während der RIFSC die erforderlichen Zugriffsrechte auf Speicher- und Peripheriebereiche bereitstellt. Darauf aufbauend konfiguriert der GPIO-Treiber die Ein- und Ausgänge sowie die alternativen Pin-Funktionen für Schnittstellen wie USART, I²C und XSPI. Die Timer-, Kommunikations- und Speichertreiber können anschließend von den übergeordneten Komponenten verwendet werden. Die funktionspezifischen Treiber für Kamera, Display, Ablaufsteuerung und NPU werden in den folgenden Kapiteln im Zusammenhang mit ihrer jeweiligen Verwendung beschrieben.

7.4.5. Einordnung der Basistreiber in die übergeordneten Komponenten

Die funktionspezifischen Komponenten der Firmware bauen jeweils auf mehreren grundlegenden Systemtreibern auf. Tabelle 7.8 ordnet den übergeordneten Komponenten die von ihnen verwendeten Basistreiber zu.

Tabelle 7.8.: Zuordnung der Basistreiber zu den übergeordneten Softwarekomponenten

Übergeordnete Komponente	Verwendete Basistreiber
Kamera- und Bildpipeline	RCC, RIFSC, GPIO, I2C, Timer, XSPI und USART
Ablaufsteuerung und Software Scheduler	RCC, Timer und USART
NPU- und KI-Modul	RCC, RIFSC, XSPI und USART
Status-LED	RCC, GPIO und Timer
Touch-Eingabe	RCC, GPIO, I2C, Timer und USART

Die Zuordnung zeigt, dass insbesondere die RCC-, und GPIO-Treiber von mehreren Systembereichen gemeinsam verwendet werden. Die Basistreiber stellen damit die gemeinsame hardwarenahe Grundlage bereit, während die übergeordneten Komponenten deren Funktionen zu anwendungsspezifischen Abläufen kombinieren.

7.5. Kamera - Display Pipeline

Die Kamera-Display-Pipeline bildet die Grundlage für die Erfassung, Aufbereitung und Ausgabe der Kamerabilder. Sie verbindet den IMX335-Kamerasensor über die CSI-Schnittstelle mit der DCMIPP und überträgt die daraus erzeugten Bilddaten in die vorgesehenen Pufferbereiche im externen PS-RAM. Das von Pipe 1 bereitgestellte Kamerabild wird anschließend über den LTDC auf dem angeschlossenen Display ausgegeben. Parallel erzeugt Pipe 2 die Eingabebilder für die Handdetektion. Die folgenden Abschnitte beschreiben die Initialisierung des Kamerasensors, die Übertragung der Bilddaten über CSI und DCMIPP, die Verwaltung der verwendeten Bildpuffer sowie die Konfiguration der Displayausgabe. Die innerhalb der DCMIPP ausgeführten Bildverarbeitungsschritte werden getrennt davon in Abschnitt 7.8.1 erläutert.

7.5.1. Initialisierung des Kamerasensors

Als Kamerasensor wird der IMX335 verwendet. Die Initialisierung ist in `CAM_Init()` zusammengefasst. Zunächst werden die GPIO-Pins für die Spannungsversorgung und den Hardware-Reset konfiguriert. Der Sensor wird durch eine festgelegte Schaltfolge mit zeitlichen Abständen in einen definierten Ausgangszustand versetzt. Die Konfiguration der Sensorregister erfolgt über I2C. Hierzu werden die zugehörigen GPIO-Pins für ihre I2C-Funktion eingerichtet und die Schnittstelle mit einer Übertragungsrate von 400 kHz initialisiert. Mit `IMX335_Probe()` wird anschließend geprüft, ob der Sensor erreichbar ist und die erwartete Gerätekennung besitzt. Bei einer fehlerhaften Identifikation wird die Kamerainitialisierung abgebrochen.

Nach der Einrichtung von CSI und DCMIPP werden die Sensorregister durch `IMX335_Init()` für den vorgesehenen Betriebsmodus konfiguriert. Der Sensor stellt die Bilddaten mit einer Auflösung von 2592×1944 Pixeln im RAW10-Format bereit. Abschließend wird die automatische Belichtungsregelung initialisiert und eine anfängliche Belichtungszeit an den Sensor übertragen.

7.5.2. Übertragung der Kameradaten über CSI und DCMIPP

Die vom IMX335 aufgenommenen Bilddaten werden über eine MIPI-CSI-2-Verbindung an den Mikrocontroller übertragen. Die CSI-Schnittstelle verwendet zwei Datenleitungen, den virtuellen Kanal 0 und den CSI-2-Datentyp '0x2B' für RAW10-Bilddaten. Nach der grundlegenden Initialisierung werden die Übertragungsparameter konfiguriert und der virtuelle Kanal aktiviert. Die empfangenen Bilddaten werden an die DCMIPP weitergegeben. Diese verteilt den gemeinsamen Kameradatenstrom auf zwei Ausgabepfade. Pipe 1 erzeugt das 800×480 Pixel große Displaybild, das gleichzeitig als Ausgangsbild für die Landmark-Erkennung verwendet wird. Pipe 2 stellt ein 192×192 Pixel großes Eingabebild für die Handdetektion bereit. Dadurch können beide Bildformate unabhängig voneinander aus demselben Kameradatenstrom erzeugt werden.

Die beiden Pipelines werden durch `CAM_DisplayPipe_Start()` und `CAM_NNPipe_Start()` gestartet. Für beide Ausgabepfade wird jeweils ein Frame-Interrupt aktiviert. Dieser signalisiert, dass die Übertragung eines vollständigen Bildes in den vorgesehenen Puffer abgeschlossen wurde. Anschließend kann der jeweilige Puffer für die Displayausgabe beziehungsweise die KI-Verarbeitung verwendet werden.

7.5.3. Verwaltung der Bildpuffer

Aufgrund des hohen Speicherbedarfs werden die Bildpuffer im externen PSRAM abgelegt. Dieser wird über die XSPI-Schnittstelle angebunden und im Memory-Mapped-Modus betrieben. Dadurch können alle beteiligten Komponenten über feste Speicheradressen auf die Bilddaten zugreifen.

Für das von Pipe 1 erzeugte Displaybild werden vier Hintergrundbildpuffer verwendet. Jedem Puffer wird abhängig vom aktuellen Zustand eine bestimmte Aufgabe zugeordnet. Ein Puffer wird von der DCMIPP mit dem nächsten Kamerabild beschrieben, während ein weiterer vom LTDC auf dem Display ausgegeben wird. Die übrigen Puffer stehen für die KI-Verarbeitung und das Einzeichnen der Region of Interest beziehungsweise der Handlandmarks zur Verfügung. Durch diese Trennung wird verhindert, dass ein angezeigtes oder noch verarbeitetes Bild gleichzeitig von der DCMIPP überschrieben wird. Nach Abschluss einer Bildübertragung wird über den Frame-Interrupt der fertiggestellte Puffer

bestimmt. Anschließend werden die Zuständigkeiten der Puffer neu zugeordnet und die Zieladresse von Pipe 1 auf den nächsten freien Aufnahmebuffer gesetzt. Der für die Anzeige vorgesehene Puffer wird dem LTDC übergeben.

Pipe 2 verwendet zwei separate Eingabebildpuffer für die Handdetektion. Diese werden im Double-Buffer-Modus abwechselnd durch die DCMIPP beschrieben. Während ein Puffer mit neuen Bilddaten gefüllt wird, kann der bereits fertiggestellte Puffer durch die KI-Verarbeitung verwendet werden. Der Frame-Interrupt kennzeichnet den jeweils vollständig übertragenen Puffer als bereit. Zusätzlich werden zwei Vordergrundpuffer für die Benutzeroberfläche bereitgestellt. Sie ermöglichen es, eine neue Darstellung vorzubereiten, während der LTDC weiterhin den vorherigen Stand ausgibt. Nach Abschluss der Zeichenoperationen werden die beiden Puffer getauscht.

7.5.4. LTDC-Konfiguration und Verwaltung der Displayebenen

Die Ausgabe der Bilddaten erfolgt über den LCD-TFT Display Controller (LTDC). Dieser liest die im externen PSRAM abgelegten Bildpuffer aus und überträgt deren Inhalt kontinuierlich an das angeschlossene Display. Für die Darstellung werden zwei übereinander liegende Displayebenen verwendet.

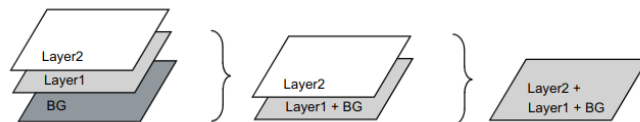


Abbildung 7.2.: Zusammenfügen von zwei Layern (STMicroelectronics, 2025)

Layer 1 bildet die Hintergrundebene und zeigt das von Pipe 1 erzeugte Kamerabild mit einer Auflösung von 800×480 Pixeln im RGB888-Format. Auf dieser Ebene werden außerdem die Region of Interest und die erkannten Handlandmarks in das jeweilige Kamerabild eingezeichnet. Layer 2 dient als Vordergrundebene für die grafische Benutzeroberfläche. Sie verwendet das ARGB4444-Format und unterstützt dadurch eine pixelweise Transparenz. Transparente Bereiche geben das darunterliegende Kamerabild frei, während Bedienelemente, Texte und Statusinformationen über diesem dargestellt werden können.

Die Abbildung veranschaulicht die Zusammensetzung der Displayausgabe. Zunächst wird Layer 1 über dem konfigurierten LTDC-Hintergrund dargestellt. Da Layer 1 das vollständige Display im RGB888-Format abdeckt, bildet er im regulären Betrieb den sichtbaren Hintergrund. Anschließend wird Layer 2 unter Berücksichtigung der im ARGB4444-Format enthaltenen Alphawerte darübergelegt. Transparente Bereiche von Layer 2 lassen das Kamerabild von Layer 1 sichtbar, während nicht transparente Bereiche die Elemente der Benutzeroberfläche darstellen. Das Ergebnis beider Ebenen wird anschließend als gemeinsames Bild an das Display ausgegeben. Die genaue Gestaltung und Aktualisierung der Benutzeroberfläche wird in Abschnitt 7.9 beschrieben.

7.6. Ablaufsteuerung und Software Scheduler

7.6.1. Architektur-Übersicht

Mit der zunehmenden Komplexität einzelner Komponenten verlängert sich die Laufzeit dieser. Dies führt bei sequenzieller Ausführung zu dem Problem, dass Komponenten, die in einer bestimmten Frequenz ausgeführt werden müssen, nicht mehr rechtzeitig bedient werden können. Ein Beispiel hierfür ist das Display, das bei zu langsamer Zykluszeit nicht mehr interaktionsfähig bleibt, während beispielsweise die Kamera-Erfassung oder Sensorabfragen deutlich mehr Rechenzeit beanspruchen. Eine rein sequenzielle Schleife (super-loop) kann diese unterschiedlichen Anforderungen nicht mehr befriedigen.

Als Lösung wurde ein prioritätsbasierter preemptiver Scheduler implementiert, der als Nicht-RTOS-Lösung auf dem ARM Cortex-M55 direct auf dem bare-metal läuft. Der Scheduler verwaltet maximal 9 Tasks, wobei Slot 8 fest dem Idle-Task zugewiesen ist, der im WFI-Zustand (Wait For Interrupt) verharrt und bei Bedarf Statistiken ausgibt.

Zeitgeber (Tick-Source): Als Zeitsystem dient TIM7 mit einer Periode von 1 ms, konfiguriert über APB1 mit 200 MHz Taktfrequenz. Der Timer generiert im TIM7_IRQHandler einen Tick, der den System-Tick-Zähler (_sys_tick_ms) erhöht und bei abgelaufener Periode die jeweiligen Tasks in den TaskReady-Zustand ueberführt, gefolgt von einer PendSV-Anforderung zur Kontextwahl.

Kontextmodell: Pro Task wird der vollstaendige ARM-Kontext gespeichert. Der Scheduler nutzt den PSP (Process Stack Pointer) im Thread-Modus, wobei die FPU-Kontextspeicherung mittels lazy-stacking (ASPEN + LSPEN) erfolgt. Der zu speichernde Stack-Frame umfasst 32 Words: die FPU callee-saved Register S16–S31 (16 Words), die Core callee-saved Register R4–R11 (8 Words) sowie den 8-Word Hardware Exception Frame (R0–R3, R12, LR, PC, xPSR).

Stack-Management: Tasks erhalten einen variablen Stack, der aus einem statischen Bump-Allocator Pool mit 8192 Words (32 KB) alloziert wird. Die Stack-Grösse pro Task ist konfigurierbar (min. 144 Words, max. 2048 Words). Jeder Stack wird mit einem Canary-Muster (0xA5A5A5A5) gefüllt, um Stack-Overflows näher betrachten zu können. Der untere Stack-Schutz erfolgt über PSPLIM (Process Stack Limit Register), wobei die Guard-Zone 25% der Stack-Groesse (64–512 Bytes) plus 32 Bytes Exception-Frame-Marge beträgt. Da der Bump-Allocator keine Deallocation unterstuetzt, wird bei einem Re-Add mit grösserer Stackanforderung der alte Block aufgegeben und ein neuer aus dem Pool alloziert.

Preemption: Tasks können durch höherpriorisierte Tasks unterbrochen werden. Die Prioritäten reichen von 0 (höchste) bis 0xFF (niedrigste, Idle-Task). Die Kontextwahl erfolgt über PendSV (Pending supervisor call) mit der niedrigsten Interruptpriorität (0xFF), sodass alle anderen Interrupts zuerst abgearbeitet werden. Die Selektion des nächsten Tasks erfolgt mittels Round-Robin mit Prioritätssuche: Es wird ab dem aktuellen Task-Index zirkulär gesucht und der höchstpriorisierte bereite Task ausgewählt.

7.6.2. Task-Lebenszyklus (State machine)

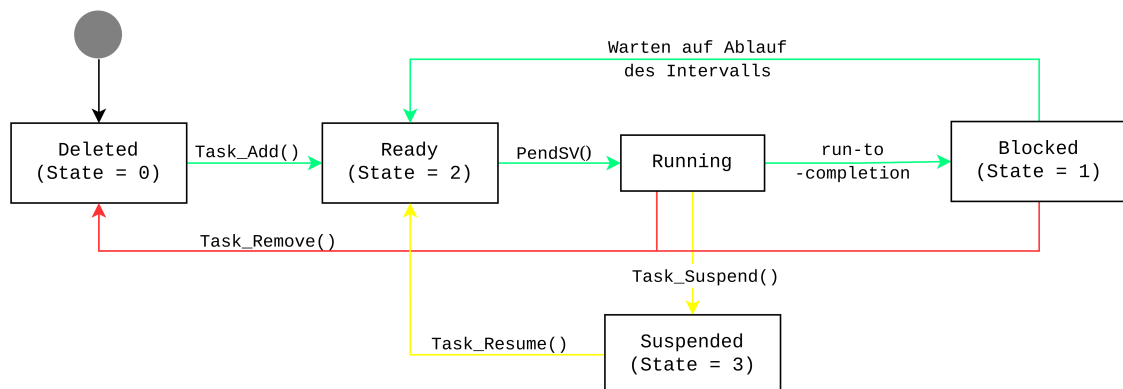


Abbildung 7.3.: Task-States des Schedulers

Ein Task hat vier Zustände:

- Deleted: Task wartet auf Zuweisung eines Prozesses
- Ready: Task kann Ausgeführt werden. Wartet auf Scheduler Auswahl.
- Blocked: Task wartet auf den Ablauf seiner Periode
- Suspended: Task wird an der gehindert Ausgeführt zu werden bis dieser weitergeführt werden kann. DMA2D pausiert Task bis die Speicheroperation erfolgt ist.

7.3 stellt die Übergänge zwischen Zuständen dar.

7.6.3. Interrupt-Hierrarchie & Context Switching

Die Kontextwaehlerung ist der Kernmechanismus des Schedulers und erfolgt auf ARM Cortex-M55 auf drei Ebenen: dem anfänglichen Bootstrap des ersten Tasks, der periodischen Tick-gesteuerten Aktivierung via TIM7_IRQHandler, und der eigentlichen Contextwaehlerung via PendSV_Handler

7.6.3.1. Anfänglicher Context-Start (Bootstrapping)

Der Start des ersten Tasks erfolgt nicht ueber PendSV, sondern ueber einen SVC-Aufruf (Supervisor Call). Der Ablauf in SCHEDULER_Tasks_run() ist wie folgt:

```
SCHEDULER_Tasks_run()
|- IRQs deaktivieren (__disable_irq)
|- Idle-Task registrieren (Slot 8, WFI-Loop)
|- NVIC-Prioritaeten konfigurieren:
|   SVCall  -> 0x00 (hoechste, fuer Bootstrap)
|   TIM7    -> 0x80 (Scheduler-Tick)
|   PendSV  -> 0xFF (niedrigste, Context-Switch)
|- FPU lazy-stacking aktivieren (ASPEN + LSPEN)
|- _Task_SelectNext() -> waehlt ersten bereiten Task
|- Fault-Handler aktivieren (Hard/Mem/Bus/Usage)
|- IRQs aktivieren (__enable_irq)
\~ SVC #0 ausführen des SVC_Handler
    |
    |
    SVC_Handler (naked, Assembler):
    1. _task_current laden (Slot-Index des ersten Tasks)
    2. TCB-Adresse berechnen: _tasks + (index × 28 Bytes)
    3. PSPLIM aus _task_psplims laden und setzen (Stack-Guard)
    4. saved_sp aus TCB laden -> PSP setzen
       (Zeigt auf den initialisierten Exception-Frame)
    5. R4-R11 vom Stack wiederherstellen
    6. PSP setzen
    7. CONTROL Register setzen:
       - SPSEL = 1 (PSP im Thread-Modus)
       - nPRIV = 0 (unprivileged - Task läuft im User-Mode)
    8. _StartTick() aufrufen:
       - TIM7 NVIC pending löschen
       - TIM7 NVIC enable
       - TIMER_Start(TIM7)
       - _scheduler_running = 1
    9. bx lr -> Sprung zur Task-Funktion (PC aus Exception-Frame)
```

Listing 7.11: Bootstrap prozess

7.6.3.2. Periodische Aktivierung (Tim7_IRQhandler)

Alle 1ms löst der TIM7-Interrupt aus und steuert den Scheduling-Ablauf. Jeder registrierte Task welcher nicht TaskDeleted oder TaskSuspended ist wird auf Ablauf des Ausführungs Intervalls geprüft. Tasks dessen Intervalle abgelaufen sind werden in TaskReady zustand versetzt. Damit der Task auswahl Algorithmus diese Auswählen kann. Im Anschluss wird die PendSV Ausführung angefordert.

PendSV wird nicht sofort ausgeführt, sondern läuft erst, wenn alle anderen ISRs abgearbeitet sind.

7.6.3.3. Preemptiver Context-Switch (PendSV_Handler)

Die eigentliche Kontextwahl erfolgt im PendSV_Handlerin reiner Assembler-Implementierung. PendSV hat die niedrigste NVIC-Priorität (0xFF), wird also erst nach Abschluss aller anderen Interrupts ausgeführt.

```

PendSV_Handler:
1. IRQs deaktivieren (cpsid i) -> Verhindert Race-Condition während Context-Save
2. Aktuellen PSP lesen (mrs r0, psp)
3. Callee-Saved Register sichern:
   -> R4-R11 werden auf den aktuellen Task-Stack abgelegt
4. FPU-Kontext sichern (bedingt):
   -> Wenn EXC_RETURN Bit 4 = 0 wurde FPU benutzt:
       S16-S31 werden ebenfalls gespeichert (16 Words)
   -> Wenn Bit 4 = 1: kein FPU-Frame, Speicher gespart
5. Stack-Pointer in TCB sichern:
   TCB->saved_sp = r0
   TCB->saved_exc_return = lr
6. _Task_SelectNext() aufrufen (C-Funktion):
   |- Scannt alle Tasks (Round-Robin ab _task_current+1)
   |- Wählt hohechstpriorisierten TaskReady-Task
   |- Falls keiner bereit: IDLE_TASK (Slot 8)
   |- Aktualisiert CPU-Zyklen-Statistiken:
       - Vorherigen Task: total_cycles += active_cycles
       - Preemption-Zähler erhöhen (falls Taskwechsel)
       - Nächsten Task: invocations++, start_cycle setzen
   \- Gibt Index des nächsten Tasks zurück
7. _task_current = neuer Index
8. Neuen TCB laden
9. PSPLIM für neuen Task setzen:
   -> isb (Instruction Synchronization Barrier)
10. Stack-Pointer vom neuen Stack wiederherstellen
11. EXC_RETURN laden
12. FPU-Kontext wiederherstellen (bedingt):
   -> S16-S31 nur wenn FPU beim letzten Speichern benutzt
13. Core-Register wiederherstellen:
   -> R4-R11 werden vom neuen Stack geladen
14. PSP auf neuen Stack setzen
15. IRQs aktivieren
16. bx lr
   -> EXC_RETURN führt zur Task-Rückkehr:
       - Thread-Modus
       - PSP als Stack-Pointer
       - Stack-Frame automatisch popped (R0-R3,R12,LR,PC)
       - CPU springt zu PC (nächste Instruktion des Tasks)

```

Listing 7.12: PendSV_Handler() in Hochsprache

7.6.4. Task-Selektion

```

Task_SelectNext():
    start = (_task_current + 1) mod IDLE_INDEX

    best = -1
    best_prio = 0xFF

    For n = 0 to (MAX_TASKS - 2):
        i = (start + n) mod IDLE_INDEX
        If (tasks[i].state == TaskReady):
            If (tasks[i].priority < best_prio):
                best = i
                best_prio = tasks[i].priority

    If best == -1:
        best = IDLE_TASK_INDEX // Slot 8

    Update Statistics
    Return best

```

Bei gleicher Priorität entscheidet die Reihenfolge der Slot-Indizes (implizierter Round-Robin). Der Algorithmus scannt zirkulär ab dem nächsten Slot und wählt den Task mit der niedrigsten Prioritätszahl (0 = höchste).

7.6.5. Fault handling

Während der Ausführung der Tasks können sogenannte *Faults* (Ausnahmestände) auftreten, die vom Cortex-M55-Prozessor erkannt und an den registrierten Fault-Handler weitergeleitet werden. Der Cortex-M55 unterscheidet zwischen mehreren Fault-Klassen, von denen vier relevant sind:

- **UsageFault:** Wird bei Ausführungsfehlern ausgelöst, wie etwa dem Versuch einer Division durch Null, dem Ausführen eines ungültigen oder nicht vorhandenen Befehls, oder bei einer Stack-Grenzverletzung (PSPLIM-Violation), die auf einen Stack Overflow hinweist.
- **BusFault:** Entsteht bei Speichertransaktionsfehlern, etwa dem Zugriff auf einen nicht vorhandenen Speicherbereich oder einem fehlerhaften Buszugriff während einer Lese- oder Schreiboperation.
- **MemManage Fault:** Wird bei Verletzungen der Speicher Protection Unit (MPU) ausgelöst, beispielsweise bei unerlaubten Speicherzugriffen auf geschützte Bereiche. **Anmerkung:** Da in dieser Konfiguration keine MPU aktiv ist, wird dieser Fault hauptsächlich durch Eskalation aus anderen Fault-Typen oder durch Speicherzugriffe auf ungültige Adressen (z. B. Null-Pointer-Dereferenzierung) ausgelöst.
- **HardFault:** Dient als Abfang-Interrupt für schwere Programmfehler und wird ausgelöst, wenn ein aufgetretener Fault nicht vom entsprechenden spezialisierten Fault-Handler (UsageFault, BusFault oder MemManage) behandelt werden kann, etwa weil dieser deaktiviert ist oder der Fehler eskaliert.

Diese Faults werden durch den Scheduler abgefangen. Dazu registriert der Scheduler in `SCHEDULER_Tasks_run()` die entsprechenden Fault-Handler (`HardFault_Handler`, `UsageFault_Handler`, `BusFault_Handler`, `MemManage_Handler`) als naked-Assemble-Funktionen, die den aktiven Stack-Frame (MSP oder PSP) ermitteln und an die C-Funktion `_FaultHandler_C()` übergeben. Der Handler erfasst den vollständigen diagnostischen Zustand des Systems, einschliesslich aller SCB-Register (CFSR, HFSR, MMFAR, BFAR), der Stack-Pointer (MSP, PSP, PSPLIM), des CONTROL-Registers, des EXC_RETURN-Wertes sowie der gesicherten Register R0--R3, R12, LR, PC und xPSR und überträgt diese Information per USART als Diagnoseausgabe. Anschliessend wird das Programm mittels Breakpoint-Befehl (`_BKPT(0)`) für den Debugging-Prozess angehalten. Eine automatische Fehlerbehandlung oder Wiederherstellung des Betriebs findet nicht statt.

7.6.6. CPU-Zyklen-Messung & Statistik

Um die Laufzeiteigenschaften der einzelnen Tasks zur Laufzeit bewerten zu können, implementiert der Scheduler eine hardwaregestützte CPU-Zyklen-Messung. Als Messquelle dient der DWT (Data Watchpoint and Trace) Modul des Cortex-M55-Prozessors, dessen Zyklenzähler `DWT->CYCCNT` bei Systemtaktfrequenz (200 MHz) hochauflösend incremental zählt. Die Aktivierung erfolgt in `SCHEDULER_System_init()` durch Setzen der `DEMCR_TRCENA` und `DWT_CYCCNTENA` Bits.

Per-Task-Erfassung

Für jeden Task werden in der Struktur `_Task_Stats_TypeDef` folgende Metriken geführt:

- `total_cycles`: Summe der reinen Applikations-Zyklen abzüglich der in Interrupt-Kontext verbrachten Zeit
- `total_preempts`: Anzahl der Unterbrechungen durch höherpriorisierte Tasks
- `total_invocations`: Gesamtzahl der Task-Aufrufe
- `min_cycles / max_cycles`: Minimaler und maximaler Zykelverbrauch pro Aufruf

Die Messung erfolgt bei jedem Context-Switch im `_Task_SelectNext()`: Der seit dem letzten Start vergangene Zeitraum wird ermittelt und von den im `_task_isr_cycles`-Array kumulierten ISR-Zyklen subtrahiert. Das Ergebnis repräsentiert den reinen Applikationsanteil (%ACT) des vorherigen Tasks.

ISR-Cycle-Tracking

Um Interrupt-Zyklen sauber von Task-Zyklen zu trennen, muss jede Peripheral-ISR am Ein- und Ausgang die Funktionen `SCHEDULER_ISR_enter()` und `SCHEDULER_ISR_exit()` aufrufen. Bei Einstieg wird der aktuelle `DWT->CYCCNT`-Wert gespeichert. Bei Ausgang wird die Differenz berechnet und dem unterbrochenen Task in `_task_isr_cycles` zugeordnet.

Ausgabe

Alle 1000 ms, getriggert durch den `TIM7_IRQHandler`, wird `_stats_pending` gesetzt und im Idle-Task die Funktion `_PrintStats()` aufgerufen. Diese erzeugt eine tabellarische Ausgabe mit den Spalten `Name`, `cycle` (Gesamt-Zyklen), `min`, `max`, `avg` (durchschnittliche Zyklen pro Invocation), `%ACT` (Aktiv-Anteil in Prozent), `Prempt` (Preemption-Zaehler) und `Invoc` (Invocation-Anzahl). Zusätzlich werden der Gesamtanteil idle-Zeit, der ISR-Overhead und die CPU-Auslastung in Prozent ausgegeben. Die Statistik wird nach jedem Druckvorgang zurückgesetzt, sodass jeder Messzeitraum den Zeitraum zwischen zwei aufeinanderfolgenden Ausgaben repräsentiert.

7.7. Integration der NPU und einheitliche KI-Schnittstelle

Die Ausführung der drei neuronalen Netze erfolgt überwiegend auf dem im STM32N657 integrierten Neural-ART Accelerator. Für dessen Verwendung müssen zunächst die zugehörigen Takt- und Speicherbereiche sowie die ATON-Laufzeitumgebung eingerichtet werden. Anschließend werden die drei konvertierten Netzwerkinstanzen für die Handdetektion, Landmark-Erkennung und Fingeralphabetklassifikation initialisiert.

Um die Middleware-spezifischen Aufrufe nicht unmittelbar in der Ablaufsteuerung verwenden zu müssen, wurde mit `simple_ai` eine gemeinsame KI-Schnittstelle umgesetzt. Diese fasst die Initialisierung der KI-Komponenten zusammen, vereinheitlicht die Statusrückgaben und stellt Funktionen für die Ausführung der neuronalen Netze bereit.

7.7.1. Initialisierung des Neural-ART Accelerators

Die Initialisierung der KI-Komponenten erfolgt zentral durch `AI_Init()`. Die Funktion prüft zunächst, ob die Initialisierung bereits durchgeführt wurde. In diesem Fall wird unmittelbar `AI_STATUS_OK` zurückgegeben. Dadurch kann die Funktion mehrfach aufgerufen werden, ohne die Laufzeitumgebung und Netzwerkinstanzen erneut einzurichten. Zu Beginn werden die benötigten Takt- und Speicherbereiche aktiviert.

Innerhalb dieser Funktion wird außerdem die Konfiguration des NPU-Caches vorgenommen. Die anschließende Initialisierung des CacheAXI erfolgt über `HAL_CACHEAXI_Init()`. CacheAXI stellt die für die NPU erforderliche Cache-Infrastruktur bereit. Da die Initialisierung durch die von STMicroelectronics bereitgestellte HAL-Komponente erfolgt, bildet sie eine Ausnahme von der ansonsten überwiegend registerbasierten Treiberimplementierung. Schlägt dieser Schritt fehl, wird die Initialisierung mit `AI_STATUS_CACHEAXI` abgebrochen.

Der verwendete CACHEAXI-HAL-Treiber erwartet für seine interne Zeitüberwachung die Funktion `HAL_GetTick()`. Da im Projekt keine separate HAL-Zeitbasis verwendet wird, wurde mit `ai_hal_adapter.c` ein kleiner Adapter umgesetzt. Die Funktion `HAL_GetTick()` greift auf den bereits vorhandenen Millisekundenzähler des Software-Schedulers zurück. Dadurch kann der CACHEAXI-Treiber seine vorgesehenen Zeitüberwachungen verwenden, ohne zusätzlich den vollständigen HAL-Zeitbasismechanismus in das Projekt zu integrieren.

Nach erfolgreicher Einrichtung der Hardware wird `LL_ATON_RT_RuntimeInit()` aufgerufen. Diese Funktion initialisiert die ATON-Laufzeitumgebung, über die die konvertierten neuronalen Netze ausgeführt werden. Anschließend werden die drei Netzwerkinstanzen nacheinander initialisiert:

1. Handdetektion über `PALM_Init()`
2. Landmark-Erkennung über `LANDMARK_Init()`
3. Fingeralphabetklassifikation über `FINGERALPHABET_Init()`

Jede Initialisierungsfunktion ermittelt die vom jeweiligen Netzwerk verwendeten Ein- und Ausgabepuffer und bereitet dessen internen Zustand vor. Gibt eine der Funktionen einen Fehlerstatus zurück, wird die weitere Initialisierung abgebrochen und der Status an die aufrufende Komponente weitergereicht. Erst nach erfolgreicher Initialisierung aller drei Modelle wird `ai_initialized` gesetzt und `AI_STATUS_OK` zurückgegeben.

7.7.2. Einheitliche KI-Schnittstelle

Die von der ATON-Middleware bereitgestellten Funktionen sind an die Laufzeitumgebung und die jeweils erzeugten Netzwerkinstanzen gekoppelt. Um diese Abhängigkeiten nicht unmittelbar in die Ablaufsteuerung und die Vor- bzw. Nachverarbeitung zu übernehmen, wurde mit `simple_ai` eine gemeinsame KI-Schnittstelle implementiert. Sie bündelt die allgemeinen Funktionen zur Initialisierung und Ausführung der neuronalen Netze und stellt einheitliche Statuswerte für alle Modellkomponenten bereit.

Für die Rückgabe von Fehlerzuständen wird der gemeinsame Aufzählungstyp `AI_Status_TypeDef` verwendet. Dadurch können die zentralen und modellspezifischen KI-Funktionen ihre Ergebnisse in einer einheitlichen Form an die aufrufenden Komponenten weitergeben. Die Ablaufsteuerung muss somit keine unterschiedlichen Fehlerdarstellungen der einzelnen Modelle auswerten.

Tabelle 7.9.: Einheitliche Statuswerte der KI-Komponenten

Statuswert	Bedeutung
<code>AI_STATUS_OK</code>	Die aufgerufene Funktion wurde erfolgreich ausgeführt.
<code>AI_STATUS_NOT_INITIALIZED</code>	Die benötigte KI-Komponente wurde noch nicht initialisiert.
<code>AI_STATUS_CACHEAXI_ERROR</code>	Bei der Einrichtung des CacheAXI ist ein Fehler aufgetreten.
<code>AI_STATUS_INVALID_BUFFER</code>	Ein benötigter Ein- oder Ausgabepuffer ist ungültig.
<code>AI_STATUS_PREPROCESS_ERROR</code>	Die Vorverarbeitung der Eingabedaten ist fehlgeschlagen.
<code>AI_STATUS_POSTPROCESS_ERROR</code>	Die Nachverarbeitung der Modellausgabe ist fehlgeschlagen.
<code>AI_STATUS_RUNTIME_ERROR</code>	Während der Ausführung eines Modells ist ein Fehler aufgetreten.

Die eigentliche Ausführung einer Netzwerkinstanz wird durch `AI_RuntimeRunNetwork()` gekapselt. Der Funktion wird ein Zeiger auf die jeweilige `NN_Instance_TypeDef` übergeben. Dadurch kann dieselbe Funktion für die Handdetektion, die Landmark-Erkennung und die Fingeralphabetklassifikation verwendet werden. Ist der übergebene Zeiger ungültig, wird die Ausführung unmittelbar abgebrochen. Intern ruft die Funktion wiederholt `LL_ATON_RT_RunEpochBlock()` auf. Die ATON-Laufzeitumgebung verarbeitet das Netzwerk dabei abschnittsweise und liefert nach jedem Aufruf einen Status zurück. Mit `LL_ATON_RT_WFE` wird signalisiert, dass zunächst auf ein Hardwareereignis gewartet werden muss. In diesem Fall erfolgt das Warten über `LL_ATON_OSAL_WFE()`. Der Status `LL_ATON_RT_NO_WFE` zeigt dagegen an, dass die Verarbeitung ohne vorheriges Warten fortgesetzt werden kann. Die Aufrufe werden wiederholt, bis die Laufzeitumgebung einen abschließenden Status zurückgibt.

Die Funktion liefert nur dann `true`, wenn die Netzwerkausführung mit `LL_ATON_RT_DONE` beendet wurde. Alle anderen abschließenden Zustände werden als fehlgeschlagene Ausführung behandelt und durch `false` signalisiert. Für die aufrufenden Modellkomponenten entsteht dadurch eine vereinfachte Schnittstelle, bei der die einzelnen ATON-Rückgabewerte nicht außerhalb von `simple_ai` ausgewertet werden müssen.

Die modellspezifischen Komponenten kapseln darauf aufbauend die jeweiligen Netzwerkinstanzen sowie deren Ein- und Ausgabepuffer. Sie prüfen ihren Initialisierungszustand und die übergebenen Puffer, führen die Netzwerkinstanz über `AI_RuntimeRunNetwork()` aus und übersetzen das Ergebnis in einen Wert des gemeinsamen Typs `AI_Status_TypeDef`. Die Vor- und Nachverarbeitung verbleibt hingegen

in den jeweiligen Modellkomponenten, da sie sich zwischen Handdetektion, Landmark-Erkennung und Fingeralphabetklassifikation unterscheidet.

Damit bildet `simple_ai` die gemeinsame Verbindung zwischen den erzeugten Netzwerkinstanzen und der übergeordneten Ablaufsteuerung. Middleware-spezifische Details der Netzwerkausführung bleiben innerhalb dieser Schnittstelle gekapselt, während die modellspezifischen Unterschiede weiterhin in den zugehörigen Komponenten behandelt werden.

7.8. Vor- und Nachverarbeitungsschritte

Die drei neuronalen Netze verwenden unterschiedliche Eingabedaten und liefern Ausgaben, die nicht unmittelbar durch das jeweils nachfolgende Modell verarbeitet werden können. Daher sind zwischen den einzelnen Inferenzschritten mehrere Vor- und Nachverarbeitungen erforderlich. Diese umfassen sowohl hardwaregestützte Operationen innerhalb der DCMIPP als auch durch den Hauptprozessor ausgeführte Transformationen der Bild- und Landmark-Daten.

7.8.1. DCMIPP-gestützte Bildvorverarbeitung

Die Kamera überträgt die aufgenommenen Bilder als RAW-Bayer-Daten mit einer Auflösung von 2592x1944 Pixeln über die CSI-Schnittstelle. Da weder die Displayausgabe noch die neuronalen Netze diese Daten unmittelbar verarbeiten können, übernimmt die Digital Camera Interface Pixel Pipeline (DCMIPP) einen wesentlichen Teil der erforderlichen Bildvorverarbeitung [12, s. 1815 ff.].

Die DCMIPP wandelt die RAW-Bayer-Daten zunächst in ein RGB-Bild um. Hierzu werden die Demosaikierung der RAW10-Bilddaten, die Farbkorrektur, die Belichtungsanpassung und die Gammakorrektur innerhalb der Bildpipeline durchgeführt (STMicroelectronics, n.d.). Das Ergebnis wird im RGB888-Format ausgegeben. Dadurch müssen diese rechenintensiven Verarbeitungsschritte nicht nachträglich durch den Hauptprozessor auf den bereits gespeicherten Kamerabildern ausgeführt werden.

Die verwendeten Parameterwerte wurden empirisch bestimmt. Hierzu wurden die entsprechenden DCMIPP-Register schrittweise angepasst und die Auswirkungen auf das ausgegebene Kamerabild visuell bewertet. Einstellungen, die unter den vorgesehenen Einsatzbedingungen eine geeignete Bild Darstellung ergaben, wurden anschließend in die feste Konfiguration der Bildpipeline übernommen.

Aus dem gemeinsamen Kameradatenstrom werden zwei getrennte Ausgabepfade erzeugt. Pipe 1 dient der Displayausgabe und stellt gleichzeitig das Ausgangsbild für die Landmark-Erkennung bereit. Pipe 2 erzeugt dagegen das Eingabebild für die Handdetektion.

Für Pipe 1 wird das Sensorbild zunächst auf das Seitenverhältnis des Displays zugeschnitten. Der horizontale Bildbereich bleibt vollständig erhalten, während das Bild vertikal zentriert beschnitten wird. Der entstandene Ausschnitt wird anschließend durch die Downscaling-Einheit der DCMIPP auf eine Auflösung von 800x480 Pixeln verkleinert und im RGB888-Format im externen PSRAM abgelegt. Die von Pipe 1 erzeugten Bilder werden über vier Hintergrundbildpuffer verwaltet. Die DCMIPP schreibt jeweils in den für die Aufnahme vorgesehenen Puffer, während weitere Puffer für die Displayausgabe, die KI-Verarbeitung und das Einzeichnen der Handregion beziehungsweise der Landmarks verwendet werden. Nach Abschluss eines Bildes wird die Zieladresse der DCMIPP auf den nächsten Aufnahme-puffer umgeschaltet.

Pipe 2 verarbeitet den vollständigen Sensorbereich und erzeugt das Eingabebild für die Handdetektion. Da deren Eingangsauflösung mit 192x192 Pixeln deutlich unterhalb der Sensorauflösung liegt, wird das Bild vor der eigentlichen Skalierung horizontal und vertikal dezimiert. Dadurch reduziert sich die Auflösung zunächst von 2592x1944 auf 1296x972 Pixel. Anschließend übernimmt der DCMIPP-Downsizer die Skalierung auf 192x192 Pixel. Da vor der Skalierung kein quadratischer Bildausschnitt gebildet wird, wird das Seitenverhältnis des vollständigen Sensorbildes dabei an die quadratische Modelleingabe angepasst. Auch dieser Ausgabepfad verwendet das RGB888-Format.

Für Pipe 2 wird der Double-Buffer-Modus der DCMIPP verwendet. Die erzeugten Bilder werden abwechselnd in zwei Eingabebildpuffern gespeichert. Während die DCMIPP einen Puffer mit einem neuen Bild beschreibt, kann der zuvor fertiggestellte Puffer für die Handdetektion verwendet werden. Der jeweils abgeschlossene Puffer wird über den Frame-Callback bestimmt und für die weitere Verarbeitung markiert.

Tabelle 7.10.: Konfigurationsparameter der DCMIPP-Bildpfade

Parameter	Pipe 1	Pipe 2	Begründung
output_width / output_height	800x480	192x192	Entsprechen der Displayauflösung, bzw. Eingangsgröße der Handdetektion.
output_format	RGB888	RGB888	Display und KI-Vorverarbeitung arbeiten mit drei Farbkanälen und jeweils 8 Bit pro Kanal.
output_bpp	3	3	RGB888 benötigt drei Byte pro Pixel (für jeden Kanal 8 Bit). Der Wert wird unter anderem zur Berechnung des Speicher-Pitchs verwendet.
enable_crop	aktiviert	aktiviert	Pipe 1 benötigt einen Zuschchnitt auf das Seitenverhältnis des Displays. Pipe 2 verarbeitet den vollständigen Sensorbereich.
crop_x	0	0	Horizontal wird kein Bereich abgeschnitten.
crop_y	Zentriert vertikaler Zuschchnitt	0	Pipe 1 entfernt oben und unten Bildbereiche, um das Sensorformat ohne Verzerrung an 800x480 anzupassen. Pipe 2 verwendet das vollständige Bild.
crop_width	2592	2592	Die vollständige Sensorbreite wird verwendet.
crop_height	1555	1944	Pipe 1 erhält das Displayseitenverhältnis 5:3. Pipe 2 behält die vollständige Sensorhöhe.
enable_downsize	aktiviert	aktiviert	Beide Ausgaben sind wesentlich kleiner als das Sensorbild.
enable_decimate	deaktiviert	aktiviert	Nur Pipe 2 benötigt vor dem Downscaling eine zusätzliche Halbierung.
decimate_h / decimate_v	–	1, 1	Die Auflösung wird horizontal und vertikal jeweils durch zwei geteilt.
enable_swap	deaktiviert	deaktiviert	Die Farbkanäle sind bereits korrekt. Ein Swap ist nicht notwendig.
enable_gamma	aktiviert	aktiviert	Die Gammakorrektur verbessert die Helligkeitsdarstellung.
enable_dbm	deaktiviert	aktiviert	Pipe 1 verwendet eine softwareseitige Rotation von vier Bildpuffern, da die Anwendung mehr als zwei Puffer benötigt. Pipe 2 verwendet zwei hardwareseitig wechselnde Eingabepuffer.

Durch die hardwaregestützte Vorverarbeitung werden nur die tatsächlich benötigten Bildauflösungen in den externen Speicher geschrieben. Dies reduziert sowohl den Speicherbedarf als auch die vom Hauptprozessor zu verarbeitende Datenmenge. Die weiteren auf der CPU ausgeführten Vor- und Nachverarbeitungsschritte können dadurch unmittelbar auf den bereits aufbereiteten RGB888-Bilddaten arbeiten.

7.8.2. Nachverarbeitung der Handdetektion

7.8.2.1. Ankerbasierte Modellausgabe

Das Modell zur Handdetektion verwendet ein ankerbasiertes Detektionsverfahren. Hierfür sind in der Ankertabelle `pd_anchors` insgesamt 2016 Ankerpositionen definiert. Diese repräsentieren unterschiedliche Positionen und Größen möglicher Handregionen innerhalb des 192x192 Pixel großen Eingabebildes. Für jeden Anker gibt das Modell einen Konfidenzwert sowie mehrere Regressionswerte aus. Der Konfidenzwert beschreibt, wie wahrscheinlich sich im Bereich des jeweiligen Ankers eine Hand befindet. Die Regressionswerte enthalten die Abweichungen zwischen dem Anker und der vorhergesagten Handregion sowie die Positionen der zugehörigen Schlüsselpunkte [33].

Die Anker bilden damit ein festes Suchraster über dem Eingabebild. Das Modell muss die Position und Ausdehnung einer Hand nicht vollständig unabhängig bestimmen, sondern sagt für jeden Anker die räumlichen Abweichungen zur tatsächlichen Handregion voraus. Da mehrere benachbarte Anker auf

dieselbe Hand reagieren können, entstehen üblicherweise mehrere ähnliche Erkennungskandidaten, die in der anschließenden Nachverarbeitung bereinigt werden müssen.

7.8.2.2. Auswahl und Filterung der Erkennungskandidaten

Für jeden der 2016 Anker wird zunächst geprüft, ob der ausgegebene Konfidenzwert den festgelegten Schwellwert erreicht. Die Konfidenzwerte liegen als Logits vor und müssen für die weitere Bewertung durch eine Sigmoidfunktion in einen Wahrscheinlichkeitswert zwischen 0 und 1 überführt werden [34]:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Eine direkte Berechnung würde die Exponentialfunktion $\exp()$ erfordern. Da die verwendete Floating Point Unit keine Exponentialfunktion als direkte Hardwareoperation bereitstellt, müsste diese durch eine mathematische Bibliotheksroutine softwareseitig berechnet werden. Zur Verringerung des Rechenaufwands wird die Sigmoidfunktion daher durch die folgende rationale Funktion approximiert:

$$\hat{\sigma}(x) = \frac{0,5 + 0,25x}{1 - 0,25x + 0,125x^2}$$

Um die Sigmoidapproximation nicht für alle 2016 Anker ausführen zu müssen, wird der festgelegte Konfidenzschwellwert zunächst in den Logit-Raum überführt [35, s. 340 ff.]:

$$t_{\text{Logit}} = \ln\left(\frac{t}{1-t}\right)$$

Die vom Modell ausgegebenen Logits können dadurch unmittelbar mit dem berechneten Logit-Schwellwert verglichen werden. Nur Kandidaten, die diesen Schwellwert erreichen oder überschreiten, werden vollständig dekodiert und mithilfe der approximierten Sigmoidfunktion in einen Wahrscheinlichkeitswert umgerechnet. Anschließend werden die Regressionswerte der verbleibenden Kandidaten unter Berücksichtigung der jeweils zugehörigen Ankerposition dekodiert. Dabei werden der Mittelpunkt, die Breite und Höhe der vorhergesagten Handregion sowie die vom Modell ausgegebenen Schlüsselpunkte bestimmt. Die berechneten Koordinaten und Abmessungen werden auf die Größe des Modelleingangs normiert. Kandidaten mit einer ungültigen Breite oder Höhe sowie nicht endlichen Zahlenwerten werden verworfen.

Um den Speicherbedarf und den Rechenaufwand der weiteren Verarbeitung zu begrenzen, werden höchstens 20 Erkennungskandidaten zwischengespeichert. Diese Obergrenze wird durch die Konstante `PALM_PP_MAX_CANDIDATES` festgelegt. Solange die maximale Anzahl noch nicht erreicht ist, wird jeder gültige Kandidat übernommen. Ist der Kandidatenspeicher bereits vollständig belegt, wird zunächst der Kandidat mit der geringsten Wahrscheinlichkeit ermittelt. Dieser wird nur dann ersetzt, wenn der neue Kandidat eine höhere Wahrscheinlichkeit aufweist. Auf diese Weise werden aus den Ergebnissen der 2016 Ankerpositionen ausschließlich die 20 wahrscheinlichsten Kandidaten für die weitere Verarbeitung berücksichtigt.

Die ausgewählten Kandidaten werden anschließend absteigend nach ihrer Wahrscheinlichkeit sortiert. Da mehrere benachbarte Anker auf dieselbe Hand reagieren können, entstehen häufig mehrere stark überlappende Handregionen. Zur Entfernung dieser Mehrfacherkennungen wird eine Non-Maximum Suppression (NMS) durchgeführt [36]. Hierzu wird jeder Kandidat mit den bereits übernommenen Handregionen verglichen. Als Maß für ihre räumliche Überschneidung dient die Intersection over Union (IoU). Sie beschreibt das Verhältnis zwischen der Schnittfläche und der Vereinigungsfläche zweier Begrenzungsrahmen [37, s. 3]:

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Die Kandidaten werden in absteigender Reihenfolge ihrer Wahrscheinlichkeit verarbeitet. Erreicht oder überschreitet die IoU eines Kandidaten mit einer bereits übernommenen Handregion den festgelegten Schwellwert, wird der Kandidat verworfen. Dadurch bleibt von mehreren stark überlappenden Erkennungen in der Regel nur der Kandidat mit der höchsten Wahrscheinlichkeit erhalten. Nach Abschluss der Non-Maximum Suppression wird der stärkste verbleibende Kandidat als Ergebnis der Handdetektion übernommen. Das Ergebnis enthält die Wahrscheinlichkeit, den Mittelpunkt und die Abmessungen der erkannten Handregion, die zugehörigen Schlüsselpunkte sowie den verwendeten Ankerindex.

Um kurzzeitige Fehldetektionen und einzelne Aussetzer zu reduzieren, wird das Ergebnis zusätzlich über mehrere Ausführungen hinweg gefiltert. Eine Hand gilt erst dann als bestätigt, wenn in zwei aufeinanderfolgenden Auswertungen eine gültige Detektion vorliegt. Bei einer ungültigen Detektion wird der positive Zähler zurückgesetzt. Umgekehrt wird der erkannte Zustand erst aufgehoben, wenn in drei aufeinanderfolgenden Auswertungen keine gültige Handdetektion vorliegt. Einzelne Aussetzer führen dadurch nicht unmittelbar zum Verlust der erkannten Handregion.

7.8.2.3. Erzeugung der initialen Region of Interest

Aus der bestätigten Handdetektion wird die initiale Region of Interest für das Handlandmark-Modell erzeugt. Die Handdetektion wird auf dem von Pipe 2 bereitgestellten Bild ausgeführt, während der Eingabeausschnitt für das Landmark-Modell aus dem Displaybild von Pipe 1 entnommen wird. Aufgrund der unterschiedlichen Auflösungen und Bildausschnitte müssen die normierten Koordinaten der Handdetektion zunächst in das Koordinatensystem von Pipe 1 transformiert werden. In horizontaler Richtung bilden beide Bildpfade die vollständige Sensorbreite ab. Die horizontale Position und Breite können deshalb unmittelbar mit der Breite des Displaybildes skaliert werden. Pipe 1 verwendet jedoch einen vertikal zentrierten Ausschnitt des Sensorbildes. Bei der Transformation der vertikalen Koordinaten müssen daher zusätzlich die Höhe und der Offset dieses Ausschnitts berücksichtigt werden. Diese Koordinatentransformation wurde aus der Konfiguration der beiden DCMIPP-Pipelines abgeleitet.

Neben der Begrenzungsbox stellt das Modell mehrere Schlüsselpunkte der Hand bereit. In Anlehnung an die MediaPipe-Referenzimplementierung werden zwei dieser Punkte zur Bestimmung der Handorientierung verwendet. Der zwischen ihnen gebildete Richtungsvektor wird auf einen Zielwinkel von 90° und damit auf die vertikale Achse der Region ausgerichtet [38]. Da die y -Achse in Bildkoordinaten nach unten zeigt, wird ihre Richtungsdivergenz mit umgekehrtem Vorzeichen berücksichtigt. Der Rotationswinkel ergibt sich damit zu:

$$\alpha = \frac{\pi}{2} - \text{atan2}(-\Delta y, \Delta x)$$

Dabei bezeichnen Δx und Δy die Koordinatendifferenzen zwischen den verwendeten Schlüsselpunkten. Der berechnete Winkel wird anschließend auf den Bereich von $-\pi$ bis π normiert. Kann aufgrund ungültiger Koordinaten kein eindeutiger Winkel bestimmt werden, wird für die Region eine Rotation von 0 verwendet.

Da die Begrenzungsbox der Handdetektion hauptsächlich die Handfläche umfasst, wird sie für die Landmark-Erkennung erweitert. Ihr Mittelpunkt wird entlang der rotierten lokalen Vertikalachse um die Hälfte der ursprünglichen Höhe in Richtung der Finger verschoben. Anschließend wird die längere Seite der Begrenzungsbox bestimmt und mit dem Faktor 2,6 skaliert. Der berechnete Wert wird für die Breite und Höhe verwendet, sodass eine quadratische Region entsteht. Die Verschiebung, Skalierung und quadratische Ausrichtung orientieren sich ebenfalls an der MediaPipe-Referenzimplementierung [38]. Aus dem Mittelpunkt, der Seitenlänge und dem Rotationswinkel werden abschließend die vier Eckpunkte der Region berechnet. Die so erzeugte Region bildet die Grundlage für den 224×224 Pixel großen Eingabeausschnitt des Handlandmark-Modells. Dessen Erzeugung wird im folgenden Abschnitt zur Vorverarbeitung der Landmark-Erkennung beschrieben.

7.8.3. Vorverarbeitung der Landmark-Erkennung

Die aus der Handdetektion erzeugte Region of Interest besitzt eine variable Position, Größe und Rotation innerhalb des von Pipe 1 bereitgestellten Kamerabildes. Das Handlandmark-Modell erwartet dagegen ein quadratisches Eingabebild mit einer festen Auflösung von 224×224 Pixeln. Daher muss die Region aus dem Kamerabild entnommen, entsprechend ihrer Rotation ausgerichtet und auf die Eingangsgröße des Modells übertragen werden. Hierzu wird für jedes Pixel des Zielbildes die zugehörige Position innerhalb der Region bestimmt. Die Zielkoordinaten werden zunächst auf den Bereich von -0,5 bis 0,5 normiert und anschließend mit der Breite und Höhe der Region skaliert. Unter Berücksichtigung des Rotationswinkels werden die lokalen Koordinaten in das Koordinatensystem des Kamerabildes überführt:

$$\begin{aligned} x_{\text{Quelle}} &= c_x + x_{\text{lokal}} \cos(\alpha) - y_{\text{lokal}} \sin(\alpha), \\ y_{\text{Quelle}} &= c_y + x_{\text{lokal}} \sin(\alpha) + y_{\text{lokal}} \cos(\alpha). \end{aligned}$$

Dabei beschreiben c_x und c_y den Mittelpunkt und α den Rotationswinkel der Region. Dadurch werden Skalierung, Position und Rotation gleichzeitig berücksichtigt. Damit wird die Hand unabhängig von ihrer Lage im ursprünglichen Kamerabild in eine einheitliche Ausrichtung überführt.

Die Transformation führt häufig zu Quellkoordinaten, die zwischen den ganzzahligen Pixelpositionen des Kamerabildes liegen. Der benötigte Farbwert wird deshalb durch eine bilineare Interpolation aus den vier benachbarten Pixeln berechnet. Dadurch werden Skalierungs- und Rotationsartefakte gegenüber einer einfachen Auswahl des nächstgelegenen Pixels reduziert (Getreuer, 2011). Reicht die Region über den Rand des Kamerabildes hinaus, werden die außerhalb des Bildes liegenden Bereiche im Modelleingang schwarz aufgefüllt. Bei der Adressierung der Bilddaten wird außerdem die im Speicher verwendete Zeilenlänge berücksichtigt.

7.8.4. Nachverarbeitung der Landmark-Erkennung

7.8.4.1. Rücktransformation der Landmarks

Das Handlandmark-Modell gibt für jeden der 21 Handlandmarks drei Koordinaten innerhalb des 224x224 Pixel großen Modelleingangs aus. Da dieser Eingabeausschnitt gegenüber dem ursprünglichen Kamerabild skaliert, verschoben und rotiert wurde, können die ausgegebenen Koordinaten nicht unmittelbar für die Anzeige oder die nachfolgende Zeichenklassifikation verwendet werden. Sie werden deshalb zunächst in das Koordinatensystem des von Pipe 1 bereitgestellten Kamerabildes zurücktransformiert.

Hierzu werden die x- und y-Koordinaten der Landmarks auf den Mittelpunkt des Modelleingangs bezogen und entsprechend der Größe der aktuellen Region of Interest skaliert. Anschließend werden sie anhand des Rotationswinkels der Region ausgerichtet und um deren Mittelpunkt verschoben. Die Transformation entspricht damit der Umkehrung der zuvor durchgeführten Erzeugung des Modelleingangs. Abschließend werden die x- und y-Koordinaten durch die Breite beziehungsweise Höhe des Kamerabildes geteilt und dadurch auf den Bereich des gesamten Bildes normiert. Die vom Modell ausgegebene z-Koordinate wird unverändert übernommen. Die zurücktransformierten Landmark-Koordinaten werden sowohl für die Anzeige und Zeichenklassifikation als auch für die Aktualisierung der Region of Interest verwendet. Dadurch kann die Hand in den nachfolgenden Kamerabildern anhand der bereits ermittelten Landmarks weiterverfolgt werden, ohne erneut eine vollständige Handdetektion ausführen zu müssen.

7.8.4.2. Aktualisierung der Tracking-Region

Für die Bestimmung der nächsten Tracking-Region werden nicht alle 21 Landmarks verwendet. Insbesondere die Fingerspitzen haben sich in Tests abhängig vom dargestellten Handzeichen stark bewegt und haben dadurch die Größe und Position der Region unnötig verändert. Stattdessen werden zwölf vergleichsweise stabile Punkte aus dem Bereich des Handgelenks, der Handfläche und der Fingerbasen berücksichtigt. Aus ihren minimalen und maximalen x- und y-Koordinaten wird zunächst eine Bounding-Box bestimmt. Deren Mittelpunkt, Breite und Höhe bilden die Grundlage der neuen Region.

Die Orientierung der Hand wird über eine zentrale Handflächenachse bestimmt. Hierzu wird zunächst der Mittelpunkt der vier Fingergrundgelenke von Zeige-, Mittel-, Ring- und kleinem Finger berechnet. Die Verbindung zwischen dem Handgelenk und diesem Mittelpunkt beschreibt die Ausrichtung der Handfläche, aus der der neue Rotationswinkel abgeleitet wird. Um abrupte Änderungen und ein sichtbares Springen der Region zu vermeiden, wird der neue Winkel nicht unmittelbar übernommen. Stattdessen fließen 20% der berechneten Winkeländerung in die nächste Region ein. Zusätzlich wird die Änderung pro Aktualisierung auf 0,08 Radiant, entsprechend etwa 4,6° begrenzt. Der resultierende Winkel wird anschließend auf den Bereich von $-\pi$ bis π normiert.

Abschließend wird die Region entlang ihrer lokalen vertikalen Achse leicht in Richtung der Finger verschoben. Die längere Seite der Begrenzungsbox wird mit dem Faktor 2,0 skaliert und sowohl für die Breite als auch für die Höhe verwendet. Dadurch entsteht erneut eine quadratische Region, die neben der Handfläche auch die Finger vollständig erfassen soll. Aus Mittelpunkt, Seitenlänge und Rotation werden die vier Eckpunkte der nächsten Region berechnet.

Die aktualisierte Region wird im folgenden Kamerabild erneut zur Erzeugung des 224x224 Pixel großen Modelleingangs verwendet. Erst wenn die Landmark-Erkennung über mehrere Durchläufe keine gültige Ausgabe mehr liefert, wird das Tracking zurückgesetzt und erneut die Handdetektion ausgeführt.

7.8.5. Vorverarbeitung des Klassifizierungsmodells

Das Klassifikationsmodell verarbeitet nicht das Kamerabild selbst, sondern die vom Handlandmark-Modell bestimmten Koordinaten. Der Modelleingang umfasst insgesamt 88 Merkmale und ist in zwei Bereiche mit jeweils 44 Merkmalen für die linke und rechte Hand unterteilt. Zu Beginn der Vorverarbeitung werden beide Bereiche mit den für eine fehlende Hand vorgesehenen Werten initialisiert. Anschließend wird anhand des vom Landmark-Modell ausgegebenen `Handedness`-Wertes bestimmt, welchem Bereich die erkannte Hand zugeordnet wird. Werte ab einem Schwellwert von 0,5 werden der rechten Hand und kleinere Werte der linken Hand zugeordnet. Für jede Hand werden die x- und y-Koordinaten der 21 Landmarks verwendet. Die z-Koordinaten werden bei der Zeichenklassifikation nicht berücksichtigt. Vor der weiteren Verarbeitung werden die horizontalen Koordinaten gespiegelt. Diese Transformation stellt die Koordinatenorientierung her, die auch bei der Erzeugung der Trainingsdaten verwendet wurde. Werte außerhalb des normierten Bereichs werden zuvor auf den Bereich von 0 bis 1 begrenzt. Um den Einfluss der Position der Hand innerhalb des Bildausschnitts zu verringern, werden die Landmark-Koordinaten relativ zum Handgelenk angegeben. Hierzu wird die Position des ersten Landmarks, das dem Handgelenk entspricht, von allen Landmark-Koordinaten abgezogen:

$$x_i^{\text{rel}} = x_i - x_0, \quad y_i^{\text{rel}} = y_i - y_0$$

Dadurch beschreiben die 42 resultierenden Merkmale überwiegend die geometrische Anordnung der Handpunkte zueinander. Zusätzlich werden die absoluten x- und y-Koordinaten des Handgelenks als zwei weitere Merkmale übernommen. Für jede Hand entstehen somit 44 Werte:

- 42 handgelenkrelative Koordinaten der 21 Landmarks
- zwei absolute Koordinaten des Handgelenks

Die normierten Koordinaten werden zunächst in vorzeichenbehaftete 16-Bit-Ganzzahlen überführt. Der normierte Wertebereich wird dafür mit dem Maximalwert 32767 skaliert. Anschließend werden die insgesamt 88 Zwischenwerte entsprechend den Quantisierungsparametern des Klassifikationsmodells in 8-Bit-Werte umgerechnet:

$$q = \frac{v}{32767 \cdot s} + z$$

Dabei bezeichnet v das 16-Bit-Merkmal, $s = 0,007842$ den Quantisierungsfaktor und $z = 127$ den Nullpunkt. Das Ergebnis wird auf den gültigen Bereich von 0 bis 255 begrenzt und als `uint8_t` in den Eingabepuffer des Klassifikationsmodells geschrieben.

7.8.6. Nachverarbeitung des Klassifikationsmodells

Das Klassifikationsmodell gibt für jede unterstützte Zeichenklasse einen quantisierten Ausgabewert zurück. Dieser beschreibt die relative Bewertung der jeweiligen Klasse durch das Modell. Da die Werte als `uint8_t` vorliegen, können sie ohne vorherige Dequantisierung miteinander verglichen werden. Zur Bestimmung des Klassifikationsergebnisses werden sämtliche Ausgabewerte durchlaufen. Der Index des größten Wertes wird als vorhergesagte Klasse ausgewählt. Nach Abschluss der Suche wird der ermittelte Klassenindex über das Array `ai_labels` dem entsprechenden Fingeralphabetzeichen zugeordnet. Das zurückgegebene Ergebnis enthält somit den Klassenindex, den quantisierten Ausgabewert und die zugehörige Klassenbezeichnung.

7.9. Visualisierung und Benutzerausgabe

Für die Bedienbarkeit und Feedback für von dem Nutzer unternommene Handlungen sind visuelle Indikatoren auf dem Display notwendig. Das Projekt erhält zum einen Darstellungen der Erkennungsergebnisse und die Kontrolle über diese Ergebnisse mittels einer Benutzeroberfläche.

7.9.1. Darstellung der Erkennungsergebnisse

Für die Entwicklung mussten die Inferenzergebnisse in einer simpel verständlichen repräsentation dargestellt werden. Ihre Sichtbarkeit sowie Aktivität kann mittels der UI gesteuert.

7.9.1.1. Handdedektion

Das Palm-Detektionsmodell erzeugt eine einzelne Bounding-Box um die erkannte Hand. Dieser wird als rotes rotiertes Rechteck auf dem Display dargestellt. Die vier Eckpunkte der ROI (Region of Interest) werden aus den Normalisierten-Koordinaten im 192×192 Inferenzraum in den 800×480 Displayraum transformiert. Dabei werden die unterschiedlichen Crop-Pipelines der Kamera (Sensorkoordinaten vs. Displaykoordinaten) berücksichtigt. Die ROI wird um einen Faktor von 2,6 vergrößert und um $-0,5$ in y-Richtung verschoben, um die Gesamtheit der Hand einzuschließen. Die Rotation wird aus den Handgelenkpunkten (Keypoint 0 und 2; 2.5) berechnet. Die Darstellung erfolgt über vier Linien, die die Eckpunkte des Rechtecks in Rotationsrichtung verbinden. Jede Linie wird mit dem Bresenham-Algorithmus gerendert und die Eckpunkte werden auf die Displaygrenzen begrenzt, um Überläufe zu verhindern. Im reinen Palm-Modus (`ai_mode == 0`) wird ausschließlich die ROI dargestellt. Im Tracking-Modus wird die ROI zusätzlich zu den Handlandmarks gezeichnet. Um Flackern zu vermeiden, wird die ROI bei jedem Frame-Interrupt (im `DCMIPP_PIPE_FrameEventCallback`) in den nächsten Display-Buffer kopiert und erneut gezeichnet, falls seit dem letzten gültigen Inferenzergebnis kein neues vorliegt.

7.9.1.2. Handlandmarks

Das Hand-Landmark-Modell erzeugt 21 Landmarkpunkte, die als Skelettstruktur auf dem Display dargestellt werden. Die Darstellung umfasst zwei Komponenten: Verbindungslinien zwischen den Gelenkpunkten und kreisförmige Marker an den Gelenkpositionen. Die 21 Landmarkpunkte werden aus dem normalisierten Koordinatenraum $[0, 1]$ in die Pixelkoordinaten des 800×480 Displays transformiert. Vor der Darstellung werden die Punkte einer Geschwindigkeitsvorhersage und einer exponentiellen Glättung unterzogen: Es wird eine lineare Extrapolation mit einer Vorhersagezeit von 25 ms durchgeführt, wobei die Geschwindigkeit auf $\pm 0,08$ pro Frame begrenzt wird. Anschließend wird ein exponentieller Mittelwert mit $\alpha = 0,75$ angewendet, um instabile Schwankungen zu dämpfen.

Die Verbindungslinien folgen einer festen Knochenstruktur, die 23 Verbindungen umfasst: Für jeden Finger wird die Kette vom Handgelenk über die Mittelhand- und Fingerknochen bis zur Fingerspitze dargestellt, ergänzt durch Verbindungen zwischen den Mittelhandgelenken. Jede Linie wird mit dem Bresenham-Algorithmus gerendert. Linien, deren Länge den Schwellwert von 180 Pixeln überschreiten, werden als unplausibel verworfen und nicht gezeichnet [2.5].

Die 21 Gelenkpositionen werden als Kreise mit einem Radius von 4 Pixeln dargestellt. Die Farben sind wie folgt definiert: Verbindungslinien in Grün, Landmark-Marker in Blau. Die Sichtbarkeit wird über den UI-Toggle „Hand“ gesteuert.

Um Flackern zu vermeiden, werden die geglätteten Punkte in ISR-sichere Speicherbereiche (`_isr_landmark_points`) kopiert und bei jedem Frame-Interrupt in den jeweils nächsten Display-Buffer neu gezeichnet, solange kein neues Inferenzergebnis vorliegt.

7.9.1.3. Fingeralphabetmodell

Das Fingeralphabetmodell klassifiziert die erkannte Handpose in eine von 26 Klassen (Buchstaben A–Z sowie *NONE* und *SCH*). Das Modell erhält als Eingabe keine Bilddaten, sondern die quantisierten Handlandmark-Koordinaten. Die Inferenz wird einmal pro Sekunde (`FINGERALPHABET_INTERVAL_MS = 1000 ms`) durchgeführt und nur im Sign-Modus (`ai_mode >= 2`) visuell ausgegeben.

Das Ergebnis wird über eine reine Argmax-Operation extrahiert: Die Klasse mit dem höchsten Konfidenzwert (`uint8`) wird ausgewählt. Das Ergebnis enthält den Klassenindex, den rohen Konfidenzwert und das zugehörige Textlabel.

Das Ergebnis wird als Textbox über das Display und per String über die serielle Schnittstelle (`DEBUG_PRINTF`) ausgegeben, sofern der UI-Toggle „Sign“ aktiviert ist.

7.9.2. Benutzeroberfläche und Bedienung

Eine Benutzeroberfläche musste entworfen werden, um die Darstellungen der Erkennungsergebnisse zu steuern. Bei Aktivität aller Modelle und der Darstellung ihrer Inferenzergebnisse ist das ausgegebene Bild unübersichtlich geworden. Bei der Umsetzung einer permanent sichtbaren UI war diese eher störend, da ein großer Teil des Displays bedeckt wurde. Um dies zu lösen, wurde das UI zu einem Drawer umstrukturiert. Ein Button öffnet und schließt das UI-Overlay.

Die Benutzeroberfläche besteht aus einem Buffer mit zwei Dimensionen. Die Dimension repräsentiert einerseits den geschlossenen, andererseits den geöffneten Zustand des Drawer. Geschlossen besitzt die UI ein einziges Element: einen Button, welcher zum Öffnen genutzt wird. Geöffnet bietet der Drawer mehr Optionen:

- Ein Hamburger-Button zum Schließen des Drawer
- Modusaufstellung der Inferenzpipeline („Palm“, „Hand“, „Sign“)
- Drei Kontrollpanels mit Darstellungs-Toggle und Deckkraftkontrolle (eines für jedes Modell)
- Toggle zur Darstellung der Systemzeit
- Toggle zur Darstellung von Systeminformationen

7.9.2.1. Grafikschicht-Architektur

Die grafische Darstellung basiert auf dem LTDC-Controller (LCD-TFT Display Controller) des STM32N6570, der zwei unabhängige Overlay-Schichten simultaneously auf das Display legt.

Schicht 1 (Hintergrund) wird als RGB888 mit 3 Bytes pro Pixel betrieben und zeigt den Kamerabildfeed (800×480 Pixel). Auf diese Schicht werden die Inferenz-Overlays (ROI-Bounding-Box, Handlandmarks, Touch-Feedback, Systemtext) direkt in den Framebuffer gezeichnet.

Schicht 2 (Vordergrund) verwendet das ARGB4444-Pixelformat (2 Bytes pro Pixel) mit aktiviertem per-Pixel-Alpha. Die UI wird auf dieser Schicht dargestellt und über Alpha-Blending mit dem Kamerabild kombiniert. Das ARGB4444-Format kodiert 4 Bits für Alpha, Rot, Grün und Blau, wobei die halbtransparente Darstellung (0x80 = 50 % Alpha) des Drawer-Elements den Kamerabildfeed durchscheinen lässt.

Die Speicherbelegung umfasst auf Schicht 1 vier rotierende Hintergrundbuffer (je 800×480×3 B = 1,15 MB), die über den DCMIPP-DMA gefüllt werden. Schicht 2 besitzt zwei Vordergrundbuffer (je 800×480×2 B = 750 KB), von denen einer den geöffneten und einer den geschlossenen Zustand des Drawer vorgerendert enthält. Das Umschalten zwischen den Zuständen erfolgt durch ein Setzen der Framebuffer-Adresse im LTDC-Register bei der nächsten vertikalen Austastlücke, wodurch der Übergang flackerfrei erfolgt.

7.9.2.2. Drawer-System

Der Drawer ist eine vertikale Leiste an der linken Displaykante mit einer Breite von 220 Pixeln. Er wird als Struktur (UI_Drawer_TypeDef) verwaltet, die den aktuellen Zustand (geöffnet/geschlossen), die Farbgebung, die Position sowie ein Array von maximal 12 Elementen speichert.

Initialisierung: Bei Systemstart werden die beiden Framebuffer (offen/geschlossen) mit vollständiger Transparenz (0x00000000) gelöscht. Anschließend wird der Drawer im geschlossenen Zustand gerendert (nur Hamburger-Button), danach im geöffneten Zustand (Panel mit allen Elementen). Der im Ausgangszustand sichtbare Buffer ist der geschlossene Zustand.

Öffnen und Schließen: Ein Tippen auf den Hamburger-Button (44×44 Pixel, oben links) toggled den Zustand. Der Wechsel erfolgt durch ein simples Umschalten des Framebuffer-Zeigers im LTDC-Layer-2-Konfigurationsregister (LTDC_Layer_Address_Set), ohne Neurenderung. Der Hamburger-Button (drei horizontale Balken, je 20×3 Pixel) ist in beiden Zuständen sichtbar.

Farbschema: Die Elemente des Drawer verwenden eine halbtransparente Farbgebung, um den Kamerabildfreigabe durchscheinen zu lassen:

- Hintergrund: Dunkelgrau mit 50 % Alpha (0x802B2B2B)

- Elementflächen: Mittelgrau mit 50 % Alpha (0x80383838)
- Akzentfarbe: Grün (0xFF00DD55) für aktive Elemente
- Text: Weiß (0xFFFFFFFF)
- Rahmen: Hellgrau mit 50 % Alpha (0x80777777)

Elementhöhe und Anordnung: Jedes Element besitzt eine Höhe von 46 Pixeln mit einem 4 Pixel großen Abstand zwischen den Elementen. Die y-Position des i -ten Elements berechnet sich als $y_{\text{pos}} + 52 + i \cdot 50$ Pixel.

7.9.2.3. UI-Elemente

Der Drawer enthält vier Elementtypen:

Selektor (Modus): Ein segmentierter Steuerknopf mit drei Segmenten („Palm“, „Hand“, „Sign“). Die Breite jedes Segments ergibt sich aus der nutzbaren Fläche von 200 Pixeln geteilt durch die Segmentanzahl. Das aktive Segment wird in der Akzentfarbe (Grün) dargestellt. Die Touch-Position wird auf den entsprechenden Segmentindex gemappt. Der selektierte Modus steuert den Inferenz-Pipeline-Zustandsautomaten.

Toggle: Ein Schalter mit beschrifteter Fläche und rundem Schieberegister (Radius 7 Pixel). Der Toggle hat zwei Zustände (ein/aus). Die Systemzeit-Toggle fügt oder entfernt dynamisch einen Scheduler-Task, der die aktuelle Systemzeit im oberen rechten Bereich (Position $x = 720$) auf Schicht 1 anzeigt. Der Systeminformationen-Toggle steuert analog die Einblendung der Inferenzzeiten aller drei Modelle.

Composite (Kontrollpanel): Kombiniert ein Beschriftungsfeld, ein Augen-Symbol zur Umschaltung der Sichtbarkeit und einen Inline-Slider zur Einstellung der Deckkraft. Das Augen-Symbol zeigt einen gefüllten Kreis (sichtbar) oder eine horizontale Linie (unsichtbar). Tippen auf das Augen-Symbol toggled die Sichtbarkeit des zugehörigen Modell-Overlays. Der Slider nach dem Augen-Symbol erlaubt eine feinere Einstellung des Wertes.

Label: Ein statisches Textelement ohne Interaktion. Wird für die Versionsanzeige („v1.0.0“) verwendet.

7.9.2.4. Touch-Controller (GT911)

Die Eingabe erfolgt über einen Goodix GT911 kapazitiven Touch-Controller, der über den I2C-Bus (I2C2) mit dem Mikrocontroller kommuniziert.

Konfiguration: Der GT911 wird über einen 248 Byte langen Konfigurationsblock (Register 0x8047 bis 0x80FF) initialisiert. Die Auflösung wird auf 800×480 Pixel gesetzt, um die Displayauflösung abzubilden. Das MSW1-Register wird auf 0x0C gesetzt, wodurch die X- und Y-Achsen invertiert werden, um die physische Display-Orientierung zu korrigieren. Nach dem Schreiben der Konfiguration wird ein Prüfsumme berechnet um die Korrekte Konfiguration zu verifizieren und Konfigurationsfehler zu tracken.

Interruptgestützte Erfassung: Bei Berührung der Displayfläche löst der GT911 einen steigenden Flanken-Interrupt über PQ4 aus. Im Interrupt-Handler (EXTI4_IRQHandler) wird über I2C der Touch-Status gelesen, der die Anzahl der berührenden Finger (0–5) angibt. Bei mindestens einer Berührung werden die 12-Bit-Koordinaten der ersten Berührung aus den Registern gelesen und in statischen Variablen abgelegt. Anschließend wird eine Pending-Flag gesetzt.

Abtastung und Weiterleitung: Ein Scheduler-Task (vTouchTask) läuft alle 20 ms und prüft das Pending-Flag. Bei einer Berührung werden die Koordinaten und der Berührungsstatus ausgelesen und an die Drawer-Logik (UI_Drawer_HandleTouch) übergeben. Zusätzlich wird als visuelles Feedback ein blauer Kreis (Radius 5 Pixel) an der Berührungsposition auf Schicht 1 gezeichnet.

7.9.2.5. Touch-Event-Verarbeitung

Die Touch-Verarbeitung arbeitet in zwei Phasen:

Drückphase: Bei Berührung wird geprüft, ob die Berührung innerhalb des Hamburger-Buttons oder eines geöffneten Drawer-Elements liegt. Die Trefferzone jedes Elements wird über die vorab berechnete y-Position und die konstante Elementhöhe bestimmt. Der Index des getroffenen Elements wird zwischengespeichert.

Loslassphase: Beim Loslassen wird der zuvor gespeicherte Elementtyp ausgewertet:

- *Hamburger-Button:* Der Drawer wird umgeschaltet.
- *Toggle:* Der Wert wird invertiert.
- *Slider:* Die Touch-x-Position wird auf einen Wertebereich von 0–100 gemappt.
- *Selector:* Die Touch-x-Position wird auf den Segmentindex gemappt.
- *Composite:* Zuerst wird geprüft, ob das Augen-Symbol getroffen wurde (Toggle der Sichtbarkeit). Andernfalls wird der Slider aktualisiert.

Nach der Verarbeitung wird das betroffene Element in den geöffneten Framebuffer neu gerendert.

Datenfluss: Die AI-Pipeline liest den Drawer-Zustand direkt aus den Element-Werten aus (`_drawer.items[]`). Die Sichtbarkeits-Flags (`palm_vis`, `hand_vis`, `sign_vis`) werden in ISR-sichere Variablen gespiegelt, damit der Frame-Interrupt-Callback sie im laufenden Frame-Zyklus verwenden kann, ohne einen Zugriffskonflikt mit dem Scheduler-Task zu riskieren.

8. Evaluation und Validierung

In diesem Kapitel wird das entwickelte Gesamtsystem anhand der in Kapitel 5 definierten Anforderungen und Abnahmekriterien bewertet. Ziel ist es, sowohl die Erkennungsleistung der KI-Verarbeitungskette als auch die technische Leistungsfähigkeit des eingebetteten Systems zu untersuchen. Dazu werden die Erkennungsgenauigkeit, das Verhalten unter unterschiedlichen Umgebungsbedingungen, die Verarbeitungsgeschwindigkeit sowie der Speicher- und Ressourcenverbrauch betrachtet.

Die Evaluation umfasst sowohl das Klassifikationsmodell als auch den integrierten Prototyp. Die Bewertung des Modells erfolgt anhand zuvor nicht für das Training verwendeter Testdaten. Ergänzend wird das Gesamtsystem unter festgelegten realen Einsatzbedingungen untersucht. Dadurch kann zwischen der isolierten Modellleistung und der Erkennungsleistung der vollständig integrierten Kamera- und KI-Verarbeitungskette unterschieden werden. Die ermittelten Ergebnisse werden abschließend den festgelegten Anforderungen und Abnahmekriterien gegenübergestellt. Auf dieser Grundlage wird beurteilt, welche Anforderungen vollständig, teilweise oder nicht erfüllt wurden.

8.1. Erkennungsgenauigkeit

8.2. Konfusionsmatrix und klassenbezogene Ergebnisse

Tabelle 8.1.: Trainingsergebnisse des Klassifizierungsmodells

Metrik	Wert
Trainingsgenauigkeit	98,34 %
Validierungsgenauigkeit	99,21 %
Finaler Trainingsverlust	0,0754
Finaler Validierungsverlust	0,0319

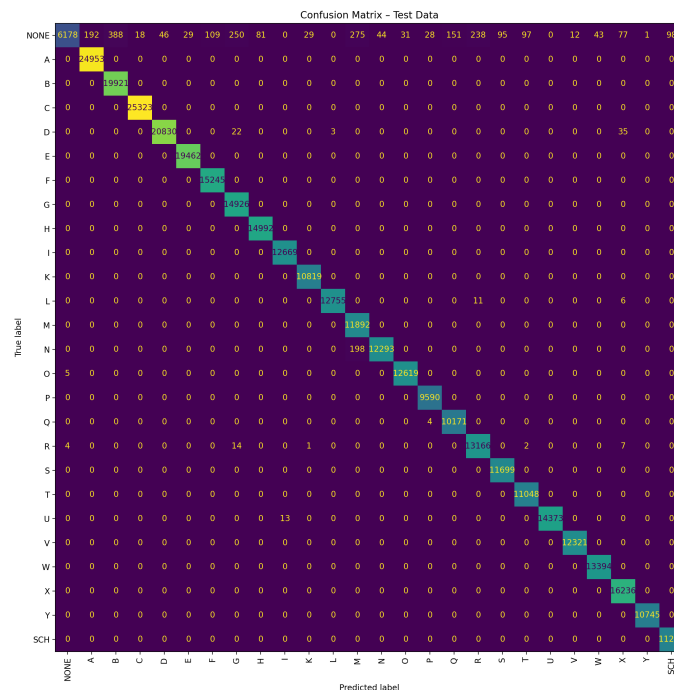


Abbildung 8.1.: Konfusionsmatrix

Das Modell zeigt keine Anzeichen von Overfitting: Der Validierungsverlust sinkt kontinuierlich über alle 20 Epochen.

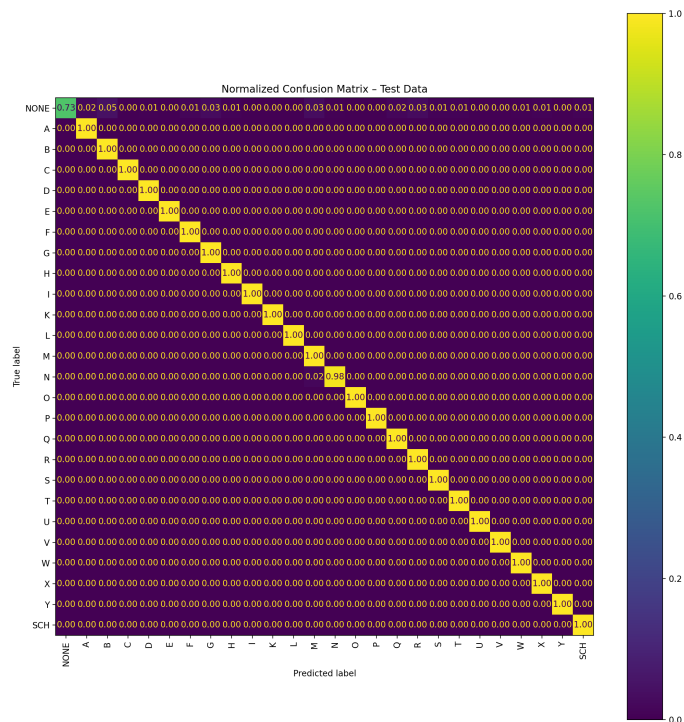


Abbildung 8.2.: Normalisierte Konfusionsmatrix

8.3. Versuchsaufbau und Testbedingungen

Das System wird während der Versuche in einer festen Position aufgestellt. Die Handzeichen werden einzeln innerhalb des Erfassungsbereichs der Kamera dargestellt. Um die Versuche Vergleichbar zu gestalten, werden der Abstand zur Kamera, die Beleuchtung, der Hintergrund und die Anzahl der Wiederholungen dokumentiert. Soweit ein Parameter nicht gezielt untersucht wird, bleiben die Bedingungen während der jeweiligen Versuchsreihe unverändert.

Tabelle 8.2.: Grundlegende Bedingungen der Systemevaluation

Parameter	Festlegung
Abstand zur Kamera	50 cm
Hintergrund	leer, weiße Wand
Beleuchtung	Lichtquelle von oben, außerhalb des Bildes
Unterstützte Zeichenklassen	A-Y ohne J, SCH
Versuchspersonen	vier
Wiederholungen je Zeichen	10

8.4. Feldversuch

8.5. Einfluss von Beleuchtung und Hintergrund

8.6. Inferenzlatenz und Ende-zu-Ende-Latenz

8.7. Bildrate und Datendurchsatz

8.8. Speicher- und Ressourcenverbrauch

8.9. Abgleich mit den definierten Anforderungen

9. Diskussion

9.1. Versuchsaufbau und Testbedingungen

9.2. Erkennungsgenauigkeit

9.3. Konfusionsmatrix und klassenbezogene Ergebnisse

9.4. Einfluss von Beleuchtung und Hintergrund

9.5. Inferenzlatenz und Ende-zu-Ende-Latenz

9.6. Bildrate und Datendurchsatz

9.7. Speicher- und Ressourcenverbrauch

9.8. Abgleich mit den definierten Anforderungen

10. Fazit und Ausblick

10.1. Zusammenfassung und Zielerreichung

10.2. Ausblick

11. Literaturverzeichnis

- [1] AktionMensch e.V., *aktion-mensch — fingeralphabet*, <https://www.aktion-mensch.de/ueber-uns/das-treibt-uns-an/was-ist-inklusion/deutsche-gebaerdensprache>. besucht am 16. Juli 2026.
- [2] Deutscher Gehörlosen-Bund e.V., *Deutsche Gebärdensprache e. V. - Beitrag über die deutsche Gebärdensprache*. besucht am 16. Juli 2026.
- [3] M. Huenerfauth, E. Gale, B. Penly, S. Pillutla, M. Willard und D. Hariharan, „Evaluation of Language Feedback Methods for Student Videos of American Sign Language,“ *ACM Transactions on Accessible Computing*, Jg. 10, Nr. 1, S. 1–30, Apr. 2017, ISSN: 1936-7228, 1936-7236. DOI: 10.1145/3046788 besucht am 16. Juli 2026.
- [4] STMicroelectronics, *STM32N6570-DK — Productpage*, <https://www.st.com/en/evaluation-tools/stm32n6570-dk.html>. besucht am 16. Juli 2026.
- [5] Y. Abadade, A. Temouden, H. Bamoumen, N. Benamar, Y. Chtouki und A. S. Hafid, „A Comprehensive Survey on TinyML,“ *IEEE Access*, Jg. 11, S. 96 892–96 922, 2023, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2023.3294111 besucht am 16. Juli 2026.
- [6] G. Benitez-Garcia, J. Olivares-Mercado, G. Sanchez-Perez und K. Yanai, „IPN Hand: A Video Dataset and Benchmark for Real-Time Continuous Hand Gesture Recognition,“ in *2020 25th International Conference on Pattern Recognition (ICPR)*, Milan, Italy: IEEE, Jan. 2021, S. 4340–4347, ISBN: 978-1-7281-8808-9. DOI: 10.1109/ICPR48806.2021.9412317 besucht am 16. Juli 2026.
- [7] B. Jacob u. a., „Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,“ in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, Salt Lake City, UT: IEEE, Juni 2018, S. 2704–2713, ISBN: 978-1-5386-6420-9. DOI: 10.1109/CVPR.2018.00286 besucht am 16. Juli 2026.
- [8] P. Marwedel, „Eingebettete Systeme,“ in *Eingebettete Systeme: Grundlagen Eingebetteter Systeme in Cyber-Physikalischen Systemen*, P. Marwedel, Hrsg., Wiesbaden: Springer Fachmedien, 2021, S. 1–30, ISBN: 978-3-658-33437-6. DOI: 10.1007/978-3-658-33437-6_1 besucht am 16. Juli 2026.
- [9] M. Wolf, *Computers as Components: Principles of Embedded Computing System Design*. Elsevier, Juni 2012, ISBN: 978-0-12-388442-8.
- [10] *Mikrocontroller*, <https://www.elektronik-kompodium.de/sites/com/1907171.htm>. besucht am 16. Juli 2026.
- [11] J. Yiu, [11] *The Definitive Guide to ARM® Cortex®-M3 and Cortex®-M4 Processors*. Elsevier, 2013. besucht am 16. Juli 2026.
- [12] STMicroelectronics, *STM32N6x5/x7xx Arm-based 32-bit MCUs Reference Manual RM0486*, RM0486, Reference Manual, Dez. 2025.
- [13] *STM32N6 Example Projects & Tips for Creating New Projects*, https://stedgeai-dc.st.com/assets/embedded-docs/stneuralart_stm32n6_projects.html?utm_source=chatgpt.com. besucht am 24. Juli 2026.
- [14] I. Gbati, „Bare-Metal Embedded C Programming: Develop high-performance embedded systems with C for Arm microcontrollers,“ eng, in *Bare-Metal Embedded C Programming*, Birmingham, United Kingdom: Packt Publishing Limited, 2024, S. 1–438, ISBN: 9781835463567.
- [15] T. Hastie, R. Tibshirani und J. Friedman, *The Elements of Statistical Learning*, 2009. besucht am 17. Juli 2026.
- [16] *Praxiseinstieg Machine Learning Mit Scikit-Learn, Keras Und TensorFlow - 11727363*, https://content-select.com/media/moz_viewer/64d9ca27-1b4c-4537-b870-43e7ac1b0007/language:de. besucht am 24. Juli 2026.
- [17] K. G. Kim, „Book Review: Deep Learning,“ *Healthcare Informatics Research*, Jg. 22, Nr. 4, S. 351, 2016, ISSN: 2093-3681, 2093-369X. DOI: 10.4258/hir.2016.22.4.351 besucht am 16. Juli 2026.
- [18] S. Raschka, Y. (Liu und V. Mirjalili, *Machine Learning with PyTorch and Scikit-Learn: Develop Machine Learning and Deep Learning Models with Python*. Packt Publishing Ltd, Feb. 2022, ISBN: 978-1-80181-638-0.

-
- [19] C. Lugaresi u. a., „MediaPipe: A Framework for Perceiving and Processing Reality,“
 - [20] Y. Zhang und G. Notni, „3D Geometric Features Based Real-Time American Sign Language Recognition Using PointNet and MLP with MediaPipe Hand Skeleton Detection,“ *Measurement: Sensors*, Proceedings of the XXIV IMEKO World Congress, Jg. 38, S. 101 697, Mai 2025, ISSN: 2665-9174. DOI: 10.1016/j.measen.2024.101697 besucht am 16. Juli 2026.
 - [21] *Post-training quantization — TensorFlow Model Optimization*, https://www.tensorflow.org/model_optimization/g besucht am 16. Juli 2026.
 - [22] S. S. Smalley Ian, *What Is Edge AI? — IBM*, <https://www.ibm.com/think/topics/edge-ai>, Aug. 2023. besucht am 20. Juli 2026.
 - [23] *ST Neural-ART Compiler Primer*, https://stedgeai-dc.st.com/assets/embedded-docs/stneuralart_neural_art.com besucht am 24. Juli 2026.
 - [24] H. Snyder, „Literature Review as a Research Methodology: An Overview and Guidelines,“ *Journal of Business Research*, Jg. 104, S. 333–339, Nov. 2019, ISSN: 0148-2963. DOI: 10.1016/j.jbusres.2019.07.039 besucht am 20. Juli 2026.
 - [25] P. López, J. Mabe, L. Etxeberria und E. Gorritxategi, „Iterative Prototyping Methodology for the Development of Innovative and Dependable Complex Embedded Systems Through SPC&KPI Techniques,“ in *Product-Focused Software Process Improvement*, Ser. Lecture Notes in Computer Science, Bd. 11271, Switzerland: Springer International Publishing AG, 2018, S. 65–80, ISBN: 978-3-030-03672-0. DOI: 10.1007/978-3-030-03673-7_5
 - [26] A. Manifesto, *Manifesto for Agile Software Development*, <https://agilemanifesto.org/>, 2001. besucht am 20. Juli 2026.
 - [27] R. Preißel, *Git dezentrale Versionsverwaltung im Team: Grundlagen und Workflows*, 5., aktualisierte und erweiterte Auflage. Heidelberg: dpunkt.verlag, 2019, ISBN: 978-3-86490-649-7.
 - [28] ISTQB, „ISTQB Certified Tester - Foundation Level Syllabus v4.0,“ 2024.
 - [29] S. Kleuker, „Anforderungsanalyse,“ in *Grundkurs Software-Engineering mit UML*, Wiesbaden: Springer Fachmedien Wiesbaden, 2025, S. 53–92, ISBN: 978-3-658-46533-9. DOI: 10.1007/978-3-658-46534-6_4
 - [30] *CMSIS: Introduction*, https://arm-software.github.io/CMSIS_6/latest/General/index.html. besucht am 24. Juli 2026.
 - [31] H. Washizaki, „Waseda University, IEEE Computer Society 2025 President,“ 2025.
 - [32] *Resource Isolation Framework overview - stm32mpu*, https://wiki.st.com/stm32mpu/wiki/Resource_Isolation.Fr besucht am 24. Juli 2026.
 - [33] F. Zhang u. a., *MediaPipe Hands: On-device Real-time Hand Tracking*, Juni 2020. DOI: 10.48550/arXiv.2006.10214 arXiv: 2006.10214 [cs.CV]. besucht am 23. Juli 2026.
 - [34] S. Sharma, S. Sharma und A. Athaiya, „ACTIVATION FUNCTIONS IN NEURAL NETWORKS,“ *International Journal of Engineering Applied Sciences and Technology*, Jg. 04, Nr. 12, S. 310–316, Mai 2020, ISSN: 24552143. DOI: 10.33564/IJEAST.2020.v04i12.054 besucht am 23. Juli 2026.
 - [35] A. Athavale, E. Bartocci, M. Christakis, M. Maffei, D. Nickovic und G. Weissenbacher, „Verifying Global Two-Safety Properties in Neural Networks with Confidence,“ in *Computer Aided Verification*, A. Gurfinkel und V. Ganesh, Hrsg., Cham: Springer Nature Switzerland, 2024, S. 329–351, ISBN: 978-3-031-65630-9. DOI: 10.1007/978-3-031-65630-9_17
 - [36] N. Bodla, B. Singh, R. Chellappa und L. S. Davis, *Soft-NMS – Improving Object Detection With One Line of Code*, Aug. 2017. DOI: 10.48550/arXiv.1704.04503 arXiv: 1704.04503 [cs.CV]. besucht am 23. Juli 2026.
 - [37] H. Rezatofighi, N. Tsoi, J. Gwak, A. Sadeghian, I. Reid und S. Savarese, *Generalized Intersection over Union: A Metric and A Loss for Bounding Box Regression*, Apr. 2019. DOI: 10.48550/arXiv.1902.09630 arXiv: 1902.09630 [cs.CV]. besucht am 23. Juli 2026.
 - [38] Google. Adresse: %7Bhttps://github.com/google-ai-edge/mediapipe/blob/master/mediapipe/modules/hand_landmark/palm_detection_detection_to_roi.pbtxt%7D

A. Anhang